BZ_AutoDrive — Admin manual

> NPC vehicle-driving framework for DayZ. Didactic edition for admins/modders. **Open source (MIT):** the code, this manual and the AI knowledge pack are public — download it, use it on your server, fork it, extend it and share your routes. *Manual 1.3 — the admin's practical guide.*

Contents

1. What is BZ_AutoDrive?
2. The idea, in one sentence
3. The usage model: download it, adapt it, it's your mod
4. Requirements
5. Quick start — your first route
6. The wizard in detail 6B. The 4 files the wizard generates 6C. The envelope — what the framework learns from your takes
7. How to record well
8. The permanent bus service
9. How the NPC drives (it follows your line and your speed) 9B. DayZ 1.30 — what changed in vehicles, and motorbikes *(experimental)*
10. Events and sequences
11. Missions — integration with Quests
12. Controls (keys)
13. The interface (UI)
14. Audio
15. If something doesn't work
16. Ideas
17. Scope and limits
18. Open frontier

- Appendix A — Complete route config reference
- Appendix B — For developers (extending with code)

---

1. What is BZ_AutoDrive?

BZ_AutoDrive makes an **NPC drive a DayZ vehicle autonomously**, following a route that **you recorded by driving**. It's not a teleport or "fake" movement: the NPC really **accelerates, brakes and steers**, using the game's real physics — the car behaves just like it does when you drive it.

What's it good for? For anything that needs an AI-driven vehicle:

- A **line bus** that runs along the coast stopping in every town.

- A **taxi** that takes players from one point to another.
- A **reinforcement convoy** that arrives at a mission zone and deploys bots.
- A motorized **patrol** that circles around an area.

> *Example:* you want a bus that links three towns. You drive it **just once** (you record the route), and from then on an NPC repeats it by itself, as many times as you want, without you being there.

---

2. The idea, in one sentence

> **You drive the route once; the framework learns your line and reads what the vehicle is like; then an NPC drives it.**

The powerful part is the "reads what the vehicle is like": BZ_AutoDrive **reads the vehicle's config** (its steering, its gears, its engine) and drives according to that. That's why **the framework drives any vehicle with no per-car setup**: you don't have to configure anything per car to make it run.

And there is a second layer, which is what's new in 1.2: besides **reading** what the config declares, the framework **measures** how that vehicle really behaves — how hard it brakes, how much it turns at each steering angle, whether it creeps on its own at idle. It learns this **from your own takes**, without you doing anything, and stores it in the *envelope* of that class (§6C). When the declared and the measured values don't match, **the measured one wins**.

> *Mind the recording:* a recording captures the run of **its** vehicle (its *fingerprint*, its measured braking, its gear) — **it belongs to that car**. Want the **same route** on another vehicle? **Record it with that vehicle.** It's not a limitation: it's the way the framework works best, because from each take it learns how **that** class behaves (§6C) — how hard it brakes, how much it turns. A borrowed take brings the measurements of the wrong car.

(Why it matters: DayZ has no "drive to here" for AI cars — it only gives you the steering wheel and the pedals. BZ_AutoDrive is the missing driver.)

---

3. The usage model: download it, adapt it, it's your mod

BZ_AutoDrive is a **starting point**, not a closed product. The arc for any admin is always the same:

1. **You download** the framework (open source).
2. **You adapt it to your server:** you record *your* routes and configure *your* scenarios. The **driving engine comes already integrated** — you don't program how the NPC drives, that's already solved.
3. **You're left with your mod**, with the framework inside + your content, running your own thing.

What you add depends on what you want it for:

- **Bus / taxi service** → you record the route(s) and leave them running (§8). No code to touch.

- **Mission system** → you add the Quests + the convoy/escort routes (§11). The framework drives your mission vehicles.
- **Something custom** → you extend it with code (Appendix B) or build your own mod on top of it as a dependency (B.6).

> The point of open source: **everyone downloads the same engine and makes it their own.** One builds a coastal bus, another an urban taxi, another a mission campaign — without anyone having to reinvent the driving.

> *If you repackage it as your own addon* (renaming it, your own Workshop): you rebuild with your own key and copy the `.bikey` to `keys/`. The MIT license allows it — just keep the credits.

---

4. Requirements

- **DayZ server** with the **BZ_AutoDrive** mod loaded + its `.bikey` in `keys/`.
- **DayZ-Expansion, via the Bundle** (`@DayZ-Expansion-Bundle`) — *essential*. It brings the three parts the mod needs: **AI (eAI)**, which provides the "body" of the NPC driver (the framework **drives**, eAI provides the body); **Vehicles**; and **Quests**, which the mod **requires in order to load** even if you don't run missions. ⚠ **Do not add** `@DayZ-Expansion-AI` **on its own**: if you already have the Bundle, those are the same classes in a different version and the server won't start.
- **The wizard** (`route_wizard.ps1`) runs on **your PC** (outside the game), not on the server. You launch it by double-clicking `tools\Wizard.bat` — no need to open PowerShell or type anything.
- **If you add a vehicle pack:** some packs need **another base mod** (a *core*) that the Workshop **doesn't download on its own**, and without it **the server crashes on boot**. After adding a pack, search the RPT for `requires addon` : if it shows up, install the mod it names and put it **before** the vehicle in `-mod=`. And remember that the **client** loads the mods from its launcher's list (DZSA), not from the server's `start.bat` : if you don't tick it there too, you won't see the new vehicles.
- **Python: already included.** The wizard's converter (`frame_to_route.py`), the tool that builds the vehicle profile (`envelope_emit.py`) and the v1 import are Python, and the mod **ships its own Python** in `tools\python\` : you don't have to install anything. *(If that folder is missing, the wizard uses your PC's Python if it finds it in the PATH, and if there is none, it tells you when it opens.)*
- **What runs by default: you don't have to turn anything on.** The 1.2 driving brain, the tape and the new steering law come **on**. If for some reason you want the previous behavior on your server, you turn it off with a file in `profiles\BZ_AutoDrive\` **with any text inside**: `CEREBRO_11.txt` goes back to the 1.1 brain, `BZ12_SIN_CINTA.txt` runs tapeless, `BZ12_SIN_FF_PRIMARIO.txt` goes back to the previous steering law. **An empty file does nothing.** A route that has no `_cinta.csv` (for example, one imported from v1) runs tapeless and says so in the log. The client filter (§13.4) also comes on.

---

5. Quick start — your first route

Let's build a route from start to finish. *Example we'll follow: a bus that goes from one town's terminal to another.*

Step 0 — Check your keys *(do this BEFORE anything else)*

The framework uses **only 3 keys**, already assigned by default. Go to **Options** → **Controls** → **the "BZ AutoDrive" category** and check them:

Action (in the menu)	Default
Open Control Panel	`HOME`
Record (start/stop)	`NUMPAD 5`
Mark Event / Stop	`NUMPAD 4`

> ⚠ **First thing: make sure they don't clash with your keys.** If you already have `HOME` or `NUMPAD 4/5` assigned to **another mod** (or to a DayZ action), they'll **override each other** and something won't respond — and **DayZ doesn't warn you about the conflict**. So, before recording: open that category, check that the 3 don't collide with yours, and **rebind** whichever one needs it (click the key → press the new one). It takes a minute and saves you the classic "*I record and nothing happens*" or "*I press HOME and the panel doesn't open*".

*(These keys only take effect for the **admin** —§13.1— even though all players SEE them in their Controls menu. Full detail on the controls in §12.)*

Step 1 — Record the route (you drive it)

Get into the vehicle and drive the route **the way you want the NPC to do it**.

- **NUMPAD 5** → start recording. Drive calmly, at the speed you want it to go.
- **NUMPAD 4** → tap it at each **stop or important point** (the terminal, a corner where something happens). It marks that point so you can later attach events to it.
- **NUMPAD 5** again → stop recording.

> You're left with a file containing your route: your **line** + your **speed** + **where you stop**. That's the raw material.

> 🔑 **The most important thing that happens when recording (and that you don't see):** that same NUMPAD 5, besides your route, **takes a complete snapshot of the vehicle** — its *fingerprint*. It ends up in a `header_*.txt` next to your recording, and it brings **all the car's data, by itself**:
>
> - the **classname**, the **wheelbase** (distance between axles) and the **steering angle** → from which the **R_min** is computed (the smallest radius it can turn),
> - the number of **gears**, the **mass**, the engine **RPM**,
> - and the **real list of its parts** (wheels, doors, battery, radiator, spark plug...).
>
> That's why **you don't configure anything about the vehicle by hand**: the wizard and the converter read that fingerprint and build the route to fit it — they compute how much it can turn, they put the correct attachments on it (*they're not guessed, they're the ones the vehicle had*) and they drive it according to *its* physics. It's the heart of "config as driving manual".
>
> *Real example:* we recorded a UAZ-452 and from **a single NUMPAD 5** everything came out: `UAZ_452`, wheelbase 2.53 m, R_min 3.9 m, 6 gears, mass 2860 kg, + its 4 wheels (including the spare), 6 doors

> and battery/radiator/spark plug. Zero configuration, zero guessing.

Step 2 — Convert the recording (the wizard)

The wizard is a **converter**: it takes your raw recording and **converts** it into a ready route, **deployed** to the Player. It stores your driving **by position** —what steering, throttle, brake and gear you had at each point of the road: the *tape*— and updates the vehicle's **measured profile** (§6C). The NPC starts from that and corrects it according to what the vehicle can do. *It doesn't calibrate anything by hand: there are no per-route settings.*

Launch the wizard by **double-clicking** `tools\Wizard.bat` (no need to open PowerShell or type anything), pick **[1] Convert**, choose your recording and give it a **name** — and it leaves the route ready for you.

> **The first time** you open the wizard it asks you for your **paths** — your server's routes folder (`RoutesDir`) and where your recordings are. You set them **once**, the wizard remembers them, and you can change them whenever you want with **[6] Configure paths** (detail in §6).

> *Why this step?* The raw recording is a record **by time**. The NPC needs your driving **by position** (what you were doing at each point of the road, even if it goes a little slower or faster) and the vehicle's measured profile. The wizard builds both. *Where's my recording?* It lives on the **client**, the PC where you drove: `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\` . The wizard finds it there on its own. The converted route is **deployed** to your server's routes folder (`RoutesDir`). All the detail —the "trio" of files and the folders— in **§6**.

Step 3 — Deploy (the wizard already does it)

On converting, the wizard builds the route, updates the vehicle profile and leaves everything **deployed and ready in the Player** (hot load, **without restarting the server**). You don't edit any file by hand. If you configured a second server in **[6] Configure paths**, the wizard also copies the route there.

Step 4 — Test it (let the NPC drive it)

Open the **Player** (the admin UI, `HOME` key) → pick your route from the list → press **LOAD & SPAWN**: the NPC appears and starts driving. **Without restarting the server.**

> *Example:* you pick "Town bus" from the list, press LOAD & SPAWN and right away the bus leaves the terminal, driven by the NPC, stopping where you marked.

That's the whole cycle: `record → wizard → deploy → play` . The rest of the manual is about getting more out of each part.

---

6. The wizard in detail — your central tool

The **wizard** (`tools\route_wizard.ps1`) turns your raw recording into a ready route, deployed to the Player. It's an interactive menu (TUI) — it runs on your PC, outside the game. *(It's a **converter**: it doesn't calibrate or*

run you through a questionnaire — the driving comes from reading the vehicle's config.)

> **Launch it by double-clicking** `tools\Wizard.bat` — it opens the wizard directly, without opening PowerShell or typing commands. (The `.bat` uses `-ExecutionPolicy Bypass`, which **only affects that run** — it doesn't change your system's policy.)

> **Principle: everything goes through the wizard.** The wizard runs the sub-tools (`frame_to_route.py`, `envelope_emit.py`, `frame_merge.py`) **for you**, in order. You do **not** edit the JSON by hand or run the scripts separately — the wizard is the only entry point.

⚠ Where each take is stored (read this)

When you record (NUMPAD 5), the file **doesn't end up where you'd intuitively look for it**. The golden rule:

- The **human recording** (`path_*.csv` / `frame_*.csv`, the one you make with NUMPAD 5) lands on the **client** (the PC where you drove): `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\`.
- **Boris's runs** (`ai_run_*.csv` and `boris_native_*.csv`) are produced with the **Player's checks** (§13, **no longer with keys**) and land on the **server that ran them**, not on the client. *They're measurements for diagnosing/comparing, not takes to convert (§15.1–16.2).*

The wizard looks for your recordings on the **client** automatically, so it normally finds them on its own. If you run the server on another PC, you can also point it at that folder:

Place	Folder
Client (where you drive)	`%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\`
Server (if you recorded there)	`<server>\profiles\BZ_AutoDrive_PathLogger\`

> If you recorded and "it doesn't show up", check which machine you were on — it lands on that PC's client.

Two more files worth knowing:

- Next to each recording there's a `header_*.txt` = the **vehicle fingerprint** (steering, gears, wheelbase). The wizard **reads** it to build the route to fit that car (it doesn't calibrate it: it reads it).
- The **finished routes** (the four files in §6) live in your server's routes folder (`RoutesDir` — see `[6]`).

The wizard menu

You launch the wizard (double-click `Wizard.bat`) and the menu appears:

> **[1] Convert & binarize · [2] Import a BrigadaZ Transport v1 route · [3] Merge recording segments (continue a recording) · [B] Binarize routes · [6] Configure paths · [Q] Quit**

[1] Convert & binarize — from recording to ready route

You take a raw recording (`frame_*.csv` + its paired `header_*.txt`) and the wizard turns it into a **route deployed and ready in the Player**, in one step. The only thing it asks you for is a **NAME** → the route comes out as `BZBusRoute_<name>` and shows up under that name in the Player. In the recordings list, each one carries the tag of the folder where it was found: `[Cl]` the client's, `[A]` your server's recordings folder (`BZ_AutoDrive_PathLogger` , next to the routes folder) and `[B]` the second server's, if you set one up.

What it generates and where it goes. From one recording come **four route files**, all in your **routes folder** (`RoutesDir` — see `[6]`):

File	What it is	Do you touch it?
`BZBusRoute_<name>_hdr.json`	the **header**: vehicle, vehicle measurements, driving config and the list of **events** from your NUMPAD 4 presses	**yes** — this is the file you edit
`BZBusRoute_<name>_wp.csv`	the **waypoints**, one per point of the route (around 900 on a 10-minute route)	no
`BZBusRoute_<name>_cinta.csv`	the **tape**: your complete driving, finely sampled along the route — position, heading, speed, steering, throttle, brake and gear— about 12 times denser than the waypoints. This is where the fidelity comes from: the NPC doesn't interpolate between waypoints, it replays this	no
`BZBusRoute_<name>_cinta.bin`	the **binary tape**: the same tape in the format the server reads fastest. The wizard generates it when converting; without it the route works the same, from the CSV (§6B)	no
`BZBusRoute_<name>.json`	the **master**: header + waypoints in a single file	no, it's a backup

> **The pair that takes priority is `_hdr.json` + `_wp.csv` .** If both exist, the server loads that pair and **the full `.json` goes unused**. That's why the events go in the `_hdr.json` .

And a fifth file, which belongs to the vehicle, not to the route:

File	What it is
`envelope_<vehicle class>.csv`	the **measured profile of that vehicle**: how far it moves on its own with no throttle, what speed it holds at each curvature, how it brakes. It's built from your takes and **all the routes of that vehicle share it**

Where it goes. To the **same folder as the routes** (`RoutesDir` , your `…\profiles\BZ_AutoDrive\`) — the wizard writes it there together with the four files. The name carries **the vehicle's class, not the route's name**: `envelope_BZ_ChernoBus_02.csv` , not `envelope_MY_ROUTE.csv` . That's why there is **one per vehicle** and not one per route: if you record three routes with the same bus, all three feed and use the same profile.

The profile is measured automatically when you end the take the way §7.1 says, and **the wizard doesn't overwrite it** on later takes: each new take **adds** what it contributed. To rebuild it from scratch on purpose: `tools\python\python.exe tools\envelope_emit.py "<take.csv>" --out "<RoutesDir>" --pisar` (or answer

[R] regenerate when the wizard asks you: it only asks if the vehicle changed since its profile was measured).

> **If you move a route to another server:** a route converted with this version **carries inside it** its vehicle's envelope (in the `_hdr.json`), so the four files are enough. A route converted earlier also needs its vehicle's envelope; without it, the framework starts without the measured data and uses the factory values until it gathers new takes. Detail of what it stores and how it fills up: **§6C**.

> **While the NPC drives**, the server also writes two things to `profiles\BZ_AutoDrive\`: `BZ12_LOG_<route>_<time>.csv`, with the detail of that run, and `AR_log.csv`, with one row for each time the AutoRecovery had to step in (§9.6). You can delete them whenever you want.

> It tells you the route is **deployed** (hot load, **no restart**). If you configured a 2nd server, it **copies the route files** there.

[B] Binarize routes — the binary for the routes you already had

Routes converted with an earlier version have no `_cinta.bin`, and a tape you converted again may have an old one. **When you open the wizard it tells you** if there is something to do (`5 routes without a binary or outdated: [B] binarize`); if not, it says nothing.

- `[B]` lists the routes in your folder with their state: `up to date`, `no binary` or `outdated` (with the reason: the CSV changed, from another mod version, truncated or damaged).
- You pick **one by number**, or `[T]` for all the missing ones. A route that cannot be converted shows in red and it moves on to the next; **the CSV is never touched**.
- If you have a mirror server configured in `[6]`, the binary is copied there too, but **only if the same tape is there** (same size); otherwise it warns, and the mirror keeps using its CSV.
- **It is not mandatory:** without a binary the route loads the same, slower, and the server says so in its RPT (`binario de la cinta descartado: se carga el CSV`).

[2] Import a BrigadaZ Transport v1 route

Were you using **BrigadaZ Transport v1** and already have a recorded route? **Don't record it again** — this option brings it into BZ_AutoDrive's format.

- **Where it looks for the file.** The v1 route **isn't in the PBO**: it lives in **your** server's profile, and that folder **can have any name** (a repack/fork renames it), so the wizard **doesn't guess it: you tell it** with **[B]** (and it remembers it). Point it at your `profiles\` and it searches **all the subfolders**. It also looks in the `_importar` folder it creates for you inside your routes folder (an inbox for the JSON you bring from **another** server), and with **[P]** you paste the path to a specific `.json`.
- **Which vehicle it's for.** It asks you for the **vehicle's identity** (`Wheelbase`, `Fingerprint`), which the v1 take doesn't know. It lists what you already have: any **recording** of that vehicle (the `header_*.txt` — a **10-second** one works) or an **already-converted route** of that vehicle (those also carry the measured braking). It puts first the one that matches the vehicle your route declares.
- **The obstacle profile.** It asks you whether Boris should get around cars stopped in the way: **Robust**, **Interceptable** or **None** (§A.5b). ⚠ **It only has an effect with the 1.1 brain** (`CEREBRO_11.txt`): the 1.2

brain, the one that runs by default, doesn't dodge; if something jams it, the AutoRecovery steps in (§9.6).

- **What it fixes for you.** v1 marked the stop where you pressed the key, not where the vehicle braked: stops declared **in motion** are flattened to 0, with the braking painted backwards; it collapses the repeated points from when you were stopped; it marks the end as a stop.

> What does **not** migrate: the driving profile is per vehicle. If your route is with another car, **record it with that car**: the recording brings *that* car's profile, not the route's.

[3] Merge recording segments — continue a recording

Did you record a long route **in parts** (you disconnected, stopped to rest, continued it another day)? This option joins those parts into **a single take**:

- You pick the `frame_*.csv` files of the parts (two or more, **in any order**): the wizard chains them **by where each one starts and ends**, not by time. The end of one part must be **less than 8 m** from the start of the next, and all of them must be **from the same vehicle** and **from the same version**: a part recorded with 1.2 doesn't join one from 1.2.1 (1.2.1 records more data per frame).
- A repeated part (same start and same end) is discarded with a warning; the NUMPAD 4 marks of each part are kept.
- You give the merged take a name and the wizard offers to **convert it into a route** right away (same as **[1]**).

> For the parts to join cleanly, **start each part where the previous one ended**: stop the vehicle, end the recording, and when you resume, start from that same point.

[6] Configure paths — the server folders

The wizard **remembers** the folders in `wizard_config.json` (next to the script) so it doesn't ask you every time. On the **first run** it asks you for them, in the language you chose; the routes folder is **required** (there is no default and it won't continue without it); after that you change them here whenever you want *(Enter = keep each one)*:

Folder	What it is
RoutesDir	where the server's **routes** go. It's your `…\profiles\BZ_AutoDrive\`. **The only essential one.**
Client recordings	where it reads your `frame_/path_*.csv` from (by default `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\`).
2nd server (mirror B)	if you have a separate test server, the wizard copies the route there when converting. `-` = none.
v1 routes	where it looks for the BrigadaZ Transport v1 takes (for `[2]`).

> The folders are **portable**: a modder with the server on another disk sets it **once** and never touches `-RoutesDir` again.

What the conversion does (and what it doesn't)

It's a **direct** conversion: `frame_to_route.py` takes your recording + its `header_*.txt` (the fingerprint) and builds the route files — it **generalizes** the line by reading the vehicle's physics from the config, and attaches the driving config. **There are no interactive linters and no "score"**: it's a pure converter, it won't ask you "*should I cap this curve?*". The quality comes from **recording well** (§7): if the take came out dirty (you climbed a curb, you braked badly), **re-record** — it's free and fast.

> **Want to see where Boris strayed from your take?** The framework can export its own trajectory (`boris_native`) in the same format as a recording, so you can compare it with yours. See §15.2.

6B. The 4 files the wizard generates — what they are and what each one is for

When you convert a take, the wizard doesn't produce one file: it produces **four** (plus an optional fifth, the tape's binary). It's the question that comes up most, and the short answer is that **each one is read at a different moment in the route's life**.

```
BZBusRoute_<NAME>.json            ~490 KB    the COMPLETE route in one file: gives it its NAME in the list.
BZBusRoute_<NAME>_hdr.json       ~3-12 KB    the ID CARD: who the vehicle is and what we know about it.
BZBusRoute_<NAME>_wp.csv           ~80 KB    the POINTS of the path (one row per waypoint).
BZBusRoute_<NAME>_cinta.csv       ~500 KB    your FAITHFUL driving, 40 times per second.
BZBusRoute_<NAME>_cinta.bin       ~350 KB    the same tape, in binary: much faster to load (optional).
```

(The example is a 500 KB tape, with the proportions measured on 41 routes: the binary weighs about seven tenths of the tape; the master, the same as the tape; the `_wp.csv`, a sixth; the ID card, 1 %. Per waypoint that is about 0.5 KB of tape, 0.35 KB of binary and 0.08 KB of `_wp.csv`. Everything scales with the length of the take.)

All four —and the binary, if you have it— go together in the same folder: `<server profile>\BZ_AutoDrive\` . And so you don't have to hunt through the manual, **where every file of the framework goes**:

file	where it goes
the recording (`frame_*.csv` + `header_*.txt`)	on the **client**, the PC you drove on: `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\` (if you recorded on the server, in `<server profile>\BZ_AutoDrive_PathLogger\`)
the route (`.json` , `_hdr.json` , `_wp.csv` , `_cinta.csv` and `_cinta.bin`)	`<server profile>\BZ_AutoDrive\` , all together (it is your wizard's `RoutesDir`)
the vehicle's sheet (`envelope_<class>.csv` , §6C)	`<server profile>\BZ_AutoDrive\` , next to the routes
the configuration (`BZAutoDrive_settings.json` , §13.1)	`<server profile>\BZ_AutoDrive\` ; if it doesn't exist, the server creates it with the default values on the first start
the run logs (`ai_run_*.csv` , `boris_native_*.csv`)	on the server that ran them: `<server profile>\BZ_AutoDrive_PathLogger\`
the wizard's memory (`wizard_config.json` : your folders)	next to the wizard script, in `tools\` ; it doesn't ship in the package: every admin starts without it and the wizard asks for their folders the first time

The master — `BZBusRoute_<NAME>.json`

What it is. The whole route in a single file: all the waypoints with their stops and names, the events, the vehicle, the attachments and that route's configuration options (Appendix A).

What it's for. It's **the route's name**: what shows up in the Player's list and what you ask for when you launch. It's also your **readable backup** — if you want to see the complete route in one place, this is the one.

> **Careful: it's not the one that is read to drive.** When the route starts, the server loads **the compact pair** `_hdr.json` + `_wp.csv` (and the tape), not the master. That's why all four have to be there: the master gives you the name, the others do the work.

The ID card — `_hdr.json`

What it is. A few kilobytes (3 without the vehicle profile, up to about 12 with it) with what defines that take:

```
VehicleClass        which vehicle you recorded with
Fingerprint         mass, gears, steering angle, wheelbase, minimum radius
Wheelbase  MaxGear  the geometry and the gearbox
Events              the stops and what happens at each one
Attachments         the wheels and other parts you recorded with
CreepKmh            this vehicle: does it creep on its own at idle?
CreepSource         where that number came from  ("coast2", "chist2", "nocreep")
CreepSamples        from how many measurements
CoastAtRest         the engine braking when nearly stopped, in m/s2
Envelope            copy of the vehicle's measured profile (§6C)
WpRows  CintaRows   how many rows the other two files have
```

What it's for. The server reads it **first and whole**, because it's small. With that it already knows which vehicle to spawn, what physics it has and **how much work reading the rest is going to cost**.

> **Since 1.2 the creep and the vehicle profile travel here**, and not only in separate files. That fixes at the root two costly failures: a loose file **gets lost** or **goes out of date** relative to the take. What travels inside can't come loose.

The points — `_wp.csv`

What it is. One row per waypoint: position, heading, gear, wheel angle, whether it's a stop, and its name.

What it's for. It's what the NPC uses to know **where the path goes**. It's loaded **in pieces**, spread over several server frames.

The tape — `_cinta.csv`

What it is. Your driving exactly as it was, **40 samples per second**: position, heading, chassis speed, front wheel angle, throttle, brake, gear and handbrake. It's indexed by **distance traveled**, not by clock.

What it's for. It's the difference between "following a path" and **reproducing your driving**. The speed the NPC uses at each point comes from here: it flies where you flew, it brakes where you braked.

> **Why by distance and not by time.** Because the NPC doesn't repeat your clock: it repeats your **course**. If you waited 6 seconds at a traffic light, that isn't reproduced on its own — the wait is set with an event (§10).

The binary tape — `_cinta.bin` *(since 1.3, optional)*

What it is. A copy of `_cinta.csv` that the server loads much faster: the same numbers, already in the format it reads. The wizard creates it when converting ("Convert & binarize"), and `[B]` generates it for the routes you already had.

What it's for. Reading the tape was the longest phase of the load. With the binary that phase weighs a fraction, and launching or relaunching several routes costs fewer frames.

It can be deleted without losing anything: without it, the server loads the CSV. And if the binary does not match its CSV (you changed the tape, or it comes from another mod version), the server discards it on its own, loads the CSV and says so in the RPT: `[CARGA] <route>: binario de la cinta descartado (<reason>): se carga el CSV`. **After updating the mod, open the wizard:** if it warns about routes without a binary or out of date, press `[B]` and then `[T]`.

★ Why four and not one

For a concrete, measured reason: **loading a whole route in a single frame hangs the server.**

A route is more than half a megabyte between points and tape. Reading it all at once freezes the simulation for as long as it takes, and if several routes are launched together, it multiplies.

The solution is a **load queue with a per-frame budget**: the server reads a little of each route on each frame and never spends more than it can afford. To split up the work it needs to **know in advance how much there is**, and that's exactly what `WpRows` and `CintaRows` declare in the ID card.

```
the ID card   is read WHOLE and instantly: who the vehicle is and how much the rest weighs
the points    are loaded in slices
the tape      is loaded in slices; it's the longest phase
```

The result: the cost per frame is constant — with 1, 2, 3 or 5 routes launched at once, the worst frame is the same.

Frequently asked questions

Can I delete any of them? No. Without `_wp.csv` the route is driven by the 1.1 brain; **without `_cinta.csv` the NPC follows the path but no longer reproduces your driving** (it runs tapeless); without `_hdr.json` the server loads the whole master, which is slow and **freezes the server** while it reads it; and without the master the route doesn't show up in the list. A route loaded through the whole master says so in the RPT, and to the connected admins too (`… is in the old format (no _wp.csv): loading it froze the server for N s…`): convert the take again with the Wizard. **The binary (`_cinta.bin`) can be deleted:** without it, the server loads the CSV.

Can I edit them by hand? The master and the ID card are JSON and can be edited. **The tape is not to be touched**: it's a measurement, not a configuration. If something about your route doesn't satisfy you, the answer is **to record it again**, not to correct the numbers.

Can I use this route with another vehicle? It's not advisable, and the reason is in the ID card: the take carries the *fingerprint* and the measurements **of that vehicle**. If you want the same route with another car, **record it with that car** — it'll come out better than any adaptation, because the framework learns from what you record (§6C).

Do I have to copy them by hand? No: the wizard puts them where they go. Just make sure you copy **all four, and the binary if you have it,** if you move a route from one server to another (they all live in the same folder); without the binary, `[B]` rebuilds it on the other side.

6C. The envelope — what the framework learns from your takes

Besides the 4 files of each route, the framework keeps **one file per vehicle class**:

```
<server profile>\BZ_AutoDrive\envelope_<CLASS>.csv
```

What it is. What the framework **measured** of that vehicle: how hard it brakes, how hard it accelerates, how much it turns with each steering angle, whether it creeps on its own. You don't configure it: it comes from your takes.

Why it exists. The framework reads the vehicle's config (engine, gearbox, steering) and with that it can already drive. But the config **declares** how it should behave; the envelope **records how it really behaves**.

When the two say different things, the measured one wins.

What it stores, field by field

```
scalars    wheel radius and wheelbase
fp         the fingerprint: the vehicle's signature, so different classes don't get mixed
medido     the provenance: from how many takes and how many frames this envelope came
k          curvature: how much it really turns with each steering angle
ratio      the transfer of each gear (rpm -> speed)
stx2       command -> wheel angle transfer
shist      steering histogram: which angles you used
ahist      acceleration: what this vehicle can do
coast2     deceleration in neutral, per surface            <- the creep comes from here
chist2     creep measured at idle (if any)
nocreep    how many times it was measured that it does NOT creep
lhist      throttle at launch, during normal driving
lcal       the launch floor measured separately            <- see below (empty in this version)
lcalrun    the signature of each floor measurement
```

How it gets filled

With the takes you record. Each conversion contributes whatever that take showed. A short take on a straight line contributes little; one with tight curves, hard braking and several speeds contributes a lot.

> **The richness depends on the variety, not the length.** Ten laps around the same roundabout teach a single curve. The envelope is complete when it **covers what that vehicle is going to have to do**: if your sedan is only going to drive 10 meters in a straight line, it doesn't need more.

> **An envelope belongs to ONE class.** It isn't shared between vehicles, not even between variants of the same model: two different liveries are two different classes and carry separate envelopes. The `fp` is there precisely so they don't get mixed.

Which profile a route uses

When it starts, the vehicle uses **its class's profile** (`envelope_<class>.csv`), the one that gets richer with every take you record of that vehicle. **For a vehicle to benefit from new takes you don't need to reconvert anything.**

Every route converted with this version also carries, inside its `_hdr.json`, **a copy of the profile as it was when the route was converted**. That copy is only used in two cases:

- **the vehicle's profile doesn't exist on the server** (for example, a route you took to another server);
- **the server's profile measured less than the copy** (a freshly installed server, with a profile from a single short take).

The RPT says which one it used, in the `[Envelope]` line. Reconverting a route only updates the copy that travels inside.

The launch floor: the only thing your takes can't measure

What it is. The minimum throttle at which that vehicle breaks from rest. Below that value, pressing the throttle does nothing.

Why it doesn't come from your takes. For two measured reasons:

1. **The keyboard is binary.** In 502 takes there are 1,873 starts from standstill, and in **1,778 of them the throttle was fully down**. The W key has no half measures, so your takes contain no intermediate value from which to deduce the floor.
2. **Starting from standstill is an engine transient.** In 2 out of every 3 launches, the speed rises to ~2 km/h and resets **without the vehicle having moved 10 centimeters**. That stretch doesn't reflect your driving: it's how the physics engine gets a vehicle out of rest.

What the framework uses instead. A **measured** fleet value: the floor of seven vehicle classes (a car, a motorcycle, a military 4x4, a truck and three buses), measured with the vehicle still on flat ground and the throttle rising little by little, between five and nine times each. They range from 0.05 to 0.25, and the framework uses the median, **0.13**, for any vehicle. You don't need to do anything.

What it implies — and what it doesn't. Let's be clear:

> **It doesn't change how it drives.** On a route without stops, between the fleet value and your vehicle's own value there is **one ramp tick**: about four tenths of a second when crossing 5 km/h.
>
> **Where it can be noticed:** when **starting on a slope**, where the framework has to hold the throttle just above the floor so as not to roll back. If one of your vehicles rolls back when starting on a steep slope, that's the place to look.

Measuring each vehicle's floor is a tool that exists in development and **doesn't ship in this version**. When it does, it will be explained here.

7. How to record well (the quality comes from here)

The NPC will drive **as well as you drove** — the recording is its driving manual. The wizard warns you about problems, but **it can't invent a good line**: you're the one who puts it in when recording. These habits make the difference:

- **Drive smooth and even.** No sudden acceleration or hard braking: the NPC copies your profile. Brake **progressively and early**, before the curve, not on top of it.
- **Take the curves with the line you want** the NPC to make — don't cut the apex or run wide.
- **Constant rhythm on the straights:** the speed you record is the reference the NPC will use.
- **One clean take is worth more than ten dirty ones.** If you climbed a curb or braked badly, **re-record** — it's free and fast.
- **Direction changes (K-turn, shed):** brake fully and, **without releasing the brake, shift gear** and continue — the **interchange is auto-detected** from the gear change (you press no key); reverse also

comes from the gear (§9.4). If you release the brake before shifting, the vehicle may keep creeping forward a few centimetres (downhill, it always does), and that creep is recorded as part of the route.

- **Stops:** one single continuous braking until you come to a complete stop, and mark there with **NUMPAD 4**. See "How to stop" below.
- **Horn and lights:** in this version **they are not recorded** — whatever you do with the horn or the lights while you record doesn't stay in the take. The **lights** are turned on by the framework according to the in-game time (`LightsMode: "auto"` , the default) and the **horn** is configured per route with `HornMode` (§9 "Horn and lights" and Appendix A.2b).

  > **For the lights to show:** the vehicle needs a **battery + installed bulbs (headlights)**. The framework energizes the battery on spawn, but the bulbs have to be among its `Attachments` (they usually already come from the vehicle's fingerprint). Without physical headlights, the L key turns nothing on.

> *Mental rule:* drive thinking "this is how I want the bot to do it". Whatever you do, it does.

7.1 — How to end the take (the NPC's exact arrival depends on this)

At the end of the take, **finish on a straight**: the last few meters with the steering straight, braking until you stop, and cut with **NUMPAD 5** with the engine running. **The end can be on a slope**: the NPC arrives all the same, uphill too.

If the end is on flat ground, before cutting **let go of everything** —throttle, brake and handbrake— and **wait for the engine to drop to idle**: it's two or three seconds, more on a sports car. During those seconds the framework measures how far the vehicle moves forward on its own, with no throttle. That measurement is what lets it nail the final point. Without it, it uses an estimated value and the arrival drifts. **If the end is on a slope, keep the foot brake pressed** while you cut (if you release it, the vehicle rolls back and that gets recorded) and do that measurement on a flat stretch of the take (§7.2).

- **The handbrake is no good for this:** with the handbrake on, it can't measure whether the vehicle moves forward on its own.
- **On a slope it isn't measured** (the measurement requires less than about 3° of slope): any flat stretch of the same take works, or a short take on flat ground with the same vehicle.
- **Doing it once per vehicle is enough.** The wizard saves the measurement per vehicle (`envelope_<class>.csv`) and doesn't overwrite it on later takes of the same vehicle. To redo it: `tools\python\python.exe tools\envelope_emit.py "<take.csv>" --out "<RoutesDir>" --pisar` .

7.2 — How to stop (intermediate stops)

One single continuous braking until you stop. Don't brake, release and creep the last few meters: the NPC may arrive a bit slower and settle on the *first* braking, meters short of your stop.

Only once you're completely stopped, mark NUMPAD 4, and continue.

- **You don't have to wait during the take.** The waiting time goes in that mark's event (§10.1); while recording, stop just long enough and set off.
- **You don't have to wait for idle at every stop**: that's a one-time thing per vehicle (§7.1). You can keep the brake pressed.

- **Shifting gear while stopped is fine** —dropping to first to set off—: the framework handles it.
- **At a direction change, keep the brake pressed until you engage the other gear.** If you stop, release and the vehicle creeps before reversing, the take records that creep and the NPC copies it. The framework still brakes at the end of every leg, but the most exact arrival comes from a clean take.

If you stop on a slope, keep the foot brake pressed while you mark or cut the take. If you release it, the vehicle rolls back a few centimeters and that rollback gets recorded. Use the foot brake, not the handbrake. And do the §7.1 measurement on any flat stretch of that same take: the most convenient is at the start — begin recording with the engine running, stay still for two or three seconds with everything released, and only then set off.

7.3 — Three things that ruin a take

Ending in a skid. If the vehicle slews or spins as you brake, re-record that ending. What the vehicle does sideways no driver can reproduce, and the framework may read it as a maneuver that doesn't exist. The wizard warns you with the exact second: `(!) TROMPO en la fila 7393 de la toma (t 184.8 s ...): el chasis gira 216 grados y desliza 179 -- la toma NO es reproducible desde ahi` (= "SPIN at row 7393 of the take (t 184.8 s ...): the chassis turns 216 degrees and slides 179 -- the take is NOT reproducible from there").

Squeezing through a tight spot. On narrow streets and tight curves next to signs, poles or walls, go slower and well centered. The NPC repeats your line with variations of centimeters from one run to the next: if you squeezed past in your take, sometimes it's going to touch.

Ignoring the jump warnings. At high speed the server sometimes corrects your position backwards while you record; it feels like a tug. The converter cleans it up on its own, but if you see `(!) glitch en racha` (= "glitch streak") or `(!) salto persistente` (= "persistent jump") in the wizard's output, check that stretch or re-record it.

Two more warnings, since 1.2.1. The wizard also tells you:

- `(!) The take has N recorded rewind(s)`: while you were recording, the server pulled your position back (the jerk). The route replays them as they are and the NPC doesn't follow that jump, but if it drifts on that stretch, record it again.
- `(!) At N gear change(s) the vehicle rolled forward without brake before shifting`: you stopped, released the brake and the vehicle crept before you engaged the other gear. When recording, keep the brake pressed until you shift (§9.4).

7.4 — Record it empty

Record with the vehicle **empty**. A full trunk and passengers don't change how the NPC drives it: speed, acceleration, trajectory and arrival come out the same. What you recorded empty works loaded.

---

8. The simplest use: a permanent bus service

It's the case the framework was **born** for: **a line bus that travels around your server by itself, all day, without you doing anything.** No missions, no admin operating it — ambient transport for your players.

(It's the "server-only" use, the most common one when you download the mod.)

How to leave it running as a permanent service:

1. **Record a route in a loop** (one that returns near the start) — e.g. the coast, stopping in every town.
2. **Leave it as the server's default route** (`BZBusRoute.json`).
3. When the server starts, the bus **appears by itself and starts driving**. When it finishes the lap (or if it's destroyed), it **respawns** after `RespawnDelay` seconds (`300` by default; `0` or a negative number count as **60 s**, they don't turn it off) → **continuous service, 24/7**. *(Routes you launch from the Player are not relaunched: when they finish, the vehicle withdraws on its own.)*

> It doesn't need a Quest or for you to be connected: just the mod loaded + your default route. You turn it on and forget about it. *Example:* the Chernarus coastal bus linking 14 stops, going around all day — players take it to get around, like a real bus.

The three ways to run a route (so you have them clear):

Way	How it starts	What for
Permanent service	by itself, at server boot (default route + loop)	ambient line bus
On-demand (Player)	you pick it in the UI, no restart	testing, one-off events
By mission (Quest)	a quest triggers it	convoys, reinforcements, ambushes (§11)

8.1 — Stop on demand: wave the bus down *(validated in-game with the 1.1 brain; not yet tested with 1.2)*

The line bus doesn't only stop at the `isStop` waypoints: **any player can ask it to stop by waving it down**, like in real life.

> **It comes disabled by default** (opt-in — it's a rule the admin chooses). To enable it, set `HailGestureEnabled = true` in the mod's global *settings*; without that, the bus only stops at the `isStop` waypoints. While it's active, the rule applies to **all** your routes.

How to use it (player):

- Stand **on the road, facing the bus** (where the driver can see you), at ≤30 m.
- Do the **OK / thumbs-up** emote (`ID_EMOTE_THUMB`).
- Boris **brakes, waits for you ~10 seconds** so you can get on, and **continues** the route.

What happens under the hood:

- The detection is **server-side**: the framework goes through the players near the vehicle, checks whether they're in the bus's **frontal cone** (dot product of bus-heading · direction-to-player > 0.25 → "the driver sees them", not behind or to the side) and whether they're performing the OK emote.
- On detecting it, it **pauses** the bus (reusing the pause-mode brake) and schedules the resume at **+10 s**.
- After 10 s it resumes. With the 1.2 brain the tracking is by position, so it picks up from where the vehicle ended up; with 1.1 it resumes by **re-locating** the waypoint to the bus's actual position (see below).

Parameters: the **on/off** is already config (`HailGestureEnabled` , default off); the rest is currently in constants (easy to expose to JSON): radius 30 m · cone `dot > 0.25` (~±75°) · wait 10 s (20 ticks) · emote `ID_EMOTE_THUMB` (9).

Two things that were hard to solve (notes for whoever extends it):

1. **Reading WHICH emote the player is doing, server-side.** `EmoteManager.GetGesture()` does NOT work: it returns `m_GestureID` (a different field, set by `SetGesture()`), not the emote in progress. The actual emote lives in `m_CurrentGestureID` , which is `protected` and has no public getter. Solution: a `modded class EmoteManager { int BZ_CurrentGesture() { return m_CurrentGestureID; } }` — it's a script class (not engine → moddable), and the `protected` is accessible from the subclass.
2. **Resuming straight after the stop** *(1.1 brain)*. When it brakes, the bus keeps going ~10-15 m while it stops, but the waypoint index stays frozen where it braked. Since the index advance is capped by speed (it doesn't advance from 0 km/h), on resuming the bus is *ahead* of its target → the control was aiming at a wp left behind → **swerve to the side**. Fix: on resume, **re-locate the index to the waypoint closest to the actual position** (once, skipping the cap).

> **The gesture as a primitive.** It's the first use of a more general pattern: *the player's emote as a control input to the framework*. Today it's wired to "OK → stop", but the same mechanism (reading the emote server-side + a proximity/vision condition) works for any command (follow, wait, turn around). It's the player↔NPC counterpart of the triggers in the event DSL (§10).

8.2 — Launching and chaining routes from your mission's `init.c` (no mod of your own) *(tested in game)*

It's the most basic way to trigger sequences: a few lines in the `init.c` of the server's mission (`mpmissions\<your mission>\init.c`) and the framework does the rest. **You don't need to build a mod:** `init.c` sees the classes of the loaded mods.

Where it goes. Inside the `CustomMission` class your `init.c` already has, add an `OnInit` (if you already have one, add the lines inside it, after `super.OnInit()`) and the methods below. Routes are named by **file**: `BZBusRoute_<name>.json` , exactly as the Wizard leaves them in `profiles\BZ_AutoDrive\` .

```
class CustomMission: MissionServer
{
    // ... what you already had ...

    override void OnInit()
    {
        super.OnInit();   // first: this is where the framework starts
        // wait for the world to be ready before launching (60 s)
        GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.BZ_Lanzar, 60000, false);
    }

    // launches a route with its own runner; returns null if it didn't start
    BZBusService BZ_Ruta(string archivo, string etiqueta)
    {
        BZBusService r = new BZBusService();
        BZBusService.GetRunners().Insert(r);
        r.SetHoldAtStops(false);          // false: at stops it waits as recorded and moves on by itself
        r.SetEtiqueta(etiqueta);          // signs the log: <route>#<label>
        r.RespawnFromPath(archivo);
        if (!r.RunnerIsActive() && !r.RunnerEsperandoCabecera())
        {
            BZBusService.UnregisterRunner(r);
            return null;
        }
        return r;
    }

    void BZ_Lanzar()
    {
        BZ_Ruta("BZBusRoute_PATRULLA.json", "P1");
        // another one, 2 minutes later
        GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.BZ_LanzarConvoy, 120000, false);
    }

    void BZ_LanzarConvoy()
    {
        BZ_Ruta("BZBusRoute_CONVOY_01.json", "C1");
    }
}
```

Chaining: when one ends, launch the next. A runner launched this way **removes itself** when it arrives complete (after `EndHoldSeconds`, 3 s by default), so it's enough to check now and then whether it ended. These methods go in the same class:

```
ref BZBusService m_BZ_Tramo;       // the runner that is running
int m_BZ_Paso = 0;

void BZ_Cadena()   // call it once (for example from BZ_Lanzar)
{
    m_BZ_Tramo = BZ_Ruta("BZBusRoute_IDA.json", "IDA");
    GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.BZ_Vigilar, 5000, true);   // every 5 s
}

void BZ_Vigilar()
{
    if (!m_BZ_Tramo || !m_BZ_Tramo.RunnerRouteEnded()) return;   // still driving
    if (!m_BZ_Tramo.RunnerRouteCompleted())
        BZBusService.UnregisterRunner(m_BZ_Tramo);             // ended badly: remove it yourself
    m_BZ_Paso++;
    if (m_BZ_Paso == 1) m_BZ_Tramo = BZ_Ruta("BZBusRoute_VUELTA.json", "VUELTA");
    else
    {
        m_BZ_Tramo = null;
        GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).Remove(this.BZ_Vigilar);   // end of the chain
    }
}
```

What you need to know:

- **`SetHoldAtStops(false)`.** With `true` —the default of `PlayRouteByName`— the vehicle stays at every stop until someone calls `ResumeFromHold()`: from an `init.c` it would sit at the first stop forever.
- **Its own runner per route**, not `PlayRouteByName`: that way each one removes itself when it ends and you can remove it with `UnregisterRunner`. It shows in the panel with its label, and you can stop it from there too.
- **If it ends badly** (`ROUTE_INCOMPLETE`, `STUCK`), the vehicle stays still: remove it yourself, as in the example. *(Since 1.3, if you neither remove it nor read the end within 60 s, the framework removes it on its own: §B.7.)*
- **If the starting point is occupied**, the runner waits for it to clear (`RunnerEsperandoCabecera()`): in an out-and-back chain, the way back leaves when the outbound vehicle is removed.
- **For a service that loops forever**, make that route the server's default route (§8), or relaunch it when it ends with the same chain pattern.
- **Each route's events** (§10) work the same: `init.c` decides when each one starts; what happens inside comes from its `Events`.
- **An error in `init.c` keeps the mission from starting.** Check the RPT (`Can't compile`). The Enforce rules of Appendix B.5 apply: no ternary operator, `new` takes no arguments, etc. The full API (state, pause, events) is in Appendix B.7.

8.3 — What happens at the end of the route *(since 1.3)*

When a run ends —it arrived, it didn't, or it got stuck— and before the framework retires or relaunches the vehicle:

- **The player riding seated gets out on their own**, with the usual animation and **no notice**. If you want to warn them, put a `ui_broadcast` event on the last waypoint (§10).
- **From that moment nobody gets in.** The vehicle stays **locked to players** (Expansion's lock: the client doesn't even offer "get in") until the framework deletes it. The same with the **panel's STOP** and in any **retirement**: first it locks, then it gets the seated player out, and only once empty does it delete. If retiring hits its cap and leaves the vehicle in the world, it **unlocks** it as it was.
- **Motorbikes too:** the same dismount and the same lock.
- Retiring **never** deletes a vehicle with a player seated or getting in: it gets them out and waits.
- **If you program a mod:** the END gets the players out. To **keep passengers between routes** (a trading service travelling with people aboard, a taxi chaining trips), **hold the vehicle with `HoldRunner()` before the end** (B.7.3) and carry on from there, as a trading service does: the hold gets nobody out and locks nothing; the END does.

---

9. How the NPC drives — it follows your line and your speed

You don't pick a "mode": there's a single control, and it's the one that reproduces your driving. The NPC:

- **follows your recorded line** (*pure-pursuit*: it aims at a point ahead on your trace), and
- **uses your recorded speed** —it flies on the straights, brakes before the curves, just like you—, **moderated** by what the vehicle **can** do in each curve: if you recorded a curve faster than that car can hold, it **caps it by itself**.

The throttle and the brake start from **your recorded pedals** (the tape) and are corrected by an **inverse model** derived from the **vehicle's config** (engine, torque, gearbox) and from what was **measured** of it: if the vehicle can do less than what you asked for, it adjusts. That's why **the same controller drives any vehicle with no per-car tuning** (config-as-manual): each one drives according to ITS physics — the small one nails the line, the heavy one does it slower but gets there. *(The recording itself belongs to its vehicle: to run the same route on another car, **record it with that car** — §6B, §6C.)*

> **You configure nothing:** the wizard produces this control on its own, reading the car's fingerprint (§5). *(There used to be "modes 1/2/3" —repeat pedals / speed by curvature / recorded speed—; they were **unified into this single control**, the one this version validates. The flags that turned them on — `FollowPath`, `UseInverseModel`...— **belong to the 1.1 brain**: with 1.2, the one that runs by default, they are not read (Appendix A.2).)*

> **Is a heavy vehicle forced along at your pace?** The best thing is to **record the route with that vehicle** — that way the run is born to fit it. *(If you later want that same route for smaller cars, **record it with one of them** instead of reusing the heavy vehicle's recording: each take feeds the envelope of ITS class — §6C.)*

> **What the NPC does not reproduce: the time spent stopped.** The NPC repeats your route **by position, not by the clock**. What you did while standing still —waiting, looking around— is not repeated: on reaching a stop it sets off as soon as it comes to a halt. You add the wait yourself with an event on that mark (§7.2 and §10.1).

9.4 — Direction changes (K-turn, backing in): auto-detected

When your route needs to **change direction** —a three-point turn (**K-turn**), turning around, backing into a shed— **you no longer mark anything with a key**. The framework does it **on its own**: it detects the **direction change** from the **gear change (forward↔reverse)** you made while recording, and cuts the leg there (`legBreak`) by itself.

How it's recorded — the rule: always STOPPED when reversing direction. You can't switch from forward to reverse (or the other way) while moving —neither you nor Boris—, and that gear change **always happens at speed ~0**. Precisely because of that, the converter can read it without ambiguity. In a K-turn:

1. You're recording in **forward**. You reach the point where you'll back up → **brake fully** → engage **reverse**. *(You press no key: the forward → reverse change is the interchange, and the converter detects it here.)*
2. You do the reverse: **slowly (3–8 km/h), with small corrections** (the arc doesn't have to be perfect, you correct it as you go). The framework **auto-detects** that this leg is reverse —from the gear— and treats it as reverse.
3. When you finish → **brake fully** → engage **forward** and continue. *(Again no key: the reverse → forward change is the second interchange.)*

You repeat this at each direction change, always the same way: **brake fully, change gear without releasing the brake, continue**. Each of those gear changes cuts the leg (`legBreak`), and Boris treats it as a **new start**: he arrives aligned, plants himself, and sets off cleanly in the new direction. It's the *cusp* (reverse↔forward), the hardest point of the control — now **resolved** (§17).

> **Why without releasing the brake.** If you stop, release the brake and the vehicle creeps at walking pace before the change (downhill it always does), the take records it. The NPC replays that creep and brakes at the end of the leg, but **what the vehicle moved below 1 km/h inside your last stop is not replayed** (typically 30-40 cm): the leg closes where the vehicle came to rest on entering that stop. With the brake pressed until you engage the other gear, the stop and the change land in the same place.

> **A 0 km/h is NOT always an interchange.** Braking fully **without changing direction** (waiting, yielding) is a **pause**, not a leg cut. The converter tells the two apart: only the 0 km/h **with a forward↔reverse gear change** produces the `legBreak` . That's why you don't need to mark anything — the gear already says it all.

What the framework does BY ITSELF (you mark nothing):

- **The interchange** — comes from the recorded **gear change** (forward↔reverse): that's where the converter puts the `legBreak` and cuts the leg.
- **The reverse** — comes from the **recorded gear**: if you drove in reverse, that leg is reverse. Boris reproduces it with his reverse controller (rear-axle steering) and **generalizes it to each vehicle by its**

physics (one with a wide turning circle reverses slower and so follows the arc better).

- **The approach** to the maneuver — **automatic**: Boris brakes predictively BEFORE the maneuver, without you marking anything.
- **The final stop** (endpoint) — self-adaptive per vehicle + surface (§17).

> **Golden rule: brake on the straight, not on the curve.** The point where you brake to reverse direction is where the auto-detect **cuts the leg** — so do it where the **trajectory IS ALREADY STRAIGHT** (the heading has already settled), not mid-curve. The details of why (the reverse's steering angles belong to the vehicle, not to the road) and the multi-vehicle validation are in §9.5.

> *Tip (config-as-manual):* record the reverse **on the safe side** — an entry as **straight** as possible and minimal corrections. The cleaner your demo, the cleaner the NPC. On an **uphill ramp** (shed) keep the momentum: **don't brake to zero on the climb**.

> *Takes that start in reverse.* The converter defines the direction of the **initial leg** by the **first real movement** (>1 km/h), not by the gear you're parked in. If your route starts by backing up, make your **first movement BE the reverse** —without creeping forward even a hair— and set off decisively, so the start is read as reverse.

Fine maneuvers. Want the EXACT arc of a parking maneuver? You get it by **recording it calmly**: brake fully, engage the gear, do the maneuver slowly (3-8 km/h) with small corrections. The framework reproduces **what you did**, so a carefully recorded maneuver comes out better than a rushed one. If you didn't like how it came out, re-record that stretch: it's faster than correcting it afterwards, and the result is faithful to that vehicle's physics.

> `maniobra` **(legacy):** there used to be a `maniobra` mode (direct replay with exit by waypoint crossing) that you marked with a key. **Deprecated** since 2026-06-17 and **removed from the menu**: cruise + automatic approach + geometry cover it. It doesn't apply to new takes.

9.5 — Making a trace with a maneuver work across vehicles (where to cut the direction change)

The **reverse** reproduces the **steering angles** you used (the tape), with a small correction so it doesn't leave the line; if the tape isn't enough, a dedicated reverse control follows the path. Since the correction is small, the **"arrive ready"** premise holds: the vehicle must reach the reverse leg in the **pose and speed** you demonstrated — a large entry error is not fully corrected.

The fine point: steering angles belong to the vehicle, not to the road. The same steering angle produces a **wider or tighter radius depending on the wheelbase** (a long car runs wide, a short one closes in). So a leg reproduced with your angles **over a curve does NOT generalize** to another vehicle: the car that recorded it closes the curve, but a longer car, with those same angles, runs wide and takes it out.

The golden rule — cut on the straight, not on the curve. Make the **direction change** (brake fully before reversing direction; that's where the auto-detect cuts the leg) where the trajectory **IS ALREADY STRAIGHT** (the heading has already settled), not mid-curve. That way:

- The **curve** is handled by the **closed-loop control** (the pure-pursuit follows the **path**, vehicle-agnostic — each car takes it with ITS own steering).
- Only the **straight + the maneuver** depend on your steering angles, and on a straight that is safe (there's no curve to run wide on).

Cross-vehicle: record it again. A **recording** belongs to **its** vehicle: it captures its *fingerprint*, its measured brake, its gear, and it feeds the envelope of **that class** (§6C). That's why **header-swapping recordings is not recommended** — giving it another car's ID card loads it with measurements that aren't its own.

To run the same route **with a maneuver** on another vehicle, **record it with that vehicle**. With a maneuver involved, the reason is even stronger: the reverse **reproduces your steering angles**, and those angles belong **to the vehicle, not to the road** — a longer car with those same angles runs wide and takes the curve out. That isn't fixed by reassigning: it's fixed by recording.

The framework, for its part, **retrains nothing per vehicle**: it reads the car's config (engine, torque, gearbox — "config as the driving manual"), adds what it measured in that class's envelope, and builds its curve advisory and its inverse model to fit THAT car.

> **Validated (the cut rule).** Testing the **same trace** on two vehicles with different wheelbases — **OffroadHatchback** (2.357 m) and **CivilianSedan** (2.935 m, longer)—:
>
> - with the **cut on the straight** → it completed the maneuver without crashing; the reverse nails the interchange well aligned (stop precision resolved — §17).
> - with the **cut on the curve** → the longer Sedan **ran wide and crashed** (those steering angles were the Hatchback's).
>
> Practical takeaway: **make the direction change on a straight, not on a curve.** And if you want that same route for another vehicle, **record it with that vehicle**: the maneuver is the first thing that stops generalizing.

The approach to the maneuver is automatic. The control brakes predictively before the direction change, without you marking anything. If the cut falls in a **fast zone** and you notice it **brakes too much** and plants itself right at the transition, the answer is to **cut the maneuver where you're already going slow** (not in the middle of a fast straight) — that way the entry is smooth.

No alignment teleport. The vehicle drives into the reverse, with no jump in position. *(The 1.1 brain had an alignment "snap", `ModeEntrySnapEnabled`, off by default; 1.2 doesn't use it.)*

Horn and lights — the `HornMode` / `LightsMode` modes

Apart from **how** it drives, a route defines **what it does with the horn and the lights**. **In this version the take records neither the horn nor the lights**, so both are configured per route (Appendix A.2b):

- `HornMode` — `off` (**default**: never) · `stops` (honks when each stop ends, before leaving) · `finish` (honks on reaching the end, when the route finishes complete). `replay` dates from when the take recorded the horn: today only the `horn` verb sounds, and the server warns about it when the route loads.
- `LightsMode` — `auto` **(DEFAULT)** · `off` · `auto_inverted` · `on` (details below). `replay` is of no use in this version: there are no recorded lights to replay (only the verbs act, and the server warns about it

when the route loads). **A `lights_on` / `lights_off` verb overrides `LightsMode` until the next spawn.**

> **Horn or lights at an exact point?** Mark that point with **NUMPAD 4** while recording and hang an event on it with the verb `horn`, `lights_on` or `lights_off` (§10).

Automatic lights by default (`auto`). Every take starts in `auto`: at night the vehicle **turns the headlights on by itself** from the moment the engine starts (as soon as Boris sits down and starts it, before it begins to roll), and during the day it leaves them off. If the time changes during the route, the lights follow it. **You don't need to record anything.** The "night" threshold is the game world's time: **lights ON from 19:00 to 06:00**, OFF the rest of the day.

The `LightsMode` modes:

- `auto` — *(default)* on at night (19:00–06:00), from the moment the engine starts and also if night falls mid-route; off during the day.
- `off` — lights **always off**. This is the override for **stealthy night missions**: the vehicle doesn't give itself away with its headlights even at night.
- `auto_inverted` — the opposite of `auto`: **turns off at night** (time-based stealth) and on during the day.
- `on` — lights **always on**, day and night.

> **How to TURN OFF the lights.** If you want a route to run **in the dark** (stealthy night mission, not giving the vehicle away), put this in the route's header/JSON:
>
> ```
> "LightsMode": "off"
> ```
>
> With `off` the headlights never come on, even at night. (To go back to the automatic behavior, remove the field or set it to `"auto"`.)

> *Example:* a normal passenger bus doesn't need to touch anything — with the `auto` default it already turns them on at night by itself. An infiltration convoy uses `"LightsMode": "off"` (or `auto_inverted`) so as not to give itself away in the dark. Remember: for the lights to show, the vehicle needs a **battery + installed headlights** (§7).

9.6 — When it gets stuck: the AutoRecovery

A long route, running unattended for hours, sooner or later runs into something: a car someone left on the roadway, a stop on a slope where the vehicle rolled back a few centimeters, a climb where the engine bogs. The **AutoRecovery** handles that. It ships **enabled**, works on its own, and **does not jump the vehicle to another part of the route**: it frees it where it is —at most it relocates it a few meters— and the route continues from there.

How it notices. It doesn't look only at speed: a vehicle that is pushing something, or spinning against a curb, reads half a km/h without advancing a meter. That's why it looks at **progress along the route**:

- if it hasn't advanced in **10 s** and the throttle is floored, it is **stuck**;
- if it hasn't advanced at all in **20 s**, whatever the reason, it is stuck too;

- a **recorded stop** (a place where you stopped while recording) **does not count** as a jam: the vehicle is still on purpose.

It works at three levels, from least to most invasive.

1. The brain got stuck; the vehicle can move. This is the typical case at a stop: the vehicle ended up a few centimeters off the point where the recording expects it and doesn't latch on again. The AutoRecovery restarts it on the spot, up to three times. Nothing odd shows on screen: the vehicle sets off and continues.

2. The engine bogged. On a climb, if the vehicle arrives slower than in the recording, the gear it's in may not be enough. The controller forces **first gear** and pulls away the way you pull away on a slope. This is done by the controller, not the AutoRecovery, and it is configured separately (`BogRecovery`). But full throttle with the engine at idle is also what you see when the vehicle **is pushing something**. So if in first gear it advances a little and gets pinned again, or if in 10 s it doesn't cover even what it would cover at 2 km/h, **it wasn't a bog**: it goes to the ladder at that very moment.

3. Physical jam: it crashed, ended up crosswise, riding on top of something, sunk or rolled over. This is where the **ladder** comes in, trying these steps in order, in the same place:

1. **Drive out.** It moves 3 m away in a straight line, at no more than 5 km/h, in the **opposite** direction to the one it was going (in reverse if it was going forward; forward if the leg is one of those recorded in reverse), and then heads back onto the route by itself. If there's an obstacle on that side —another vehicle, a bus shelter, a post, a guardrail, a wall—, it doesn't move: it detects it and goes to the next step.
2. **Retrace its steps.** It relocates to its last good ground, one wheelbase further back, with the orientation the human had while recording there.
3. **Safe zones ahead.** The first one is at half the vehicle's length plus a meter and a half, which is the first position where the rear bumper clears the point of the jam. After that it advances one meter at a time.

All three steps also work on reverse legs (backing out of a bay, backing in). If a destination is occupied by another vehicle, or the vehicle ends up riding on something, that attempt isn't spent: it shifts the position one meter and tries again. Each relocation forces the start from that point, and the ladder resets when the vehicle gets moving again.

Only if the ladder is exhausted is the route declared stuck (`STUCK`), and the log says why. If the ladder is in the middle of a step, the route is not cut off.

What it doesn't do. It doesn't brake for, or steer around, a vehicle stopped in the way: if it pushes it without advancing, after 10 s it treats that as a jam. *(The `ObstacleSlow` in Appendix A.5b belongs to the 1.1 brain.)*

None of this hides the problem. Every intervention is logged:

- In the RPT, with the place and the evidence that triggered it: `[AR] <ruta> wp 380 s 594 · FISICO (...) · escalon 1: maniobra` (= "PHYSICAL (...) · rung 1: maneuver"). When the ladder opens you get `[C12] BLOQUEADO en wp N · <motivo>` (= "BLOCKED at wp N · "), and when it closes, `[C12] BLOQUEO terminado tras X s` (= "block ended after X s"): there must be as many of one as of the other. If your mod needs to know (for example, so that another vehicle doesn't yield to one that can't move), query `RunnerBloqueado()` (Appendix B.7).

- At the end of the route, on the `FIN` line. **Since 1.2.1, a physical AutoRecovery intervention (maneuver or relocation) leaves the route `ROUTE_INCOMPLETE`, even if it arrives:** `*** ROUTE_INCOMPLETE en wp 335 · AR fisico con accion: wp 135 maniobra: … · REPOSO tras 0.5 s: a 0.106 m …` (= "physical AR with action: wp 135 maneuver … · AT REST after 0.5 s: 0.106 m …"). Soft restarts (`NO-FISICO`) don't invalidate it, but they are listed: `AR x1 (wp 93 no-fisico)`. See §17 (the verdict).
- In `profiles/BZ_AutoDrive/AR_log.csv`, one row per intervention across all runs: date, route, vehicle, waypoint, type, evidence, action and result.

If a route needs the AutoRecovery **always in the same place**, that's information: go to that point and look at what's there. It's usually a stretch of the recording that needs to be redone.

Manual intervention. If you're watching a run and want to free it yourself, write the route's name into `profiles/BZ_AutoDrive/BZ12_AR_MANUAL.txt`. The vehicle relocates 10 m further along the route, counted from its last good ground. If that spot is occupied, it does nothing.

All the values are configured per route in Appendix A.5.

9B. DayZ 1.30 — what changed in vehicles, and motorbikes

9B.0 — What changed in vehicles with 1.30 (cars too)

1.30 changed vehicles under the hood. BZ_AutoDrive **1.2.1** already handles it, and the same package runs on 1.29 and on 1.30. What you should know as an admin:

what changed	what happened	what AutoDrive 1.2.1 does
Lights use a new system; the old headlight function no longer exists	a mod that turns headlights on the old way **doesn't compile** on 1.30. Neither does AutoDrive 1.2: **the server doesn't start**	uses the new system on 1.30 and the old one on 1.29, and makes the NPC's lights visible to the other players
Deleting a vehicle with someone seated	on 1.30 it **crashes the server** (everyone kicked)	when a route ends, it first gets the NPCs out and then deletes the empty vehicle. If a **player** is seated, it waits until they get out
Where the game looks for files	with 1.2, the **route list comes out empty** on 1.30	reads the routes from the profile folder in another way. It needs the server to start with a **relative** `-profiles` (e.g. `-profiles=profiles`): with an absolute path the list stays empty and the RPT warns (§15)
Tires have new parameters (lateral and longitudinal friction, roughness, type)	—	**measured:** routes recorded on 1.29 drive the same on 1.30, with the same precision
Who simulates the vehicle: with a player on board, **their** client simulates it	—	nothing to do: the NPC drives the same with player passengers in a car
Motorbikes are a new class, separate from cars	—	it drives them (see below)
The speedometer of a freshly spawned vehicle may not give a number for a few seconds (measured on the M1025, which has an automatic gearbox)	the route got stuck at the start	it uses the vehicle's real speed while it lasts and says so in the RPT (`[Velocimetro] … no da un numero`)

> ⚠ **If you use another mod that deletes vehicles** (cleanup, respawn, events): on 1.30 it **must never delete a vehicle with someone seated**. Check it before updating the server.

Car vs motorbike, at a glance:

	car (`CarScript`)	**motorbike (`MotorbikeScript` , 1.30)**
reverse	yes (direction changes, §9.4)	**no**: going backward means walking it
brakes	brake + handbrake	brake = **front**; handbrake = **rear**
falls over	no	yes: the NPC picks it up
leans in curves	no	yes: the lean in your take is the reference
walking it	—	yes, with the engine on or off: the NPC repeats it by pushing
back seat / passengers	yes	no, on 1.30
record and convert	NUMPAD 5 and the wizard	the same
unsticking	the AutoRecovery ladder (§9.6)	walk 3 m back, and if that isn't enough, relocate

Motorbikes *(experimental)*

Since 1.30, DayZ has motorbikes, and **a motorbike is not a car**. The game builds them on their own class (`MotorbikeScript`), a separate branch from cars (`CarScript`). AutoDrive recognizes them **by that hierarchy**, so there is nothing to configure by name:

- **Both vanilla motorbikes** of 1.30 (`Motorbike_01_*`, the Moped, and `Motorbike_02_*`, the Enduro) work as they are.
- **A modded motorbike built on the new script** works too, without touching anything.
- **An old modded motorbike built as a car** (`CarScript`) keeps going the car way, as until now.

Everything about motorbikes exists **only on 1.30**. The same package runs on 1.29, and there that part simply isn't there.

9B.1 — What the NPC does differently with a motorbike

You record, convert and launch the same as with a car. What changes is how it rides:

- **The brakes.** The motorbike's brake (the brake key) is the **front** one, the main one; the handbrake is the **rear** one. The NPC uses both that way.
- **No reverse.** A motorbike route has no direction changes (§9.4). To go backward, the bike is **walked** (see §9B.2).
- **Gears and starting.** When starting off, the game sometimes "loses" the first shift: the NPC insists until it engages. It starts without the kick, and raises the kickstand when it takes control (it lowers it when it lets go).
- **Leaning.** In curves the bike leans. The NPC takes as reference **how much you leaned at that same point**: as long as it doesn't lean more than in your take (with a 2° margin), it doesn't brake or trim the steering.
- **The line.** The motorbike rider corrects with damping, so it doesn't zigzag, and with the sensitivity of **that** bike, which the wizard measures from your takes and stores in its envelope (§6C). **Above 65 km/h** it corrects differently: a bike takes half a second to lean after the handlebar turns, so the NPC leans it according to the upcoming curve of the path instead of correcting with the handlebar like a car.
- **The arrival.** It brakes until it stands still on your final point, measuring in that same braking how hard the bike really brakes. In testing it arrives within 1-4 cm, even when coming in fast.
- **If it falls,** it picks the bike up and seats the rider again.
- **If it hits something head-on or gets stuck,** it walks **3 m back** and tries again. If it happens again at the same place, it relocates it a few meters ahead on the route, like the car's AutoRecovery (§9.6). The log says so: `[MOTO] DESTRABE (choque de frente) en el arco 800 m: camina 3 m hacia ATRAS…` (= "UNSTICK (head-on crash) at arc 800 m: walks 3 m BACK…").

9B.2 — How to record on a motorbike

Same as a car: **NUMPAD 5** to start and to stop (§5, §7). Get on, start the engine, record, ride and finish **stopped**. **Finish on a straight**: the last few meters with the handlebar straight, braking until you stop. If you finish turning at walking pace, the bike can end up half a meter from the point (§7.1).

Walking the bike is fine. If you get it out of a spot by walking it backward, or you walk a short stretch forward, record it that way, with the engine on. The wizard detects those stretches (`[moto] caminando: … se`

`reproduce por empuje` = "walking: … replayed by pushing") and the NPC repeats them **at your speed**, pushing the bike with the rider seated. It looks like walking, with the game's animation, **backward and forward**. If you turned the bike while pushing it (for example, backing it out of a spot and pointing it another way), the NPC reproduces that turn and corrects if it drifts off the line. Afterwards it respects the wait you made and starts off in first gear, at full throttle.

Avoid doors and gates. The state of the map's doors (open or closed) **changes between server restarts**. If your route goes through a gate or ends inside a garage, the day it's closed the bike crashes. It unsticks itself, but the arrival is no longer clean. AutoDrive **doesn't open doors**: what to do with them is your decision as admin. Record on roads without doors.

Debris isn't a crash. Riding over debris or objects on the ground is logged as a contact, but without a loss of speed it doesn't count as a crash.

Wizard warnings specific to motorbikes:

- `take with a broken clock … is NOT converted`: the take was recorded with an old version of the recorder. Record it again.
- `walking: … replayed by pushing`: a walked stretch. That's normal.
- `(!) take with a crash at arc X m`: in the take you really crashed (you lost speed against something). The NPC will repeat your line **toward** that crash: record that stretch again.

9B.3 — Limits (why it says experimental)

- **Tested** on 1.30 experimental and on the **stable** release (15/10): full routes with the Moped (with backward walks that turn) and with the Enduro **up to 120 km/h** (maximum offset 0.4-0.56 m over the whole route, arrival within 3-4 cm). Only the two vanilla bikes: modded ones will be added as their authors adapt them to 1.30.
- **The back seat** can't be used: on 1.30 experimental the game had it half-done (not re-tested on the stable release), and a passenger ends up overlapping the rider. AutoDrive doesn't seat passengers on motorbikes.
- **eAI NPCs on motorbikes:** tested on 15/10 with Expansion 1.9.79. The eAI driver rides the Enduro and the Moped (walking included) and the retirement gets it off and deletes it just like in a car.

---

10. Events and sequences — make things happen on the route

A route isn't just driving from A to B: you can make **things happen** at defined points — the vehicle stops, people get off, an audio plays, bots appear. That's built with the **NUMPAD 4** marker + the events.

10.1 — Your first sequence, step by step (from scratch)

Goal: make the bus STOP at a point, play a message, wait 3 seconds and continue. We do it from start to finish.

Step 1 — Record and mark the point. You drive the route normally (NUMPAD 5 to start recording). When you reach the place where you want something to happen (the stop), you tap **NUMPAD 4** *once*. You keep driving to the end and tap **NUMPAD 5** to stop recording.

> Each NUMPAD 4 leaves a mark at that exact point. You can mark several points in the same lap.

Step 2 — Run it through the wizard. You run the wizard → **Convert** → pick your recording. It generates the route and, in the summary, tells you **how many events** were created (`eventos N`): one per NUMPAD 4, in the order you pressed them.

> You'll see the **waypoint (wp) number** of each mark in the next step, in the `_hdr.json` . It's the address of your mark — you'll use it to hook the action.

Step 3 — Where the mark ended up in the file. Open `BZBusRoute_<name>_hdr.json` in your routes folder. The wizard has already left there, in the `Events` list, one event **per NUMPAD 4** (in order), with its `wp` and an empty action list. If you pressed NUMPAD 4 only once, there's only one; note its `wp` (in the example, 88):

```
{ "wp": 88, "trigger": { "type": "wp_reached", "wp": 88 }, "actions": [] }
```

You fill in the actions in that same block.

> ⚠ **Edit the `_hdr.json` , not the full `.json` .** When `BZBusRoute_<name>_hdr.json` and `BZBusRoute_<name>_wp.csv` both exist —which is the normal case, the wizard generates them— the server loads **that pair**, and the full `.json` is left only as a backup. Events you write in the `.json` have no effect. *If you placed several marks:* the first one in the `Events` list is your first NUMPAD 4, the second one is your second, and so on.

Step 4 — Hook the sequence from there. In that same JSON, you add (or edit) the `Events` block, pointing to that wp:

```
"Events": [
  {
    "wp": 88,
    "trigger": { "type": "wp_reached", "wp": 88 },
    "actions": [
      { "verb": "freeze_vehicle" },
      { "verb": "play_sound", "value": "MiVoz_SoundSet", "delay": 0.5 },
      { "verb": "unfreeze_vehicle", "delay": 3.0 }
    ]
  }
]
```

> The `"wp": 88` **is the hook**: it connects your mark (the NUMPAD 4) with what happens there. Change `88` to the `wp` of your mark. Each action can carry a `delay` in seconds, **counted from when the event fired** (not from the previous action).

> ⚠ **Every verb is undone by its own pair, and a `delay` on its own stops nothing.** `delay` holds *that* action back N seconds; it doesn't brake the vehicle. A wait is always two actions: one that stops the

vehicle and its opposite with the `delay` on it.

```
freeze_vehicle   <->  unfreeze_vehicle     (pins the vehicle: brake + handbrake)
stop_route       <->  resume_route         (pauses the route's advance)
```

Mixing them leaves the vehicle pinned forever: `resume_route` **does not unfreeze** what `freeze_vehicle` froze. While it's frozen the driving brain doesn't run, so the recording doesn't interfere with the wait.

Step 5 — Deploy and test. You save the file and **relaunch the route from the Player** (LOAD & SPAWN): that rereads the `_hdr.json`. *(A vehicle that was already running, or that relaunches by itself, keeps using the previous version of the file.)* The bus reaches wp 88 → **brakes** → **the message plays** → **waits 3s** → **continues.** That's your first sequence. 🎉

> *For more actions (dropping bots, starting the engine, etc.) and more triggers (when a player approaches, when it takes damage), keep reading — the complete list is below.*

10.2 — How it works

When you recorded, you tapped NUMPAD 4 at the important points. Each one ended up as an **event node**. Then you hang **actions** on it: "*when the vehicle gets here, do this*".

Each event has two parts:

- **When it fires** (the *trigger*): on reaching a waypoint, when a player approaches the vehicle or a point on the route, when a player gets in, when the vehicle takes damage, X seconds after the vehicle appears.
- **What it does** (the *actions*, the "verbs"): each one with an optional delay (`delay` , in seconds **from when the event fired**) to choreograph them.

> ⚠️ **The event does NOT force a stop.** Marking a NUMPAD 4 while **stopped** creates a real **stop** (`isStop`) — that's what you want for a taxi. But an event is just "*trigger → actions*": you can hang it on **any** waypoint (by editing the `Events` of the `_hdr.json` — §6B, Appendix A) and have it fire **mid-cruise, without braking** — to turn on lights, honk, load the trunk or leave a line in the log as it passes. And the triggers that are **not** `wp_reached` don't even depend on where the vehicle is.

Examples:

- *Taxi (real stop):* at each NUMPAD 4 you hang `stop_route` → `play_sound` (`delay` 0.5, "a passenger got on") → `resume_route` (`delay` 3). It stops, picks up a passenger and continues.
- *Sets off when someone gets in:* two events. One at wp 0 with `freeze_vehicle` and `stop_engine` (with a `delay` that falls after the startup wait, for example 10 s): the bus waits **with the engine off**. Another one with the `player_enter_vehicle` trigger and `unfreeze_vehicle` : when a player sits down, the bus **leaves on its own** (the engine starts by itself when the vehicle is released).
- *Change the inventory en route (`add_cargo`):* on reaching the depot, it puts boxes/supplies in the trunk — without braking if you hang it mid-cruise.

- *Make it fragile in the ambush (`set_vehicle_mortality` with `"mortal"`):* it comes unbreakable; at the hot spot you make it **destructible** so the player can blow it up.
- *The vehicle as a reward (`set_vehicle_parts` with `"unlocked"`):* it comes **tamper-proof**: while the framework drives it, nobody can remove its wheels or parts. When the player earns it, you unlock it. If the driver dies and the vehicle is left as loot, the framework unlocks it on its own.
- *Accident (mission hook):* you hang `freeze_vehicle` and `stop_engine` (`delay` 1). The vehicle stays stopped with the engine off until another event releases it with `unfreeze_vehicle`. *(`stop_engine` alone, without freezing, lasts half a second: the framework starts the engine again.)*

(Conditional branching —"if X happens, do Y"— doesn't live here: events have no conditions that an admin can use. That's handled by the Quests system, which is coupled separately. See §11.)

10.3 — Complete reference: triggers, verbs and conditions

Everything the `Events` list accepts, taken from the code of this version. It works the same with both brains; where the effect changes, each entry says so in its **Brain 1.2** line.

The basics: format, `delay` and when they are checked

An **event** is a "when" plus a "what": a **trigger** (what sets it off) and a list of **actions** (the verbs). Events go in the `Events` list of the route header, `BZBusRoute_<name>_hdr.json`. If the `_hdr.json` and the `_wp.csv` exist (the normal case: the Wizard generates them), the server loads that pair and **the full `.json` is not read**: an event written there has no effect.

The Wizard already leaves one event for each NUMPAD 4 you pressed, with the trigger set and the action list empty:

```
{ "wp": 88, "trigger": { "type": "wp_reached", "wp": 88 }, "actions": [] }
```

Event format

Field	Type	If missing	What it is
`wp`	integer	`0`	the event's waypoint. If there is no `trigger`, it is the point where it fires
`trigger`	object	`wp_reached` on `wp` is used	when it fires (see *Triggers*, below)
`actions`	list of actions	the event does nothing	what it does (see *The verbs, one by one*, below)

`trigger` fields:

Field	Type	Unit	If missing	Used by
`type`	text	—	empty = `wp_reached`	all
`wp`	integer	waypoint index	`0`	`wp_reached`, `player_near_waypoint`
`radius`	decimal	meters	`0` (= never fires)	`player_in_radius`, `player_near_waypoint`
`threshold`	decimal	health fraction, 0 to 1	`0` (= never fires)	`vehicle_health_below`
`seconds`	decimal	seconds	`0` (= fires right away)	`timer`

Fields of an action (an action uses only the ones that apply to it; the rest are ignored):

Field	Type	Unit	If missing	Used by
`verb`	text	—	the action does nothing and leaves a warning in the log	all
`delay`	decimal	seconds	`0` = immediate	all
`msg`	text	—	empty	`log_event`, `ui_broadcast`
`format`	text	—	`"toast"`	`ui_broadcast` (where: `"toast"`, `"chat"` or `"both"`)
`value`	text	—	empty	`set_vehicle_mortality`, `set_driver_mortality`, `set_vehicle_parts`, `set_var`, `play_sound`, `spawn_guard`, `ui_broadcast`
`var`	text	—	empty	`set_var`
`items`	list of `{ "cls": text, "qty": integer }`	—	empty list	`add_cargo`
`count`	integer	bots	`1`	`spawn_guard`
`faction`	text	—	`"Raiders"`	`spawn_guard`
`loadout`	text	—	`"BanditLoadout"`	`spawn_guard`
`radius`	decimal	meters	`8` (`guard_stop`) · `150` (`ui_broadcast` with `"near"`)	`guard_stop`, `ui_broadcast`
`holdS`, `clearS`, `maxS`, `boardS`, `unreachableS`	decimal	seconds	`20`, `5`, `120`, `30`, `15`	`guard_stop`
`slot`	integer	—	`-1`	reserved: no verb reads it today
`fvalue`	decimal	—	`0`	reserved: no verb reads it today

> Numbers go **without quotes** and text goes **with quotes**. Verb and trigger names are compared as exact text: `Freeze_Vehicle` or `freeze-vehicle` do not exist. What happens if a number comes in quotes or the JSON has a syntax error: *(not verified)*; check that the `[EVT-LOAD]` line for each event appears when the route loads.

Example: an event with two delayed actions

```
"Events": [
  {
    "wp": 120,
    "trigger": { "type": "wp_reached", "wp": 120 },
    "actions": [
      { "verb": "freeze_vehicle" },
      { "verb": "play_sound", "value": "MiVoz_SoundSet", "delay": 1.0 },
      { "verb": "unfreeze_vehicle", "delay": 8.0 }
    ]
  }
]
```

What happens: on reaching wp 120 the vehicle locks in place (second 0); at second 1 the audio plays; **at second 8** it is released and carries on. Not at second 9: each `delay` counts from the moment the event fired.

How `delay` works

- **It counts from when the event fires**, not from the previous action. Actions **are not chained**.
- Actions without `delay` (or with `0` or a negative value) run at the same instant the event fires, in list order.
- Delayed actions run when their time comes: **the order is set by the number, not by the position in the list**. An action without `delay` written at the end runs before a delayed one written above it. Two actions with the same `delay`: which order they come out in *(not verified)*.
- A delayed action acts on whatever vehicle exists when its turn comes. If there is no vehicle anymore, it does nothing.
- If the vehicle despawns or respawns first, the pending delayed actions are cancelled. *(Whether all pending ones are cancelled or only one: not verified.)*
- Nothing else cancels them: a delayed `unfreeze_vehicle` runs even if something else happened in between (for example, a `guard_stop` halfway through).

When triggers are checked

- **Every 0.5 seconds**, per vehicle, going through the `Events` list in order. Several events can fire in the same check.
- They are also checked with the vehicle frozen, braked by `stop_route`, paused from the panel, during the start wait (the *spawn-hold*) and even with the route finished, as long as the vehicle is still in the world.
- They are **not** checked while the driver or the crew are getting into the vehicle (the animated boarding, including the `guard_stop` re-boarding). Checking resumes when they finish boarding.
- If the driver dies and nobody can drive, the framework releases the vehicle as loot and its events stop being checked.
- **They work the same with both brains.** The event check runs before the point where the runner chooses between brain 1.2 (the default) and 1.1. What changes in some verbs is the *effect*: it is in each entry, in the **Brain 1.2** line.

Once per run; re-armed on every spawn

- **Each event fires only once per run.** If the trigger is met again, nothing happens.
- **Every time the vehicle spawns** (respawn at the end of the line, relaunching it from the Player, a spawn retry, a respawn from the panel or through the API) **all events are re-armed**, and everything the verbs changed is reset as well: the vehicle unfrozen and without `stop_route`, vehicle and driver mortality and the parts lock back to what the header says (`VehicleInvincible`, `DriverInvincible`, `VehiclePartsLocked`), the `set_var` variables empty and the `timer` clock at zero.
- **The automatic respawn reuses the route already in memory.** If you edit the `_hdr.json`, the change only takes effect when the route is loaded again: relaunch it from the Player (that does re-read the file) or restart the server.

What you see in the server log (RPT)

- **When the route loads**, one line per event: `[EVT-LOAD] Event <n> wp=<wp> actions=<count> trigger=type='<type>' verb0='<first verb>'`. If the trigger type does not exist: `evento <n> wp=<wp>: trigger '<type>' desconocido -> nunca dispara (validos: ...)` (= "event : unknown trigger '' -> never fires (valid: ...)").
- **When it fires**, one line per action: `[EVENT <n> @ wp <m>] <verb>: ...`. `<n>` is the position of the event in the list, **starting at 0**. These lines **do not carry the route name**: with several vehicles running, they do not say which one they belong to.
- **Misspelled verb:** `[EVENT <n>] verbo no implementado todavia: '<verb>'` (= "verb not implemented yet"). It appears **when the event fires**, not when the route loads.

The verbs (summary)

All of them also accept `delay`.

Verb	What it does	Fields
`add_cargo`	puts items in the trunk	`items` (`cls` , `qty`)
`log_event`	writes a line in the server log	`msg`
`ui_broadcast`	a notice, for admins only by default; also everyone, those nearby or the passengers, on screen or in chat	`msg` , `value` , `format` , `radius`
`play_sound`	3D sound attached to the vehicle, once	`value` (SoundSet name)
`freeze_vehicle`	locks the vehicle in place: full brake + handbrake, no progress	—
`unfreeze_vehicle`	releases what `freeze_vehicle` locked	—
`stop_route`	stops progress with full brake (no handbrake, engine running)	—
`resume_route`	lifts the `stop_route`	—
`start_engine`	starts the engine	—
`stop_engine`	stops the engine (only lasts with the vehicle frozen)	—
`set_vehicle_mortality`	vehicle destructible or indestructible	`value` (`"mortal"` or anything else)
`set_driver_mortality`	driver mortal or invincible	`value` (`"mortal"` or anything else)
`set_vehicle_parts`	vehicle tamper-proof (nobody can remove its parts) or with removable parts	`value` (`"unlocked"` or anything else)
`repair_vehicle`	brings the vehicle's overall health to maximum	—
`refuel`	fills the tank	—
`drain_fuel`	empties the tank	—
`lights_on`	turns the headlights on	—
`lights_off`	turns the headlights off	—
`horn`	a short honk (0.6 s)	—
`set_var`	stores a run variable (**nothing reads it today**)	`var` , `value`
`spawn_guard`	creates armed bots standing beside the vehicle	`value` (class), `count` , `faction` , `loadout`
`dismount_guard`	dismounts the crew (`Crew`) and leaves it patrolling an 8 m ring	—
`guard_stop`	stop with full escort: brakes, dismounts the crew, waits, boards it again and moves on	`radius` , `holdS` , `clearS` , `maxS` , `boardS` , `unreachableS`
`despawn_vehicle`	deletes vehicle, driver and bots; does not respawn	—

Verb	What it does	Fields
`lock_seat` · `unlock_seat` · `eject_passenger`	**reserved: they do nothing** (only a warning in the log)	(`slot` , unused)

The verbs, one by one

`add_cargo`

Puts items in the vehicle's trunk. What **stacks** (the item itself says so: its maximum quantity is greater than 1) is created as **a single item** with `qty` units, capped at that item's maximum. What **does not stack** (weapons, tools, wheels) is created **`qty` times**. Whatever does not fit is **discarded**: it never drops anything on the ground.

- **Fields:** `items` , a list of `{ "cls": "<classname>", "qty": <integer> }` . If `qty` is missing or less than 1, it counts as 1.
- **If it fails:** a classname that does not exist (or a full trunk) leaves `[CARGO] No se pudo crear '<cls>' (baul lleno o classname invalido?)` (= "could not create '' (trunk full or invalid classname?)") and moves on to the next item. If you asked for more than it stacks: `se pidieron N y la pila topa en M` (= "N were requested and the stack caps at M"). If the trunk fills up with non-stackable items: `entraron K de N (baul lleno). El resto se descarta` (= "K of N fit (trunk full). The rest is discarded"). Without `items` or with an empty list, it creates nothing.
- **Example:**

```
{ "verb": "add_cargo", "items": [ { "cls": "Rag", "qty": 6 }, { "cls": "Hammer", "qty": 2 } ] }
```

- **Pitfalls:** no nesting: a weapon with a magazine attached shows up empty; add the magazine as a separate item. For items whose "quantity" is not a count (a bottle measures liquid), `qty` is read as the item's quantity, not as a number of bottles *(not verified in game)*. How ammunition and magazines behave with `qty` *(not verified)*.
- **Brain 1.2:** yes, it does not depend on the brain.

`log_event`

Writes a line with your message in the server log (RPT). It is not visible in the game. Use it to mark in the log that the route went past a point.

- **Fields:** `msg` (text). Without `msg` , the line comes out empty.
- **What appears:** `[EVENT <n> @ wp <m>] log_event: <msg>` .
- **Example:**

```
{ "verb": "log_event", "msg": "The convoy crossed the bridge" }
```

- **Brain 1.2:** yes.

`ui_broadcast`

Sends your message **to whoever you choose**, on screen or in chat. Without `value` it goes to admins only: to tell players something, choose the recipient. What each one sees, in §13.3.

- **Fields:** `msg` (text); `value`, to whom: `"all"` (everyone connected), `"near"` (within `radius` meters of the vehicle; without `radius`, 150 m), `"passengers"` (seated in the vehicle), `"admins"` or empty (the admins); `format`, where: `"toast"` (on screen, default), `"chat"` or `"both"`.
- **Example** (to everyone, on screen, when leaving the terminal):

```
{ "verb": "ui_broadcast", "value": "all", "format": "toast", "msg": "The bus left the terminal" }
```

- **Pitfalls:** a misspelled `value` goes **to admins only** (on purpose: a typo never shows players something that was meant for you); a misspelled `format` shows on screen. Each player receives it once even if they meet two conditions (an admin riding as a passenger). The RPT says how many it reached: `[EVENT n @ wp X] ui_broadcast (all → 3 jugadores, toast): <msg>`.
- **Brain 1.2:** yes.

`play_sound`

Plays a sound **once**, in 3D, attached to the vehicle (it moves with it). You declare the sound yourself in your addon (`CfgSoundShaders` + `CfgSoundSets`).

- **Fields:** `value` = the **SoundSet** name. Without `value`, it does nothing (it does not even leave a line in the log).
- **Example:**

```
{ "verb": "play_sound", "value": "MiVoz_SoundSet" }
```

- **Pitfalls:** the sound is launched from the server; the code itself warns that it **may not reach the clients** and asks for it to be checked in game *(not verified)*. What happens if the SoundSet does not exist *(not verified)*.
- **Brain 1.2:** yes, it does not depend on the brain.

`freeze_vehicle`

Locks the vehicle where it is: every tick it applies full brake and handbrake, the route does not advance and **the driving brain does not run** (neither 1.2 nor 1.1). This is the "hard" stop. While it is frozen, the framework **does not restart the engine**: it is the only state where `stop_engine` lasts.

- **Fields:** none.
- **Pair:** `unfreeze_vehicle`. **`resume_route` does not release it.**
- **Example:**

```
{ "verb": "freeze_vehicle" }
```

- **Pitfalls:**
  - During the start wait (spawn-hold) the freeze is not in charge yet: that phase runs first (engine on, brake applied) and the vehicle switches to driving anyway; only then is it locked in place. An event

at the start of the route with `freeze_vehicle` works, but a `stop_engine` at the same time gets undone (see `stop_engine`).
    - `guard_stop` with the hard stop (without the 1.2 brain) uses the **same** freeze: when it finishes it releases it, even if you froze it separately. With 1.2 it brakes smoothly and does not touch it.
    - Events are still checked with the vehicle frozen: a `timer` or a `player_in_radius` can release it.
- **Brain 1.2:** yes. With the vehicle frozen, the 1.2 actuator steps aside and brake and handbrake are applied.

`unfreeze_vehicle`

Releases the freeze of `freeze_vehicle`: it takes off the handbrake and the vehicle resumes the route.

- **Fields:** none.
- **Pair:** `freeze_vehicle`. **It does not lift a `stop_route`**: if there was also a `stop_route`, the vehicle stays stopped until a `resume_route`.
- **Example:**

```
{ "verb": "unfreeze_vehicle", "delay": 10 }
```

- **Pitfalls:** it also releases the freeze of a **hard** `guard_stop` in progress: the vehicle leaves with the guards outside.
- **Brain 1.2:** yes.

`stop_route`

Stops progress: every tick it applies **full brake**, **no handbrake**, with the **engine running** (if it is off, it starts it). The driving brain does not run while it lasts. This is the "soft" stop.

- **Fields:** none.
- **Pair:** `resume_route`. **`unfreeze_vehicle` does not lift it.**
- **Example:**

```
{ "verb": "stop_route" }
```

- **Pitfalls:** since it restarts the engine every tick, **it is no use for waiting with the engine off** (for that: `freeze_vehicle` + `stop_engine`).
- **Brain 1.2:** yes, same as `freeze_vehicle` (the 1.2 actuator steps aside).

`resume_route`

Lifts the `stop_route` and the vehicle carries on.

- **Fields:** none.
- **Pair:** `stop_route`. **It does not unfreeze** what `freeze_vehicle` froze: if you mix up the pairs, the vehicle stays locked in place.
- **Example:**

```
{ "verb": "resume_route", "delay": 3 }
```

- **Pitfalls:** it does not reopen a route that has already finished.
- **Brain 1.2:** yes.

`start_engine`

Starts the engine if it is off.

- **Fields:** none.
- **Pair:** `stop_engine`.
- **Example:**

```
{ "verb": "start_engine" }
```

- **Pitfalls:** you almost never need it: the framework starts the engine on its own every tick in every state except frozen. After an `unfreeze_vehicle` the engine starts anyway.
- **Brain 1.2:** yes.

`stop_engine`

Stops the engine if it is running.

- **Fields:** none.
- **Pair:** `start_engine`.
- **Example** (wait with the engine off; the `delay` lets the `freeze_vehicle` take effect first):

```
{ "verb": "stop_engine", "delay": 2 }
```

- **Pitfalls: on its own, it lasts half a second.** Every tick the framework restarts the engine: while driving, with `stop_route`, while paused and during the start wait. **It only stays off with the vehicle frozen** (`freeze_vehicle` first). At the start of the route, give it a `delay` that falls after the start wait (`SpawnHoldSeconds` plus the route load time): during that wait the engine is restarted even if the vehicle is frozen. *(This last point comes from the order of the code; it has not been tested in game.)*
- **Brain 1.2:** same as in 1.1: the automatic engine start runs before the brain is chosen.

`set_vehicle_mortality`

Switches the vehicle between **destructible** and **indestructible**. The framework reapplies it every tick, so the change holds. When indestructible, it also repairs itself if it gets ruined.

- **Fields:** `value`. Only `"mortal"` (exactly, in lowercase) makes it destructible. **Anything else, or leaving out `value`, makes it indestructible.** For clarity, use `"invincible"`.
- **Pair:** the same verb with the other value. The initial state of each run comes from `VehicleInvincible` in the header (default `true`).
- **Example:**

```
{ "verb": "set_vehicle_mortality", "value": "mortal" }
```

- **Pitfalls:** `"Mortal"` with a capital letter leaves it **indestructible**. `vehicle_health_below` only fires with the vehicle destructible.
- **Brain 1.2:** yes.

`set_vehicle_parts`

Switches the vehicle between **tamper-proof** (players can't remove its wheels, doors, battery, spark plug, radiator or any other part) and **with removable parts**. With the vehicle stopped (at a stop, on a plateau, at the end of the route), the game lets anyone remove its parts like on any car: two wheels fewer and it is stuck forever; and "remove wheel" competes with "get in" at the driver's door. Tamper-proof, the prompt doesn't even show up. **It doesn't touch the cargo**: what is in the trunk stays accessible, and getting in, getting out and opening doors work as usual.

- **Fields:** `value`. Only `"unlocked"` (exactly, in lowercase) makes the parts removable. **Anything else, or leaving out `value`, keeps it tamper-proof.** For clarity, use `"locked"`.
- **Pair:** the same verb with the other value. The initial state of each run comes from `VehiclePartsLocked` in the header (default `true`).
- **Example** (the vehicle is the reward):

```
{ "verb": "set_vehicle_parts", "value": "unlocked" }
```

- **Pitfalls:** `"Unlocked"` with a capital letter keeps it **tamper-proof**. If the driver dies and the vehicle is left as loot, the framework unlocks it on its own. A vehicle mod that changes its parts rules without honoring the base class ones can bypass the lock.
- **Brain 1.2:** yes.

`set_driver_mortality`

The same for the **driver** (whoever is driving at that moment). It does not touch the crew (`Crew`) or the `spawn_guard` bots.

- **Fields:** `value`, same as the previous verb: `"mortal"` makes the driver mortal; anything else, invincible.
- **Pair:** the same verb with the other value. Initial state: `DriverInvincible` in the header (default `true`).
- **Example:**

```
{ "verb": "set_driver_mortality", "value": "mortal" }
```

- **Pitfalls:** for another crew member to take the wheel when the driver is killed (`Drivers` block in the header), the driver has to be mortal.
- **Brain 1.2:** yes.

`repair_vehicle`

Brings the vehicle's **overall health** to maximum.

- **Fields:** none.
- **Example:**

```
{ "verb": "repair_vehicle" }
```

- **Pitfalls:** the code only touches the overall health. Parts (wheels, battery, radiator…) have their own health; whether it repairs them, and whether it repairs the damage zones of the engine or the body, *(not verified)*.
- **Brain 1.2:** yes.

`refuel`

Fills the fuel tank (it asks for a huge amount and the game trims it to the tank capacity).

- **Fields:** none.
- **Pair:** `drain_fuel`.
- **Example:**

```
{ "verb": "refuel" }
```

- **Brain 1.2:** yes.

`drain_fuel`

Empties the tank. The engine eventually stalls for lack of fuel.

- **Fields:** none.
- **Pair:** `refuel`.
- **Example:**

```
{ "verb": "drain_fuel" }
```

- **Pitfalls:** the framework keeps trying to start the engine every tick. With brain 1.2, a vehicle that does not move forward is taken as stuck and AutoRecovery kicks in; whether that ends up closing the route *(not verified)*. If you want a vehicle "out of fuel" that sits still and tidy, freeze it.
- **Brain 1.2:** the verb runs; the brain's reaction *(not verified)*.

`lights_on`

Turns the headlights on.

- **Fields:** none.
- **Pair:** `lights_off`.
- **Example:**

```
{ "verb": "lights_on" }
```

- **Pitfalls:** it coexists with `LightsMode` in the header, and the verb **wins**:
  - With both brains, a `lights_on` / `lights_off` **overrides** `LightsMode` **until the next spawn**: after that, on each spawn, `LightsMode` decides again.
  - The framework sends a network pulse so that other players see the beam steadily, with the 1.2 brain too.
  - For the lights to be visible, the vehicle needs a battery and headlights installed.

`lights_off`

Turns the headlights off. Same rules as `lights_on`.

- **Fields:** none.
- **Pair:** `lights_on`.
- **Example:**

```
{ "verb": "lights_off" }
```

- **Brain 1.2:** see `lights_on`.

`horn`

A short honk: the horn sounds for 0.6 seconds and cuts off by itself.

- **Fields:** none (if you set `value`, it is ignored).
- **Example:**

```
{ "verb": "horn" }
```

- **Brain 1.2:** yes.

`set_var`

Stores a text variable for the run (`var` = `value`).

- **Fields:** `var` (name), `value` (value). Both are text.
- **Example:**

```
{ "verb": "set_var", "var": "phase", "value": "ambush" }
```

- **Pitfalls: nothing reads these variables today.** The only part of the code that reads them (the `var_equals` condition, see *Conditions*, below) is not called from anywhere. In practice the verb only leaves the line `set_var: phase = ambush` in the log. The variables are cleared on every spawn.
- **Brain 1.2:** it runs all the same; it has no effect with either brain.

`spawn_guard`

Creates armed bots **standing on the right side** of the vehicle: the first one at 2.5 m and each of the next ones 1.5 m further out. They stay in a group of their own, with the faction you set and the loadout you set. **They do not get in** the vehicle or follow it.

- **Fields:**
  - `value`: the bot class. If missing, `eAI_SurvivorM_Boris`.
  - `count`: how many. If missing or 0, 1.
  - `faction`: the eAI faction. If missing, `Raiders` (hostile to the player, friendly to the driver).
  - `loadout`: the Expansion loadout they are equipped with. If missing, `BanditLoadout`.
- **If it fails:** a class that cannot be created leaves `[spawn_guard] no se pudo crear '<class>'` (= "could not create '"") and moves on to the next one.
- **Example:**

```
{ "verb": "spawn_guard", "count": 3, "faction": "Raiders", "loadout": "BanditLoadout" }
```

- **Pitfalls:** they are not the crew (`Crew`): `dismount_guard` and `guard_stop` do not command them. The code does not make them invincible. They are **deleted** when the vehicle despawns or respawns.
- **Brain 1.2:** yes, it does not depend on the brain.

`dismount_guard`

Dismounts the **crew** (the `Crew` bots) that is seated, except the one driving. They get out with the animation (they open the door), become **mortal**, form a group of their own and **patrol an 8 m ring** around where the vehicle is, on a 10 m leash. If they see a threat they fight; once things calm down they return to the ring.

- **Fields:** none (the radius is fixed: 8 m).
- **Example** (first lock the vehicle in place, then dismount):

```
{ "verb": "dismount_guard", "delay": 3 }
```

- **Pitfalls:**
  - **The vehicle has to be stopped**: DayZ does not let anyone get out while the vehicle is moving. Use it after a `freeze_vehicle`, with `delay`.
  - **They do not come back on their own.** There is no verb to board them again: if you release the vehicle, it leaves without them. The full cycle (get out, wait and board again) is `guard_stop`.
  - They are **deleted** with the vehicle when it despawns or respawns, even if they are already out.
  - The group's faction is that of the **first** `Crew` bot (or `Raiders`).
- **Brain 1.2:** yes, it does not depend on the brain (but it needs the vehicle frozen).

`guard_stop`

The **stop with full escort** in a single verb:

1. **Brakes.** With the 1.2 brain, a **smooth stop**: it follows the line and brakes until it stands still at a point a few meters ahead (if the take has a recorded stop there, at that stop), and the deployment starts with the vehicle still. Without 1.2, on a motorbike, or if the smooth stop is not possible at that moment (a

recovery in progress, the end right ahead), it **freezes** the vehicle and waits for it to drop below 3 km/h (checks every 0.5 s, up to 10 s; after that it deploys anyway).
2. **Deploys**: same as `dismount_guard`, but with a ring of `radius` meters (leash 2 m further out).
3. **Waits** and exits on the first of these three:
   - **area clear**: `holdS` seconds have passed **and** there have been `clearS` seconds in a row without combat;
   - **threat out of reach**: there is combat but no target closer than `radius` + 5 m, for `unreachableS` seconds;
   - **cap**: `maxS` seconds have passed, no matter what.
4. **Fall back**: the survivors walk back to their seats, to wherever the vehicle is.
5. **Boarding**: waits until all survivors are seated, or `boardS` seconds at most. Whoever did not make it **stays behind**.
6. **Releases the stop** and the route carries on from there.

"Combat" means: any crew bot with a high threat level (the same threshold eAI uses to start fighting), or the group in combat.

- **Fields** (all decimal; 0 or negative = the default): `radius` 8 m · `holdS` 20 s · `clearS` 5 s · `maxS` 120 s · `boardS` 30 s · `unreachableS` 15 s.
- **Example:**

  ```
  { "verb": "guard_stop", "radius": 10, "holdS": 30, "clearS": 5, "maxS": 180, "boardS": 40,
  "unreachableS": 15 }
  ```

  `{ "verb": "guard_stop" }` on its own already works with the defaults.
- **What you see in the log:** `[MISION] wp N: guard_stop · parada SUAVE (siguiendo la linea hasta el hold) · radio ...` (= "smooth stop (following the line up to the hold) · radius ..."; or `parada DURA (congelado)` = "hard stop (frozen)"), `despliegue X/Y` (= "deployment X/Y"), `combate -> retengo` (= "combat -> holding"), `zona limpia tras ... -> repliegue` (= "area clear after ... -> fall back") (or `amenaza fuera de la correa` = "threat outside the leash", or `tope de` = "cap of"), `embarcados X/Y ... -> sigue` (= "boarded X/Y ... -> moves on").
- **Pitfalls:**
  - It needs a crew declared in `Crew` in the header. Without `Crew` nobody gets out: the vehicle stays stopped for about `holdS` seconds and moves on *(from the code; not tested in game)*.
  - With the **hard** stop it uses the **same** freeze as `freeze_vehicle`: when it finishes it releases it even if you froze it separately, and an `unfreeze_vehicle` in between releases the vehicle with the guards outside. With the smooth stop (1.2 brain) the freeze is not touched.
  - **Where to put it** *(measured)*: a `guard_stop` where the take doesn't brake, right before a final climb, leaves the vehicle without the take's momentum to climb it (an 8-10 % grade in the last meters got it stuck 6 times out of 7). Put it on flat ground and away from the end, or better: **mark the stop with NUMPAD 4 while recording** and the vehicle brakes as you did.
  - While the crew re-boards, events are not checked.
  - The deployed group's faction is that of the first `Crew` bot; the `faction` field of the others is not used. With `Deploy.IgnoreInfected` (default `true`), the guards and the infected ignore each other.
- **Brain 1.2:** yes (it works on the same freeze).

`despawn_vehicle`

End of the story: after 50 ms it deletes the vehicle, the driver, the crew (even if already out) and the `spawn_guard` bots, cancels anything pending (delayed actions, respawn) and sends `BZ AutoDrive vehicle stopped.` to the admins. **It does not respawn.**

- **Fields:** none.
- **Example:**

```
{ "verb": "despawn_vehicle", "delay": 60 }
```

- **Pitfalls:** there is no going back: the route does not relaunch by itself. For mods that use the API: the runner is not removed from the registry.
- **Brain 1.2:** yes (it also stops brain 1.2).

`lock_seat` · `unlock_seat` · `eject_passenger` **(reserved)**

They do nothing. The `slot` field exists for them, but the dispatcher has no code for these verbs: when the event fires, they leave `verbo no implementado todavia` (= "verb not implemented yet") in the log and move on. Any other name that is not in this reference behaves the same way.

The triggers

Trigger	Fires when…	Fields
`wp_reached` (or `waypoint`, or no trigger)	the vehicle reaches a waypoint (or passes it)	`wp`
`player_in_radius`	a real player is less than `radius` meters **from the vehicle**	`radius`
`player_near_waypoint`	a real player is less than `radius` meters **from a fixed waypoint**	`wp`, `radius`
`player_enter_vehicle`	a real player is inside the vehicle	—
`vehicle_health_below`	the vehicle's overall health drops below `threshold`	`threshold`
`timer`	`seconds` seconds have passed since the vehicle spawned	`seconds`

For all of them: they are checked every 0.5 s, **fire only once per run** and **re-arm on every spawn** (see *The basics*). A type that does not exist (or is misspelled) gives a warning when the route loads and **never fires**. A "real player" is a connected player: eAI bots (the driver, the crew, the guards) do not count.

`wp_reached`

Fires when the vehicle's waypoint index is **greater than or equal to** the one you ask for. Because it is "greater than or equal", if the vehicle skips points (an AutoRecovery relocation) the event still fires as soon as it passes them. It is the trigger the Wizard writes. `waypoint` is another name for the same thing, and an event **without** a `trigger` (or with an empty `type`) is also a `wp_reached`.

- **Fields:** `wp` (integer, waypoint index). If the trigger's `wp` is 0 or missing, the event's `wp` is used.

- **Once / re-arming:** once per run; it re-arms on every spawn.
- **Active from:** the first tick in which events are checked. An event at wp 0 (or with no `wp` and no trigger) **fires as soon as the vehicle spawns**, during the spawn hold.
- **Example:**

```
{ "wp": 340, "trigger": { "type": "wp_reached", "wp": 340 }, "actions": [ { "verb": "horn" } ] }
```

- **Pitfalls:**
  - If you want an event at wp 0 using the trigger's `wp`, you also have to put `"wp": 0` on the event: a `wp` of 0 in the trigger does not count, and the event's `wp` is used instead.
  - With the **1.2 brain** the index is the brain's cursor (the nearest point of its own copy of the route). That copy discards the points where the recording goes backwards (glitches). On a clean route the numbers match; if the RPT says `[Route] saneo de retrocesos: N` (= "backtrack cleanup: N") with N greater than 0 when the route loads, the points after a discard are shifted and **the event may fire a few points later** *(by how much: not verified)*.

`player_in_radius`

Fires when **any real player** is `radius` meters or less **from the vehicle** (3D distance, height included). The circle moves with the vehicle.

- **Fields:** `radius` (decimal, meters). 0 or not set = never fires.
- **Once / re-arming:** once per run.
- **Example:**

```
{ "wp": 0, "trigger": { "type": "player_in_radius", "radius": 25 }, "actions": [ { "verb": "stop_route" }, { "verb": "resume_route", "delay": 10 } ] }
```

- **Pitfalls:** it also fires during the spawn hold, if someone is near the starting point.

`player_near_waypoint`

A **fixed zone** anchored to a waypoint of the route: it fires when any real player is `radius` meters or less from that waypoint's position, **wherever the vehicle is**. Useful for ambushes or for zones off the road.

- **Fields:** `wp` (integer: the waypoint that is the center; 0 is valid), `radius` (decimal, meters). A `wp` outside the route or a `radius` of 0 = never fires.
- **Once / re-arming:** once per run.
- **Example:**

```
{ "wp": 500, "trigger": { "type": "player_near_waypoint", "wp": 500, "radius": 60 }, "actions": [ { "verb": "spawn_guard", "count": 2 } ] }
```

- **Pitfalls:** here the number is the row of the route file as it is, without the 1.2 brain shift that can affect `wp_reached`.

`player_enter_vehicle`

Fires when **any real player is inside** the vehicle, in any seat.

- **Fields:** none.
- **Once / re-arming:** once per run: **only the first one who gets in**. If a player is already inside when events are checked, it fires right away.
- **Example** (wait with the engine off until someone gets in; the waiting event goes separately, see *Pitfalls*, below):

```
{ "wp": 0, "trigger": { "type": "player_enter_vehicle" }, "actions": [ { "verb": "unfreeze_vehicle" }, { "verb": "ui_broadcast", "msg": "A passenger got on" } ] }
```

`vehicle_health_below`

Fires when the vehicle's **overall health** (from 0 to 1) is **below** `threshold`.

- **Fields:** `threshold` (decimal, from 0 to 1; `0.5` = half). 0 or not set = never fires.
- **Once / re-arming:** once per run.
- **Example:**

```
{ "wp": 0, "trigger": { "type": "vehicle_health_below", "threshold": 0.6 }, "actions": [ { "verb": "freeze_vehicle" }, { "verb": "guard_stop" } ] }
```

- **Pitfalls: with the vehicle unbreakable (the default) it almost never fires**: the framework blocks its damage on every tick. Put `"VehicleInvincible": false` in the header, or a `set_vehicle_mortality` with `"mortal"` before it.

`timer`

Fires when `seconds` seconds have passed **since the vehicle spawned**. It does not count from when the vehicle starts driving: the clock includes the spawn hold (`SpawnHoldSeconds`), the loading of the route and the crew getting in.

- **Fields:** `seconds` (decimal, seconds). 0 = on the first check.
- **Once / re-arming:** once per run; the clock goes back to zero on every spawn. If the starting point was occupied and the runner waited, the clock counts from when the vehicle really spawned.
- **Example:**

```
{ "wp": 0, "trigger": { "type": "timer", "seconds": 600 }, "actions": [ { "verb": "despawn_vehicle" } ] }
```

- **Pitfalls:** since events are not checked while the crew gets in, a short timer can go off later than its exact second. Resolution: half a second.

Conditions ("if X happens, do Y")

In the code there is a **condition** class with these types: `AND`, `OR`, `NOT` (they combine other conditions), `player_in_radius`, `player_in_vehicle`, `vehicle_health_below`, `wp_reached`, `var_equals` (compares a variable from `set_var`), `engine_running`, `route_stopped` and `vehicle_frozen`. The function that evaluates them also exists.

But they are not used anywhere you can reach:

- **No part of the code calls that function.**
- **No field of the route JSON accepts a condition.** An event has only `wp`, `trigger` and `actions`; an action has no condition fields.
- The `check_once` verb (with `condition`, `on_true`, `on_false`) **was removed on 2026-06-11**: an action inside another action, or a condition inside another, broke the JSON load and the whole `Events` list did not load (not even the simple events). Since then the action is flat.

Conclusion: an admin cannot use conditions from the route JSON. If you write a `condition` key in an event, it is ignored *(not verified how the loader treats an unknown key)*. For the same reason, **`set_var` has no effect today**: nobody reads its variables. "If X happens, do Y" belongs to Expansion Quests.

Event pitfalls

1. **Edit the `_hdr.json`**, not the full `.json`. Then **relaunch the route from the Player**: the automatic respawn reuses the route in memory and does not see your changes.
2. **`delay` counts from when the event fires**, not from the previous action. `freeze` + `unfreeze` with `delay: 5` = 5 seconds stopped, even if you put other delayed actions in between.
3. **Each verb is undone by its pair.** `freeze_vehicle` ↔ `unfreeze_vehicle`; `stop_route` ↔ `resume_route`. **`resume_route` does not unfreeze** and **`unfreeze_vehicle` does not lift a `stop_route`**: mixing them leaves the vehicle stopped forever. `guard_stop` uses the same freeze as `freeze_vehicle` and releases it when it finishes.
4. **A `delay` on its own stops nothing.** A wait is always two actions: one that stops the vehicle and its pair with a `delay`.
5. **`stop_engine` on its own lasts only half a second.** The engine is started again on every tick unless the vehicle is **frozen**. Waiting with the engine off = `freeze_vehicle` + `stop_engine` (with a `delay` if it is at the start of the route); setting off = `unfreeze_vehicle` (the engine starts on its own). With `stop_route` the engine cannot be turned off.
6. **Without `value`, only admins see `ui_broadcast`** (`AdminSteamIDs`). For players, choose `"all"`, `"near"` or `"passengers"` (§13.3).
7. **`set_var` has no effect**: nobody reads the variables and there are no usable conditions (see *Conditions*, above).
8. **`vehicle_health_below` needs a destructible vehicle.** With `VehicleInvincible` set to `true` (the default) it takes no damage.
9. **`set_vehicle_mortality` and `set_driver_mortality`:** only `"mortal"`, exactly and in lowercase, makes them mortal. With no `value`, or with `"Mortal"`, they stay **invincible**. Same with `set_vehicle_parts`: only `"unlocked"`, exactly, unlocks it; with `"Unlocked"` it stays tamper-proof.

10. **Each event fires once per run.** `player_enter_vehicle` only sees the **first** passenger; `player_in_radius`, the first player who comes near. On every spawn everything re-arms.
11. **An event with no `wp` and no `trigger` fires when the vehicle spawns** (it is a `wp_reached` at wp 0).
12. **With the 1.2 brain, `wp_reached` can fire a few points later** on routes with recording glitches (a `[Route] saneo de retrocesos: N` line with N greater than 0 in the RPT).
13. **Lights:** a `lights_on` / `lights_off` overrides `LightsMode` until the next spawn, with both brains.
14. **`dismount_guard` does not put them back in.** If you release the vehicle it leaves without them, and when the vehicle despawns they are deleted. To get them out, wait and put them back in, use `guard_stop`. The vehicle has to be stopped for them to get out.
15. **The `spawn_guard` bots are not crew**: neither `dismount_guard` nor `guard_stop` gives them orders.
16. **Name errors show up at different moments:** a misspelled trigger gives a warning **when the route loads**; a misspelled verb gives a warning only **when the event fires**. Names are case-sensitive.
17. **While the driver or the crew are getting in, events are not checked.** With the vehicle frozen, braked or paused from the panel they **are** checked: a `timer` can fire while you have it paused.
18. **The `[EVENT ...]` log lines do not say which route they belong to.** With several vehicles, go by the event number and the wp.
19. **The reserved verbs** (`lock_seat`, `unlock_seat`, `eject_passenger`) and any name that is not in this reference **do nothing**.

---

11. Missions — BZ_AutoDrive + Quests *(the star integration)*

This is the most powerful piece of the framework: making an NPC-driven vehicle **part of a mission** — reinforcements that arrive, a convoy that escapes, an ambush on the road. For that, BZ_AutoDrive connects to **DayZ-Expansion-Quests**.

11.1 Why two systems? (the division of labor)

The key is that **each system does what it is good at**:

- **Quest** = the **bots** + the **mission logic**. It spawns live, armed bots, and it owns everything "mission-related": killing them gives a reward, progression, win/lose conditions.
- **BZ_AutoDrive** = the **vehicle**. It spawns it, **drives** it along your recorded route, and coordinates the bots getting in and out.
- **eAI** (Expansion's base AI) = the bot's basics: walking, getting in and out of a car.

> *Why doesn't the framework do everything?* A bot that the framework spawns on its own **has no mission logic** — killing it gives nothing (no reward, no progression). And without a Quest, the bots don't even "live" properly (they show up like mannequins). **Quest is the only owner of armed bots with a purpose.** That is why the two work together: Quest provides the people and the purpose; you provide the vehicle and the driving — the piece Quest is missing.

11.2 How a mission works, end to end

1. The player **accepts the quest** (it defines where and how many bots).
2. The bots **appear** when the player gets near the zone (they are "lazy": they materialize by proximity).

3. The framework **detects** those bots and **spawns the vehicle** at the start of your route.
4. Following the script: the bots **get in** (walking or already seated) and the NPC **drives** the route.
5. On arrival (or when a trigger fires), the bots **get out** — to deploy, to camp, to whatever the mission asks for.
6. When the mission ends, the vehicle is **cleaned up** (it does not keep driving around).

11.3 Example 1 — "The convoy that flees" *(validated in-game)*

The idea: there is a camp with 5 bots. If the player starts killing them, the survivors **escape in a vehicle**.

How it is built:

- In the **quest**: a camp with 5 armed bots at a terminal.
- In the **framework**: you record the route from the terminal to an escape yard, and mark it as "convoy that flees when 1 is killed".

What happens live:

1. The player arrives → the 5 bots appear and the vehicle (a Cobra) appears on its own at the terminal.
2. The player kills 1 → **the escape fires**: the 4 survivors **stop shooting** (they are "pacified"), **walk** to the vehicle and **get in**, with the door animation.
3. The NPC **drives** to the yard.
4. On arrival, it **brakes** and the 4 **get out** (animated door).

> The trigger is the quest's **kill count** (kill 1). The framework does not invent the logic — it *listens* to the quest and drives the vehicle.

11.4 Example 2 — "Intercept the convoy" *(ambush)*

The idea: an enemy convoy crosses the zone; the player ambushes it.

How it is built:

- In the **quest**: 3 bots at the start of the circuit.
- In the **framework**: a circuit route, marked as "ambush on taking damage", with the vehicle **destructible**.

What happens live:

1. The 3 bots start **already on board, armed**, and the NPC drives them around the circuit.
2. The player shoots at them → **any damage** (to the vehicle or to a bot) fires the ambush.
3. The vehicle **brakes**, waits until it has fully stopped, and the bots (+ the driver) **get out and camp**, hostile.

> *Design insight:* at its core, this ambush is a **generalized NUMPAD 4 marker** — a trigger ("on taking damage") that fires a sequence ("brake + get out + fight"). Today it comes pre-built as a mode; the plan is that you define it yourself in the events (§10), without touching code.

11.5 The vehicle lifecycle (no ghost cars left behind)

In a mission, the vehicle **must not keep driving around** when the mission is over. When the quest marks the objectives as complete, the framework **cleans up** the vehicle (despawns it and stops the driving). Other possible policies: leave it as **loot**, leave it in place for a chained objective, etc.

> *Example:* in the convoy that flees, when the player finally kills the 4 in the yard, the quest marks "completed" → the Cobra despawns on its own. No ghost car is left on the map.

11.6 What you need to build it

- **Dependency:** Quests, which already comes in the Expansion Bundle and which the mod requires anyway (§4).
- **Your part (framework):** you record the vehicle's route + configure it with the convoy mode and, if you want, the bots that travel from the start.
- **The quest's part:** you define the bots and the logic (reward, conditions) in the Expansion-Quests editor.
- **The hook:** the framework *listens* for when the quest starts and where its bots are, and from there it coordinates the vehicle.

> ⚠ **The mission vehicle has to be the one you recorded with.** The `VehicleClass` in the JSON is not a label: it is **the class you made the take with**. If you build the mission with a Cobra, the route has to be recorded with a Cobra — not with "something similar". Each take feeds the *envelope* of **that** class (§6C), and that is where the framework learns how hard that vehicle brakes and how much it turns.
>
> If you want the same scene with another vehicle, **record the route with that vehicle** and point the mission to the new route. It is faster than fighting with a borrowed take, and the result is faithful to the physics of the car the player is going to see.

11.7 — How it is written (config examples)

The framework's part is configured in the **route JSON** (`BZBusRoute*.json`). Here are the real examples of the two scenes. *(You define the bots and the reward separately, in the Expansion-Quests editor.)*

Scene 1 — convoy that flees. The vehicle waits at the terminal; when the quest detects that you killed 1 bot, the others get in and flee:

```
{
  "VehicleClass": "Star_APC_Cobra_white",
  "SpawnHoldSeconds": 600,
  "ConvoyMode": "flee_on_kill",
  "Crew": [],
  "Events": [],
  "Waypoints": [ /* your route: from the terminal to the yard */ ]
}
```

- `ConvoyMode: "flee_on_kill"` → turns on the mechanic (pacify → get in → drive → get out).
- `Crew: []` → empty on purpose: **the quest places the bots**, not the route.

- `SpawnHoldSeconds: 600` → the vehicle waits without moving until the escape fires.

Scene 2 — ambush. The bots start on board, armed; any damage fires the deployment:

```
{
  "VehicleClass": "x5mcompetition_orange",
  "ConvoyMode": "ambush_on_damage",
  "VehicleInvincible": false,
  "Crew": [],
  "Waypoints": [ /* the circuit */ ]
}
```

- `ConvoyMode: "ambush_on_damage"` → bots on board instantly + freeze + dismount on taking damage.
- `VehicleInvincible: false` → **key**: if the vehicle is unbreakable, it never takes damage and the ambush **does not fire**.

Bots that travel from the start (`Crew[]`). If you want the vehicle to spawn with bots inside (without depending on the quest), you list them:

```
"Crew": [
  { "cls": "eAI_SurvivorM_Boris", "seat": 1, "faction": "Raiders",
    "loadout": "BanditLoadout", "offsetRight": 2.0, "offsetForward": 0 }
]
```

- `seat`: 1+ (0 is the driver). `faction` / `loadout`: how it comes armed. `offsetRight/Forward`: where it spawns **outside** before getting in (so it does not end up inside the car's body).

A sequence of events at a point (`Events[]`). To attach actions to a NUMPAD 4 node — e.g. a stop with sound:

```
"Events": [
  {
    "wp": 88,
    "trigger": { "type": "wp_reached", "wp": 88 },
    "actions": [
      { "verb": "freeze_vehicle" },
      { "verb": "play_sound", "value": "MiVoz_SoundSet", "delay": 0.5 },
      { "verb": "unfreeze_vehicle", "delay": 3.0 }
    ]
  }
]
```

- `trigger` = when (here: on reaching wp 88). `actions` = what, in order, with `delay` (seconds) to set the timing.
- Available triggers: `wp_reached`, `player_in_radius` (with `radius`), `player_near_waypoint` (with `wp` and `radius`), `player_enter_vehicle`, `vehicle_health_below` (with `threshold` 0..1), `timer` (with `seconds`). Full reference in §10.3.

11.8 — The hook in code (how the framework "listens" to the quest)

The integration is server-side, in two pieces. **(1)** A `modded class MissionServer` that tells the framework when a quest starts:

```
modded class MissionServer {
    override void Expansion_OnQuestStart(ExpansionQuest quest) {
        super.Expansion_OnQuestStart(quest);
        BZBusService.GetInstance().OnQuestStart(quest);
    }
}
```

(2) The framework stores the quest ID and **polls**, because the bots are *lazy* (they appear only when the player gets near the camp):

```
void OnQuestStart(ExpansionQuest quest) {
    if (!quest) return;
    ExpansionQuestConfig qc = quest.GetQuestConfig();
    if (!qc) return;
    m_QuestCheckID = qc.GetID();                        // we store the ID
    GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.CheckQuestBots, 2000, true); // poll every
2s
}

void CheckQuestBots() {
    array<eAIQuestPatrol> patrols = new array<eAIQuestPatrol>();
    bool exists = ExpansionQuestModule.GetModuleInstance().QuestPatrolExists(m_QuestCheckID, patrols);
    int totalBots = 0;
    if (exists)
        for (int i = 0; i < patrols.Count(); i++)
            if (patrols[i] && patrols[i].m_Group)
                totalBots += patrols[i].m_Group.Count();
    // totalBots > 0  -> the convoy materialized (we store the initial count)
    // the count DROPS -> one was killed -> we fire the trigger (escape / ambush)
}
```

> **The way in:** `QuestPatrolExists(questID, patrols)` gives you the quest's live patrols, and `patrol.m_Group` holds the bots. From there the framework can reach them — they are the references you need to put them in the vehicle.

11.9 — How it boards the bots (the mechanic + the gotcha)

When the trigger fires, the framework takes the quest's live bots and puts them in. The non-obvious part — and it took hours— is that you have to **pacify them first**: a bot *in combat* does not walk to a waypoint (the eAI FSM requires "no threat"):

```
void BoardQuestBots() {
    Transport transport = Transport.Cast(m_Bus);
    array<eAIQuestPatrol> patrols = new array<eAIQuestPatrol>();
    if (!ExpansionQuestModule.GetModuleInstance().QuestPatrolExists(m_QuestCheckID, patrols)) return;
    int seat = 1;
    for (int i = 0; i < patrols.Count(); i++) {
        eAIQuestPatrol p = patrols[i];
        if (!p || !p.m_Group) continue;
        for (int m = 0; m < p.m_Group.Count(); m++) {
            if (seat > 5) break;                          // Cobra: 5 passenger seats
            eAIBase b = eAIBase.Cast(p.m_Group.GetMember(m));
            if (!b) continue;

            // 1) PACIFY (otherwise they don't walk):
            b.eAI_SetPassive(true);
            b.eAI_SetThreatDistanceLimit(0.0);            // so it does not lock onto the player again
            for (int tt = 0; tt < 16; tt++) {             // drain the targets it already acquired
                eAITarget tg = b.GetTarget(0);
                if (!tg) break;
                b.eAI_RemoveTarget(tg);
            }

            // 2) Walk to the door of ITS seat and get in (animated):
            vector door, ddir;
            transport.CrewEntryWS(seat, door, ddir);
            // ... SetMovementSpeedLimits + AddWaypoint(door); the Tick puts it in through the door ...
            seat++;
        }
    }
}
```

> **The gotcha that cost us:** without pacifying, the bot in combat does not navigate → it ended up getting in by *teleport* (ugly, "mannequin"). With the combo `eAI_SetPassive(true)` + `eAI_SetThreatDistanceLimit(0)` + **draining the targets**, the bot "forgets" the player, **walks and opens the door** (animated, looks good). *Trade-off:* they stay passive during the escape (which makes sense, since they are fleeing).

> *Even more detail (all the lifecycle functions, the full list of verbs, the eAI API gotchas) in the **AI knowledge pack**.*

11.10 — Example 3 — "Take the bus" (Travel) *(validated in-game)*

The simplest objective type to integrate, and the **first one validated beyond the convoys**: a **Travel** objective (`ObjectiveType` 3) is completed when the player **reaches a position**. If that position is the **end of**

a route of the framework, then **riding in the vehicle completes the objective**. The bus *is* the trip.

The new piece: the `QuestTravelID` field. A route declares which quest triggers it. When the player **claims** that quest, the framework auto-spawns the route, waits `SpawnHoldSeconds` (so you can get there and get in) and drives to the destination. No admin action needed — "you claim it and the bus appears and drives on its own".

What you build (all config; no code is touched on the quest side):

1. The Travel objective (`Objectives\Travel\Objective_T_8.json`) — the destination = the end of the route:

```
{
  "ObjectiveType": 3,
  "ObjectiveText": "Ride the bus to the terminal.",
  "Position": [3322.8, 200.0, 13029.7],   // = last waypoint of the route
  "MaxDistance": 30.0,
  "TriggerOnEnter": 1
}
```

2. The quest (`Quests\Quest_42.json`) that wraps it, with its giver (a chalkboard) and its reward.

3. In the route (`BZBusRoute_..._hdr.json`), two fields:

```
{
  "QuestTravelID": 42,        // the quest that auto-spawns THIS route
  "SpawnHoldSeconds": 20.0    // the vehicle waits 20s before setting off (so you can get in)
}
```

The in-game flow:

1. You claim the quest at the chalkboard.
2. The vehicle appears at the start of the route and **waits 20s**.
3. You get in as the passenger.
4. Boris drives to the destination → you enter the radius → **objective completed**, you collect the reward.

The hook (in `OnQuestStart`): when a quest starts, the framework scans the routes; if one of them declares `QuestTravelID` == that quest, it loads it (`LoadConfigFromPath`) and spawns it (`RespawnBus`); the route's `SpawnHoldSeconds` does the waiting. It is the **same pattern** that is later extended to Escort/VIP (the vehicle carries the VIP) and Delivery (the vehicle carries the cargo).

11.11 — Example 4 — Escort the VIP (AIVIP) *(validated in-game)*

The objective that **completes** the integration: an **AIVIP** (`ObjectiveType` 9, "escort the VIP to a place"). Expansion spawns a **VIP** (an eAI NPC, e.g. `eAI_SurvivorF_Gabi`); the framework **puts the VIP in a vehicle** and Boris drives them to the destination, while you **escort** them (you protect them on the way). The VIP arrives → objective completed.

Two different NPCs (do not mix them up):

- **Boris** = the driver (always the framework's).
- **The VIP** = the quest's NPC, the passenger. You do not drive: you make sure the VIP arrives alive.

The new piece: the `QuestEscortID` field. Same as `QuestTravelID` (it auto-spawns the route when the quest is claimed), but it also **puts the VIP in the vehicle**:

```
{
  "QuestEscortID": 10,         // the AIVIP quest that triggers this route
  "SpawnHoldSeconds": 35.0,    // generous: the VIP must spawn, walk and get in BEFORE it sets off
  "VehicleClass": "CivilianSedan"
}
```

The key insight — how the VIP is found. Here we had to read Expansion's code: the VIP is **NOT** in `QuestPatrolExists()` (that list is only for AICamp/AIPatrol). It is **flagged with `Expansion_IsQuestVIP()`** and comes from the global enum `eAIBase.eAI_GetAll()`. The framework scans that list, filters by the flag + proximity to the vehicle, and boards the VIP in its own group (without touching the player's group). *(Useful technique: Expansion's `.c` files can be read as text inside the PBO — no need to extract them.)*

The in-game flow:

1. You claim `Quest_10` at the NPC that gives the quest.
2. The **CivilianSedan + Boris** appear at the start of the route and wait 35s.
3. **The VIP** spawns nearby → the framework finds them (`eAI_GetAll` + `Expansion_IsQuestVIP`) → the VIP walks over and gets in.
4. Boris takes the VIP to the destination (end of the route) → the VIP gets out → **objective completed** (with `Autocomplete: 1`, it finishes on its own).

Gotchas we learned (skipping them costs a lot):

- **The grab is NOT `QuestPatrolExists`** — for AIVIP it is `eAI_GetAll()` + `Expansion_IsQuestVIP()`.
- **The vehicle matters:** it must have valid crew seats + **vanilla doors**. A carpack car (we tried the X5M) **sank** (odd bbox) and its door **crashed the eAI FSM** (`OpenVehicleDoor`) → error spam → overloaded server. The **CivilianSedan** (vanilla, 4 doors) worked perfectly.
- **VIP's seat:** send the VIP to a **rear seat**, not the front passenger seat — otherwise the player and the VIP fight over the same seat and merge (you get stuck inside the car).
- **Generous hold:** the VIP needs time to spawn, walk and get in before the car sets off (`SpawnHoldSeconds` ~30-40s).
- **If the quest gets stuck** (changing an objective's config while the quest is active breaks it): set `Autocomplete: 1` + restart — the completed quest finishes on its own.

What this completes: with AIVIP validated, the objective types where the vehicle has a role — **AICamp/AIPatrol, Travel, AIEscort/VIP** — all work. The others (Collection, Crafting, etc.) **coexist** without conflict. That is the full Quest integration.

12. Controls (the keys)

The framework uses **only 3 keys** — the recording cycle + opening the UI. Everything else (launching / stopping / pausing a vehicle, spawning empty vehicles, arming the loggers) is done **from the Player** (§13), not with keys. The 3 keys live in the **game menu**: `Options → Controls → "BZ AutoDrive" section`, they are **rebindable** like any DayZ control, and they come **bound by default**:

Action (name in the menu)	Default	What it is for
Open Control Panel	`HOME`	**open / close** the Player (the door to the whole UI). `ESC` also closes it (without opening the game's pause menu). If you assign it another key in Controls, that key opens and closes it
Record (start/stop)	`NUMPAD 5`	start / stop recording a route
Mark Event / Stop	`NUMPAD 4`	mark a **stop / event node** (§10) while you record

> *The direction change is no longer a key:* the **interchange / K-turn** (§9.4) is **auto-detected** from the forward↔reverse gear change — you brake to a full stop, change gear and go on; the converter cuts the leg there and Boris treats it as a new start.

> *Typical flow:* you open the panel with `HOME`; to record, `NUMPAD 5` (with `NUMPAD 4` at the stops/events; at direction changes you only brake, change gear and go on — they are auto-detected), `NUMPAD 5` again to cut; then you pick the route in the Player and watch how it does. Your bindings are saved in your profile (`Documents\DayZ\<profile>.dayz_preset_User.xml`).

> ⚠ **Everyone SEES the controls, but they only work for the admin.** DayZ shows the "BZ AutoDrive" category in Controls to **all** players (it registers the inputs globally — it cannot be hidden per admin). But the keys **do nothing** if you are not an admin: the gate is **server-side**, checked on every action against your Steam ID (§13.1). That is why the menu was kept **minimal** (3 keys): a regular player sees a short, inert category, not a debug board.

> *Note:* the `ai_run` (Boris's black box) **no longer has a key** — it is armed from the check in the Player (§13, §15.1). The old tuning/debug tools (live Parking/Reverse/Approach, SysID, gear markers, spawn slots) were removed from the menu; maneuvers come from the **auto-detected interchange** (gear change) while you record.

---

13. The interface (UI)

A single panel for the admin — the **Player** — which you open with a key (**HOME** by default, configurable). It brings everything together on one screen: the route list, what is being driven live, and the spawning of empty vehicles. It is split into three zones:

① **ROUTES (left) — your route list.** All the saved routes. You click one to **select it** and launch it with **LOAD & SPAWN**: the NPC drives it right away, **without restarting the server**. When you select it, below it shows its **summary card** (vehicle, number of waypoints, distance, max speed) and a **scrubber** to step through the route wp by wp. Below the list are the **LOAD & SPAWN** button (launch the selected route) and the **logger checks** (see below).

> *Example:* you have "Town bus", "Downtown taxi", "AB - Jeep" in the list; you pick one, press LOAD & SPAWN and it starts.

② **ACTIVE RUNNERS (right, top) — your live board.** Every vehicle being driven **right now** — your routes, mission routes, the ones started at server boot — with its **name + waypoint** and its **status** (`DRIVING` / `PAUSED` / `STOPPED`). Each one has four buttons: **RST** (restart from wp 0 of its route, without reloading), **TP** (teleport next to it to intercept it), **II** (pause / resume), **[]** (stop it and remove it). If there are many, the list **scrolls** with the mouse wheel. At the top right, **STOP ALL** takes them all down at once.

> *Example:* you see "Town bus — wp 88/166 — DRIVING". You pause it with **II**, or TP next to it to get on.

③ **ACTIVE SPAWN VEHICLE (right, bottom) — the empty vehicles you placed.** Every vehicle you spawned empty (with START/HERE/END, see below) shows up here as a row —**route · wp · position**— with two buttons: **TP** (go next to it) and **DELETE** (delete it). You can leave **several** empty vehicles placed around the map and manage them from here. The list scrolls if there are more than fit.

The bottom bar — empty vehicle spawn + scrubber. It shows the summary card of the selected route + the scrubber, and on the right **SPAWN EMPTY VEHICLE AT: START · HERE · END**: it spawns the route's vehicle **empty and drivable** at the start, at the scrubber's wp (**HERE**) or at the end. Use it to **continue a recording** (you spawn the car where you left off and keep driving) or to place cars. **If the spot is taken** by another vehicle, it doesn't spawn and says so on screen (`Spawn point occupied by <class> (N m away)`). It **no longer teleports you** to the vehicle: the vehicle appears in the **ACTIVE SPAWN VEHICLE** list (③), and from there you TP to it or delete it.

The logger checks (opt-in, before you press play). Below the route list there are two boxes — `[ ] boris_native` and `[ ] ai_run` — that you **tick BEFORE launching the route** (LOAD & SPAWN). If they are ticked, that run records its trajectory **in sync with play**:

- `ai_run` — Boris's **black box** (the run's telemetry; see §15.1). It is armed **from this check** (no longer with a key) for that one run.
- `boris_native` — Boris's **server-side trajectory**, in the same shape as a human recording: use it to compare it with your take and see where he drifted (detail in §15.2).

> They come **unticked** (they record nothing). It is opt-in per run: you tick, press play, and only that run writes the log. Unticked means normal use and zero files.

> *Note:* the keys are rebound from the game's Controls menu (§12). The checks arm the loggers from the UI itself, without depending on the keys.

13.1 — Who is admin? — configuring `AdminSteamIDs`

We just said that the controls and the UIs **"only take effect for you (admin)"**. But how does the framework decide who is admin? By your **Steam ID**.

How it identifies you. The server compares your Steam ID (the Steam64, also called *PlainId*) against a list called `AdminSteamIDs` in its settings file. If your ID is on that list, you are an admin.

The file. `<server>\profiles\BZ_AutoDrive\BZAutoDrive_settings.json`. The server **creates it empty** the first time it starts with the mod: edit it. The field that matters to you:

- `AdminSteamIDs` — the list of authorized Steam IDs. **Each ID goes in quotes**, as in the example below: without quotes the server doesn't take it and you are still not an admin.

*(The key that opens the panel is not set here: you change it in **Options** → **Controls** → **BZ AutoDrive**, §12.)*

⚠️ **THE MOST IMPORTANT PART (this is the security point):** if `AdminSteamIDs` is **EMPTY, NOBODY is admin** — not even you. The panel does not open and every action is rejected server-side (fail-closed). **Put your Steam ID(s) in the list: it is the first, mandatory step to use the framework.** If you press the panel key and "nothing happens", look at the server log: you will see a line `[Panel] acceso denegado a <your SteamID>` (= "access denied to") that tells you exactly what to edit.

JSON example:

```
{
  "AdminSteamIDs": ["76561198000000000"]
}
```

You can put several, each one in quotes and separated by commas: `["76561198000000000", "76561198000000001"]`. The server reads the file at startup: **restart it after editing**. To check that it took it, look at the startup RPT: it must say `[Settings] Cargado. Admins: 1` (or however many you added). If it says `Admins: 0` and `AdminSteamIDs is empty` shows up, the ID is not written correctly (check the quotes).

How to find your Steam ID (the Steam64 / *PlainId*): on your Steam profile, on **steamid.io**, or in the **server log** when you connect.

Why it is safe. The check is done **server-side on every action** (open panel, spawn, load route, stop…), comparing your Steam ID as authenticated by Steam. It cannot be faked from a modified client: the client-side gate is only a visual convenience.

> *Honest note:* a regular player can still **SEE** the "BZ AutoDrive" category in their Controls menu (DayZ shows the registered inputs to everyone), but the keys **do nothing** for them if they are not an admin.

13.2 — Why the route takes a few seconds to start

When you launch a route, the vehicle appears and stays in **loading** for a few seconds before it starts driving. **It is not slowness: it is on purpose.**

The route and its tape are loaded **spread over many frames**, so that the server does not stall. Loading them all at once freezes the server for **almost two seconds** (measured: 1.84 s with a route of 1,193 waypoints and 19,075 tape frames), and **it is not only the starting vehicle that suffers**: everything stalls — the vehicle that was already driving, the bots, and anyone who is playing.

In a **convoy** the problem multiplies: the second vehicle is launched **while the first one is already on the road**, so each load hits the vehicle that left earlier while it is driving. With three vehicles, that would be three freezes of almost two seconds, one on top of the other. And on a real server the convoy is not alone: there is the **line bus** running its permanent service, the other routes that relaunch themselves every few minutes, and whatever each one has running.

There is ONE single load queue for the whole server, with a fixed budget of milliseconds per frame. That means **launching five routes at once does not make the server worse**: it only takes longer to have them all ready. Routes load **in order of arrival**. The card appears in **Active Runners** as soon as Load is pressed, and its status line says what it is doing:

Status	What it means
`WAITING wp0 occupied 12s`	the start point is occupied by another vehicle: it waits for the spot to clear (it retries every 5 s) and only then creates its own vehicle, at its exact spot. It does not shift it to the side: a vehicle that is not born on its tape cannot follow it
`QUEUED 2`	waiting for its turn: there are 2 loads ahead
`LOADING 43%`	loading the route and its tape
`BOARDING CREW 2/3`	already loaded; waiting for the crew to sit down
`PREPARING`	the driver is getting in, or the last second before starting
`DRIVING` / `STOPPED` / `PAUSED`	as usual

If you press Load again on a route that is already running, **it is not launched again** and a notice says so (`PATRULLA ya esta en marcha (LOADING 43%): no se lanza de nuevo` = "PATRULLA is already running (LOADING 43%): not launched again").

The percentage comes from the route header: the converter writes into the `_hdr.json` how many rows the `_wp.csv` and the tape have (`"WpRows"` , `"CintaRows"`). A route converted before this still loads, but shows `LOADING (8400 rows)` instead of the %. To give it the number, reconvert the route with the Wizard.

The number stays still for a moment when the route part is done and then goes on with the tape: it is a % of what has been read, not a made-up bar, and it does not show 100 until loading has really finished.

Since 1.3, everything heavy queues up as well. Not only the load: creating the vehicle, the driver and each crew bot, boarding, retiring and the brain's setup steps go through **a single gate, one operation per frame**, in order of arrival. Two heavy things no longer land in the same frame, not even when launching fifteen routes at once: each one takes a few frames longer, and the server does not feel it. In the RPT it shows as `[EMBUDO] <route> <operation>: X ms · espero N cuadro(s)` .

And a route already read is not read again. When a route is relaunched (a service that loops, a mod that launches the same course every so often), its waypoints come from the server's memory, with a copy per vehicle: `[CACHE] <route>: N waypoints copiados de la cache, sin abrir el CSV`. It is read again only if the file changed (compared by size and fingerprint; the header, whole). **The first read** goes in slices of a few milliseconds: `[LOADTIME] ... X ms de trabajo en N tajada(s)`.

The knob (optional) goes in `<server>\profiles\BZ_AutoDrive\BZAutoDrive_settings.json`:

```
{
  "AdminSteamIDs": ["76561198000000000"],
  "LoadBudgetMsPerFrame": 5.0
}
```

- `LoadBudgetMsPerFrame` — milliseconds per frame that the server spends on loading routes, **shared among all the routes in the queue**. Default **5.0**, which is what a normal driver tick costs: you do not notice it.
- **Lower** (2-3): the server does not even notice; routes take longer to be ready.
- **Higher** (10-15): routes start sooner, at the cost of more expensive frames. Do not overdo it: this is exactly the lever that caused the problem.
- `0` or a negative value is ignored and the default applies (a badly set knob cannot freeze the server again).

And why it also waits for the crew. If the route carries crew bots, the vehicle **does not start until they are all seated**, whether there are 2, 3 or 10. It is not a timer: it is the seat being taken. Before, it left after a few seconds and the bots chased it on foot to the first stop — and during that first stage there was nobody on board to get out and defend it.

If a seat does not get filled (a bot that died on spawning, one that cannot find the door), the vehicle **leaves anyway** once `CrewBoardTimeoutS` is up, and the log says how many seats it left with:

```
[MUSTER] plazas completas 3/3 en 4.2 s -> arranca
[MUSTER] TOPE de 120 s: arranca con 2/3 plazas ocupadas -- 1 bot(s) no subieron
```

There is a safety net before the cap *(since 1.3)*: if after **15 s** a living bot is still on foot **with everything it needs to get in** (the formation, the driver seated, a free seat with its door clear, no threat), the framework **seats it by teleport** in the first free seat and carries on: `[MUSTER] RED a los 15 s: el bot 0 no subio con las condiciones de eAI cumplidas -> teleport a la plaza 1` (= "NET at 15 s: bot 0 did not get in with eAI's conditions met -> teleport to seat 1"). A vehicle with a door the bot can't work out (measured on an armored vehicle) no longer costs two minutes. The 120 s cap stays for what the net can't solve: a threat, no seats, no driver. Every `[MUSTER]` line carries the route's name.

- `CrewBoardTimeoutS` — seconds of waiting before it leaves anyway; it goes in the same `BZAutoDrive_settings.json`. Default **120**. It is a **safety net, not a tuning knob**: it must be far above what a healthy boarding takes. If you see `[MUSTER] TOPE` (= "cap") in normal runs, the number is not the problem.

If you would rather it started sooner, record shorter routes: the cost is proportional to the size of the route, not a fixed wait. In the server log, every load leaves its numbers:

```
[CARGA] PATRULLA LISTA: 1193 wps + 19075 frames de cinta en 264 tajadas / 8200 ms de reloj
[CARGA] PATRULLA reparto: leer ruta 358 ms · armar ruta 330 ms · leer cinta 1105 ms · armar cinta 12 ms · PEOR TAJADA 6.1 ms
```

How much it costs your server *(measured with 20 vehicles at once)*. We don't give you fps or milliseconds: they depend on your CPU and your mods, and a number measured on another machine is of no use to you. What holds on any server is the **proportion** against something already running on it:

- **A vehicle on a route costs about as much as three seated eAI bots, plus its own driver** (which is one more bot). If your server is slower, both get slower in the same proportion. The comparison is against a **seated** bot: one walking or fighting costs more than that.
- **It grows evenly:** every vehicle adds the same, whether you have 5 or 20. There is no point where the framework takes off.
- **The heavy work goes one at a time:** loading a route, creating the vehicle, the driver and the crew, boarding and retiring all go through a single gate, **one operation per frame**, however many vehicles there are. The only thing you might notice is the engine creating the vehicle when a new lap starts, and two never land on the same frame.
- **Tested with up to 20 vehicles at once.** If your server is already tight with its bots, count each vehicle as four more seated bots.

13.3 — Notifications: who sees what

The framework reports things through three channels, and each one has its own audience:

Channel	What	Who sees it
Recording	"Recording started", "Stop #N marked", the `REC` counter	only whoever is recording (recording is for admins: NUMPAD 5 and NUMPAD 4 only work for admins)
Automatic notices	arrivals and administrative ones (below)	**admins only** (`AdminSteamIDs` , §13.1), on screen or in chat, your choice
Your messages	the `ui_broadcast` verb in an event (§10.3)	whoever you choose: admins, everyone, those nearby or the passengers

Every automatic notice is also written to the server log (`[GLOBAL] …` in the RPT). `log_event` writes **only** there: that never shows up on screen.

What you see: the automatic notices. They come in two types, and each type is set separately in `BZAutoDrive_settings.json` (the same file as §13.1):

- `AdminArrivalNotifications` — **arrivals**: reaches a stop and how long it waits, the distance to the stop, end of the line, route complete, "left the terminal".
- `AdminSystemNotifications` — **administrative**: vehicle stopped, pause and resume, spawn failed, the gesture to Boris.

Each one accepts `"toast"` (on screen: the default), `"chat"` (in the game chat, with the `BZ AutoDrive:` prefix), `"both"` (both) or `"off"` (not shown; it stays in the RPT). If the key is missing, or the value is misspelled, it counts as `"toast"` . The server reads the file at startup: restart it to pick up the change.

```
{
  "AdminSteamIDs": ["76561198000000000"],
  "AdminArrivalNotifications": "chat",
  "AdminSystemNotifications": "toast"
}
```

Regular players **never** see these notices. The panel's replies (for example, "is already running: not launched again") always reach you on screen: they are the answer to your click.

What players see: you build it, with events. A regular player only sees what you send with `ui_broadcast`:

- `value`, **to whom**: `"all"` (everyone connected), `"near"` (those within `radius` meters of the vehicle; without `radius`, 150 m), `"passengers"` (those seated in the vehicle) or `"admins"`. Without `value`, or with a misspelled value, it goes **to admins only**: a typo never shows players something that was meant for you.
- `format`, **where**: `"toast"` (on screen, the default), `"chat"` or `"both"`.

For example, telling everyone, on screen, that the bus left the terminal (an event on a waypoint right after the departure):

```
{ "wp": 3, "actions": [ { "verb": "ui_broadcast", "value": "all", "format": "toast", "msg": "The bus left the terminal" } ] }
```

And by chat, to those on board: `{ "verb": "ui_broadcast", "value": "passengers", "format": "chat", "msg": "Next stop: Chernogorsk" }`. Each player receives it once, and the RPT says how many received it: `[EVENT n @ wp X] ui_broadcast (all → 3 jugadores, toast): …`.

13.4 — Vehicles look smooth: the client filter *(since 1.3)*

A vehicle driven by the framework is moved by the server; every player receives it over the network, and the orientation arrives **in steps** even when the vehicle turns evenly. It looked like a constant shake, especially standing on top of a bus or watching it up close.

Since 1.3 the client smooths what it draws, and nothing else: not the position, not the speed, not the physics, not the driving. The server marks which vehicles it drives; the filter runs only on those, only on the client, and a vehicle with a **player at the wheel** is not filtered. It comes **on** and there is nothing to configure.

To turn it off (for every player, including those who join later), in `<server>\profiles\BZ_AutoDrive\BZAutoDrive_settings.json`:

```
{
  "DisableClientVehicleSmoothing": true
}
```

If it is off, vehicles look as they did before 1.3.

14. Audio (.ogg) — give your events a voice

With the `play_sound` verb (in a NUMPAD 4 event) you play a **3D sound attached to the vehicle**. You provide the audio in your addon:

1. Put your `.ogg` in your addon.
2. Declare it in your `config.cpp` (a `CfgSoundShaders` + a `CfgSoundSets` — copy the example pattern).
3. In the event: `play_sound` with the SoundSet name.

> *Example:* your taxi says "good morning" when it picks up a passenger — you record `buen_dia.ogg`, declare it, and fire it in the stop's event. *Tip:* to test without creating an OGG, use a sound that already exists (e.g. from the radio mod). *Careful:* sometimes a sound fired from the server does not reach the client — if it does not play, it has to be re-sent over the network (the infrastructure for that is already there).

---

15. If something doesn't work

Some classics (the fine detail + more cases are in the **AI knowledge pack**, see below):

- **Fresh install: the panel doesn't open and NUMPAD 5 doesn't record** → you are not an admin yet. The startup RPT says so: `AdminSteamIDs is empty: nobody is admin…` (or `Admins: 0`). Add your SteamID64 **in quotes** to `BZAutoDrive_settings.json` and restart (§13.1).
- **The RPT says `No routes yet` at startup** → there are no routes yet: record one (NUMPAD 5) and convert it with the Wizard (§5). If it says `No default route (BZBusRoute.json): nothing starts at boot; N route(s) available in the panel`, everything is fine: your routes are in the panel and none starts by itself when the server boots.
- **On 1.30 the route list is empty and the RPT says `DayZ 1.30 cannot list files under an absolute -profiles path`** → your server starts with an absolute `-profiles=` path (e.g. `-profiles=C:\DayZServer\profiles`), and on 1.30 the game doesn't allow listing files there. Use a path relative to the server folder: `-profiles=profiles`.
- **The vehicle appears but doesn't move** → almost always, the route's coordinates don't match the map (you recorded on another map). Record your own route on the map you run.
- **My NPC run was not recorded** → the `ai_run` check was unticked. Tick it in the Player *before* pressing play (§13).
- **The NPC can't take a tight curve** → it is the most demanding part (§17); the control **caps the speed** on its own, but if the vehicle is **heavy**, the best thing is to **record the route with THAT vehicle** (the route is then made to its measure). If it is truly a tight maneuver (shed, direction change), it goes as **reverse** — see §9.4–9.5.
- **The audio doesn't play** → see §14 (server→client replication).
- **The ending says `AR x2` (route complete) or `AR fisico con accion` (`ROUTE_INCOMPLETE`)** → AutoRecovery stepped in (§9.6): soft restarts don't invalidate the route; a maneuver or a relocation does (§17, the verdict). Look for the `[AR]` lines in the RPT or open `profiles/BZ_AutoDrive/AR_log.csv`: they tell you the waypoint and what happened. If it is always the same point, go and look at it on the map.

- **The route breaks off at the same place every time and there is nothing there** → look at the slope and the recorded speed on that stretch. A downhill that suddenly gets steeper, taken at high speed, can make the vehicle hit the ground with its nose.
- **The vehicle appears sunk and won't start** → on spawn, the RPT writes a `[SpawnProbe]` block (at 1.5 s and at 3.5 s) with the vehicle's height relative to the ground and how many wheels touch the ground. If it says `RESUMEN: 0/4 ruedas en contacto` (= "SUMMARY: 0/4 wheels in contact"), it was born resting on its body and won't be able to move. It is a rare case, seen on a single third-party vehicle; if it happens to you, report it with that block.
- **The route launches itself again while I am testing something else** → it is the server's **default route** (or one launched through the API), which relaunches after `RespawnDelay` seconds (300 by default; `0` means 60 s, it does not turn it off — §8). The routes you launch from the Player are not relaunched. So that it doesn't get in your way while you test, stop that runner with **[]** in the panel.
- **The vehicle does not appear and the RPT says** `La clase '<X>' EXISTE pero NO ES UN VEHICULO` (= "class '' EXISTS but IS NOT A VEHICLE") → some "car" packs are really inventory items (toy cars, scale models). Check the `VehicleClass` in the header.

> **This framework comes with an AI knowledge pack:** a document meant to be handed to an AI (Claude/GPT/Gemini) that helps you configure your case, fix errors and build your routes. If you get stuck, that is the fast way — the AI has the whole context of the framework loaded.

> **Performance tip — run the server on another PC.** If you iterate a lot (record, convert, test), it is worth **running the server on a 2nd PC** while your main PC runs the **game client + the Wizard and the tools**. That way neither of the two chokes and the runs are smoother. To use it: you *copy* the `@BZ_AutoDrive` to the 2nd server (the same one your client uses) and set its routes folder in **[6] Configure paths**: the Wizard copies the routes to it when converting.

New RPT lines since 1.3, and what they mean:

Line	What happened
`[CACHE] <route>: N waypoints copiados de la cache, sin abrir el CSV`	the route was already read: the relaunch did not read the file
`[CACHE] <route>: la toma o su header cambiaron ... se relee`	you changed the `_wp.csv` or the `_hdr.json`: it is read again and replaced
`[CARGA] <route>: binario de la cinta descartado (<reason>): se carga el CSV`	the `_cinta.bin` is missing, or does not match its CSV: the route works anyway; `[B]` in the wizard rebuilds it
`[EMBUDO] <route> <operation>: X ms · espero N cuadro(s) · quedan P`	a heavy operation waited its turn; normal when launching several routes at once
`[C12] RED: nadie leyo el FIN en 60 s ... -> retiro`	a route launched by a mod ended badly and nobody claimed it: the framework retired it
`[C12] RED: FIN visto hace N s y el vehiculo sigue`	a mod read that route's end and did nothing with the vehicle (every 5 min)
`[CLEANUP] ... paso 5: N ocupante(s) sentado(s) -> el vehiculo NO se borra`	a player is seated: retiring waits for them to get out

15.1 — AI logger — Boris's black box (ai_run)

When a route **doesn't go as you expected** and you want to know *why*, the framework lets you record a **black box** of the NPC's run. It works like an airplane's *flight recorder*: it records, second by second, what Boris did while driving, so that you can diagnose the problem afterwards.

What it is. A **telemetry log** of Boris's runs. It does not change how he drives: it only **observes and takes notes** (position, speed, gear, which pedals he pressed, which mode he used). It is the raw material for understanding odd behavior.

It is OPT-IN and comes off. It is armed by **ticking the `ai_run` check in the Player BEFORE pressing play** (§13, **no longer with a key**) → it records in sync with that one run. **If you don't tick it, nothing is written.** It is not part of daily use: an admin who only wants their buses or convoys running **does not need it**.

When to use it. ONLY when a route misbehaves —Boris **runs wide in a curve**, **gets stuck**, **goes slow**— and you want to understand **why**, or when you are **tuning the config by hand**. The rest of the time, leave it off.

Where it lives. On the **server that ran Boris** (not on your client): `<server>\profiles\BZ_AutoDrive_PathLogger\ai_run_*.csv`. *Careful:* if you tested the route on another server, the `ai_run` stays on **THAT** server — look for it on the machine where it ran.

What it records. About **~27 telemetry columns**: the **position** and the heading, the **actual and target speed**, the **lateral deviation** (already computed), the **inputs** it applied (`throttle`, `brake`, `steering`), the **rpm**, the gear, the **active mode**, the **event markers**, and the **waypoint it is chasing** (`wp_idx`) at that instant.

> ⚠ **Important principle — let this be crystal clear:** the `ai_run` is a **MEASUREMENT to compare AGAINST your take** (functional *feedback* on how Boris did), **NOT a new recording**. **Do not give it to the Wizard as if it were a take** — in fact the Wizard **filters them out on purpose**. Why? Because feeding Boris's output back **clones his own errors**: the route degrades generation after generation (it is Machine Learning's *model collapse* — the photocopy of the photocopy). The source of truth is always **your** human recording; the `ai_run` only serves to **measure the deviation** against that truth.

How to get the most out of it. Two ways:

- **With AI (recommended):** you give the `ai_run` **together with the route** to an AI with the **AI Knowledge Pack** loaded, and it diagnoses it for you — where Boris struggled and what to adjust (which config field to change).
- **By hand (if you are technical):** you open it (it is a CSV) and see for yourself where it **drifted**, **got stuck** or **lost speed** — cross-checking it against your original take.

> *Example:* the bus always runs wide on the same curve. You tick the `ai_run` check, run the route, grab the `ai_run_*.csv` from the server's `profiles`, and give it to the AI with the route → it tells you "*at wp 340 the steering reaches its limit and cannot make the turn: your take goes through that curve faster than this vehicle can*". The answer is usually to **re-record that stretch** (slower, or with that vehicle); you do **not** feed the `ai_run` back as a take.

15.2 — boris_native — the trajectory Boris drove

Besides the `ai_run` (the diagnostic telemetry), the framework can record the **trajectory** of Boris's run: where he went, at what speed, with what heading — in the **same format as a human recording**. It is called `boris_native`.

What it is. A CSV with the line Boris **actually drove** (position, heading, speed, `throttle` / `brake` / `steering`, gear, wheel angle), server-side, at ~40 Hz. Since it is in the same format as your `path_` / `frame_`, it can be **compared against your take** to see where Boris followed your line and where he ran wide.

How it differs from the `ai_run` — the key distinction:

- `ai_run` = **diagnostic telemetry** (lateral deviation, saturations, corridor, pedals) → it answers **"why"** Boris did something odd. You read it with the AI / by hand (§15.1).
- `boris_native` = the **raw trajectory** (the line + speed, in recording format) → it answers **"where"** he went → you compare it against your take.

> One **measures**, the other **shows the path**. To **diagnose** you look at the `ai_run`; to see where he went, the `boris_native`.

It is OPT-IN. It is armed by **ticking the `boris_native` check in the Player BEFORE pressing play** (§13). It lives on the **server that ran Boris**: `<server>\profiles\BZ_AutoDrive_PathLogger\boris_native_*.csv` (one per run, with a timestamp — it is never overwritten).

> ⚠ **Same as the `ai_run`: it is a MEASUREMENT, not a new take.** Do not give it to the Wizard as a recording — feeding Boris's output back **clones his errors** (§15.1, *model collapse*). It is for **comparing** against your human recording (the source of truth), not for regenerating the route.

15.3 — Diagnostics and black boxes: `Diagnostics` and `BlackBox` *(since 1.2.1)*

Two switches in `profiles\BZ_AutoDrive\BZAutoDrive_settings.json`, both **off by default**. They are read **when the server starts** (changing them needs a restart).

```
"Diagnostics": 1,
"BlackBox": 1,
"BlackBoxKeepPerRoute": 10
```

- **`Diagnostics`** — with `0` (the default) the RPT keeps the essentials: loading, the end of each route (`REPOSO` and `FIN`), errors and every AutoRecovery intervention. The lines that repeat while the vehicle drives (steering at every step, each route's heartbeat, the arrival `SLAM`, the cursor rescues…) only come out with `1`. With routes running, the RPT weighs **more than 90% less** with it off. Turn it on when something is wrong and you need to look.
- **`BlackBox`** — with `1` every run writes its black box, `profiles\BZ_AutoDrive\BZ12_LOG_<route>_<time>.csv`: the state of the vehicle and the brain 40 times per second (position, speed, pedals, steering, sideways offset, mode, who is in control). It's what you use to understand a subtle failure. With `0` nothing is written.

- `BlackBoxKeepPerRoute` — how many boxes are kept per route; older ones are deleted automatically (tracked in `BZ12_LOG_indice.txt`). Default **10**.

---

16. Ideas — what you can build (and what could be added)

Framework + Quest open up a big space. These ideas are meant to inspire you: some you can build **today** with what exists; others are **extensions** to add (new verbs/triggers). The rule: *if you can describe it as "a vehicle that goes from here to there and on the way X happens", you can probably build it — or add the missing verb.*

Archetypes you can build TODAY

- **Public transport** — bus/taxi with stops and events (§10).
- **Convoy** — reinforcements, escort, escape, ambush (§11).
- **Motorized patrol** — a vehicle looping around an area.
- **Logistics / supply run** — a truck that carries loot (`add_cargo`) to a point and leaves it there.
- **Rescue / extraction** — the vehicle arrives, picks up bots or players, and takes them out.
- **Ambient event** — a car that drives through the town and leaves; life in the world, no mission.

> *Combined example:* a **"protect the convoy"** mission — a framework truck drives the route, the quest places enemies that ambush it, and the player has to defend it all the way to the destination.

Actions that could be added *(extension of the event DSL)*

- **Lock / eject seats** (`lock_seat` / `unlock_seat` / `eject_passenger`) — so that **players cannot get in** (or to get them out). The `slot` field is already **reserved** in the DSL (`BZAction`); the handler in `ExecuteAction` still has to be written.
- **Change the faction** of the occupants at runtime (`set_faction`) — so that Boris or his crew switch sides mid-mission (from neutral to hostile, for example).
- **Driver change** (`swap_driver`) — replace the driver at a point (Boris gets out, someone else gets in; or a quest bot takes over the driving).
- **Gestures** — wave, salute, point (on the roadmap; not implemented yet).
- **Beacons / hazard lights** — beacons on a breakdown. *(The regular **horn and lights** already work: as event verbs — `horn`, `lights_on`, `lights_off` — and as a route mode — `HornMode` / `LightsMode`, Appendix A.2b.)*
- **Drop loot / open the trunk** — leave an item at a point; open the trunk on arrival.
- **Call reinforcements** — spawn *another* framework vehicle (convoy chains).
- **Map marker** — make the vehicle show up on the player's map (follow the bus live).
- **Give / advance a quest objective** — make reaching a point **complete an objective** (the framework as an objective engine).

Triggers that could be added

- **By occupancy** (`occupancy >= N`) — so that the vehicle waits to **fill up** (N passengers/bots seated) before starting. Today `player_enter_vehicle` already fires when **the first one** gets in; what is missing is

the "when all N are on board" threshold. Combined with `start_engine` + `resume_route` = the bus leaves **when the quota is filled**.
- **On shooting** (`on_player_shoot`), **at night**, **by number of players** in the zone, **on entering an area**.

Framework + Quest combos *(the big space)*

- **Taxi that is a quest-giver** — you get in, it takes you, and during the trip it offers you a mission.
- **Boss who arrives in a vehicle** — the boss shows up driving, gets out and fights.
- **Mobile trader** — a vendor drives a route; players stop it to buy. *(Needs integration with Market — parked idea.)*
- **Dynamic events** — random spawns of vehicles + routes, so that the world feels alive.
- **The framework as an objective engine** — make the vehicle's events (arrived / was destroyed / delivered) the **completion conditions** of a quest. *This is the project's north star.*

Framework + each type of Quest objective

The integration was validated in-game with **AICamp/AIPatrol** (the convoys, §11), **Travel** (§11.10) and **AIEscort/AIVIP** (§11.11) — the types where the vehicle plays a role. Objectives without a vehicle (Collection, Crafting, etc.) coexist without conflict. The full table:

Quest objective	Role of the vehicle (BZ_AutoDrive)	Example
AICamp / AIPatrol ✅	brings/takes away the bots, boards them and drives	the convoy that flees, the ambush (§11)
AIEscort ✅	the vehicle **is what is being escorted**	"escort Boris's convoy to the base"; if it arrives intact, completed
AIVIP ✅	the VIP **rides in** the vehicle	"protect the VIP" while the NPC takes them to a safe zone
Travel ✅	the vehicle **takes you**	"travel to Cherno" is completed by getting on the bus that goes there
Delivery	the vehicle **makes the delivery**	supply-run: the NPC truck takes the cargo to the point; the objective is for it to arrive
Target	the target **rides in** the vehicle	"eliminate the boss" who escapes in the convoy → you chase him
Action / Collection	the vehicle **leaves something** or **is the place of the action**	inspect the accident, repair the truck, pick up the loot it dropped

> *What is validated:* the framework "listens" to the quest and provides the vehicle, whatever the objective is. The **three axes where the vehicle plays a role** already work in-game — **AICamp/AIPatrol** (convoys, §11.3/§11.4), **Travel** (§11.10) and **AIEscort/AIVIP** (§11.11). The others (Delivery, Target, Action/Collection) **reuse the same hook** with their own declarative field: the hook already exists; what is missing is the use case.

> *For modders:* if you think of an action or trigger that is not there, it almost always **fits into the existing DSL** (a new verb in `ExecuteAction` , a new trigger in the list). The AI knowledge pack shows you how to

> add it.

---

17. Scope and limits (what to expect)

So you know what to ask of the framework and what not to:

- **Nominal regime:** it drives at the vehicle's *normal* limit. Extreme maneuvers (drifts, hard oversteer, countersteer) it does **not** reproduce faithfully — the engine's receiver doesn't deliver them. That's out of scope, and that's fine: the framework aims at realistic driving, not at stunts.
- **High speed:** it works, with **graceful degradation**. Tested up to ~190 km/h: the line loosens a bit with speed (from ~0.2m at standstill to ~0.8m at +120 km/h) but **doesn't break** (no oscillation, 99% within 2m).
- **Tight 90° curves:** this is the most demanding point. They come out, but that's where the vehicle shows — a heavy one runs wider than a small one.
- **The CONTROLLER generalizes** over a **wide R_min range** (from a small hatchback to a truck) with **a single config**: the same trace works for all of them, and each one's physics adds the nuance.
- **Maneuvers (parking / reverse / K-turn):** the **reverse→forward interchange** —the hardest point of the control, a K-turn inside a shed— is **solved**: it is **auto-detected from the direction change** you made when recording (you don't need to mark it — §9.4), it works with physical rules and **no per-vehicle constants**, and it was tested on a broad bank (Golf FWD, BMW E60, OffroadHatchback, Hatchback FWD, mid-engine Porsche GT2RS, Sedan, Toyota 86 RWD and the Truck_01 truck) with **0 AutoRecovery recoveries**. **The condition is yours, and there is only one: make the direction change on a straight, not in a curve** (§9.5). The steering angles of the reverse belong to the vehicle, not to the road. **For the same route on another vehicle, record it with that vehicle** — when there is a maneuver in the route, that is the only thing that generalizes.
- **Arrival precision (endpoint).** The final brake is built from what the vehicle's config declares and what was measured on it (the faster it arrives, the earlier it is armed: with the braking distance its real speed needs, plus a margin), and it **ends up about 10 cm from the point**: it aims at the point, and that is where the real brake leaves it. The brake is planned with **the braking you measured yourself in the take** when the take ends braking hard (`[C12] FRENADA de la toma: 5.4 m/s2 …` in the RPT): the brake a config declares tends to be optimistic. From a motorbike to a bus, the same rule: it isn't relaxed by size, because the framework has to work for vehicles we don't know yet. **The numbers, vehicle by vehicle, are in §17.1.**
- **The verdict: when a route is complete** *(since 1.2.1)*. The end of a route is decided **with the vehicle at rest**: half a second after it stops (at most about 3 s after arriving) the RPT prints `[C12] REPOSO tras T s: a X m de la pose final (longitudinal a m, lateral l m)` (= "AT REST after T s: X m from the final pose"), and right after it the `FIN` . `ROUTE_COMPLETE` requires **all three**: (1) at rest, **0.25 m or less** from the take's final point; (2) **no crash against an object** during the whole run (a bump or a hit against the terrain don't count); (3) **no physical AutoRecovery intervention** (maneuver or relocation; soft restarts don't count, but are listed). If any fails, `ROUTE_INCOMPLETE` , and the same line says why: `quedo a X m de la pose final en reposo` (stopped X m from the final pose at rest), `choco contra un objeto: wp N contra <class>` (hit an object), `AR fisico con accion: wp N <action>` (physical AR). Same for cars and motorbikes. *(In 1.2 an arrival between 0.25 m and 1 m counted as complete with a warning: not anymore.)*

- **Which vehicles we tested.** More than 40 classes from several authors: cars, sports cars, 4x4s, vans, buses, trucks (up to a 7.5 m wheelbase), military vehicles, tractors and motorbikes (§17.1).
- **What it does NOT do:** it doesn't invent routes (it needs your recording), and it doesn't discover behavior beyond what you demonstrated. It's deterministic and inspectable — that's its appeal.

> In one sentence: **faithful and predictable in the normal regime; the honest limit is in the extreme.**

17.1 — Arrival, vehicle by vehicle (measured)

These numbers come from the test server's logs, **with the 1.2 brain**. Each row is a vehicle, and each "arrival" is a run that finished its route; the distance is what was left between the vehicle and the recorded final point.

How to read it:

- **The median sits around 0.10-0.12 m.** The brake aims at the point and ends up about **10 cm** from it, which is where the real brake leaves it. The target is **0.25 m**: since 1.2.1, more than that at rest is `ROUTE_INCOMPLETE` (the verdict, above). *(The first table is from 1.3; the two below, from 1.2.1 and 1.2, stay as history.)*
- **p90** = 9 out of 10 arrivals ended up at that distance or closer.
- **Runs that did not arrive are not in the table**: they are counted separately, below.
- A run repeated from the same starting point can give almost the same number: more arrivals don't always mean more variety.

With 1.3 (November 2026): a mountain circuit with climbs, 13 vehicles at once and, in half of the runs, vehicle wrecks on the route. 227 arrivals: median **0.11 m**, **97 %** within 0.25 m. Cars, 4x4s, the truck and the armored vehicle all close at a **0.10-0.15 m** median, the heavy ones and uphill too; of their 159 arrivals, only one ended up more than 0.25 m away (the armored vehicle, after hitting a tanker wreck from the event). Motorbikes close with their own driver (§12): two of them at a fixed 0.02 and 0.19 m; the `Motorbike_01` arrives within 0.02-0.05 m four times out of ten and at 0.4-0.7 m the other six, coming in crooked (5-12°).

Type	Vehicle (class)	Arrivals	Median	p90	Within 0.25 m
car	`Sedan_02`	20	0.10 m	0.10 m	100 %
car	`minicooper`	18	0.11 m	0.11 m	100 %
car	`V140`	18	0.11 m	0.11 m	100 %
sports car	`rx8hachi`	14	0.11 m	0.11 m	100 %
sports car	`Star_Agera`	12	0.15 m	0.16 m	100 %
4x4 / pickup	`Expansion_Landrover`	17	0.10 m	0.10 m	100 %
4x4 / pickup	`Star_Jeep_Rubi_Extra`	18	0.11 m	0.17 m	100 %
4x4 / pickup	`Star_Chevrolet_Z71`	14	0.11 m	0.13 m	100 %
truck	`Star_Oshkosh_M1083`	16	0.12 m	0.14 m	100 %
military	`Star_APC_Cobra_camoorange`	12	0.10 m	0.11 m	92 %
motorbike	`Motorbike_02_Yellow`	26	0.02 m	0.03 m	100 %
motorbike	`Motorbike_02_Red`	32	0.19 m	0.19 m	100 %
motorbike	`Motorbike_01_Blue`	10	0.43 m	0.56 m	40 %

The `Motorbike_01_Blue` comes from a take that ends turning at walking pace: that is why it ends up far. With a take that ends on a straight (§7.1) it closes within centimeters.

With 1.2.1 (since 2026-09-17). 52 arrivals: median **0.11 m**, **87 %** within 0.25 m, none farther than 0.36 m. In that period, 7 runs did not arrive (vehicle stuck with no way out, or driver dead).

Type	Vehicle (class)	Arrivals	Median	p90	Within 0.25 m
4x4 / pickup	`MBM_LandCruiser_Grey`	21	0.18 m	0.31 m	67 %
van	`MBM_Sprinter_Green`	1	0.10 m	0.10 m	100 %
bus	`BZ_ChernoBus_02`	17	0.11 m	0.18 m	100 %
bus	`MBM_ApocalypticPAZ_White`	8	0.11 m	0.11 m	100 %
bus	`MBM_ApocalypticPAZ_Black`	4	0.11 m	0.11 m	100 %
bus	`BZ_ChernoBus`	1	0.11 m	0.11 m	100 %

From 2026-09-02 to 2026-09-16, with a much more varied fleet. The last two arrival adjustments were still missing (entering the final brake 40 times per second, and correcting the lateral drift in the last meters), so these numbers are **the worst to expect**: today's are equal or better. 430 arrivals across 41 vehicles: median **0.19 m**, **77 %** within 0.25 m. In that period, 40 runs did not arrive.

Type	Vehicle (class)	Arrivals	Median	p90	Within 0.25 m
car	`Hatchback_02`	50	0.28 m	0.57 m	48 %
car	`Sedan_02`	11	0.21 m	0.63 m	64 %
car	`MBM_ImprezaWRX_Rusty_Orange`	4	0.83 m	0.94 m	0 %
car	`a_m5`	3	0.13 m	0.25 m	100 %
car	`CivilianSedan`	3	0.11 m	0.20 m	100 %
sports car	`jesko`	26	0.16 m	0.23 m	100 %
sports car	`m4w_Blue`	7	0.25 m	0.51 m	57 %
sports car	`Star_Agera`	6	0.11 m	0.12 m	100 %
sports car	`Gemera`	6	0.21 m	0.23 m	100 %
sports car	`lfa`	5	0.19 m	0.66 m	60 %
sports car	`r32`	3	0.21 m	0.22 m	100 %
4x4 / pickup	`Expansion_Landrover`	47	0.22 m	0.32 m	87 %
4x4 / pickup	`OffroadHatchback`	5	0.22 m	0.60 m	60 %
4x4 / pickup	`MBM_F350_Monster_Red`	4	0.11 m	0.12 m	100 %
4x4 / pickup	`MBM_LandCruiser_Grey`	3	0.16 m	0.17 m	100 %
van	`MBM_Sprinter_Green`	13	0.10 m	0.10 m	100 %
bus	`BZ_ChernoBus`	39	0.22 m	1.02 m	54 %
bus	`MBM_ApocalypticPAZ_Yellow`	11	0.11 m	0.11 m	100 %
bus	`BZ_ChernoBus_02`	6	0.20 m	4.65 m	83 %
bus	`MBM_ApocalypticPAZ_Black`	6	0.12 m	0.34 m	83 %
bus	`MBM_ApocalypticPAZ_Blue`	3	0.10 m	0.10 m	100 %
truck	`AEC_Matador`	48	0.17 m	0.37 m	90 %
truck	`hx58`	9	0.17 m	0.24 m	100 %
truck	`Star_Oshkosh_M1083`	9	0.13 m	0.19 m	100 %
truck	`MBM_Apocalypse_Truck`	3	0.11 m	0.11 m	100 %
military	`m4_tractor_winter`	50	0.10 m	0.41 m	74 %
military	`Star_APC_Cobra_grey`	4	0.23 m	0.62 m	75 %
tractor	`ExpansionTractor`	30	0.22 m	0.44 m	67 %

With one or two arrivals (no statistics): `MBM_HarleyDavidsonFatBoy_Black` 0.11 m, `MBM_ImprezaWRX_WRC` 0.11 m, `OffroadHatchback_White` 0.22 m, `PzKpfw_VI_Tiger_camo` 0.52 m, `SVJ` 0.15 m, `Star_Camaro_1969` 0.24 m, `Star_Chevrolet_Z71` 0.10 / 0.17 m, `Star_Porsche_944` 0.11 m, `Toyota_86_RB_Red` 0.22 / 0.71 m, `Truck_01_Covered` 0.11 / 0.19 m, `VW_T6` 0.22 m, `ZiS5_Benzovoz` 0.24 m, `esc` 0.13 m.

> **The cases that end up far off repeat on the same vehicle and the same route.** The `MBM_ImprezaWRX_Rusty_Orange` ended up 0.8 m **off to the side** all 4 times; the `Hatchback_02` tends to **overshoot** the point. If one of your vehicles always arrives off the point, first look at how its take ends (§7.1) and try re-recording that ending.

---

18. Open frontier — take it further

This is open source for a reason: **I got this far; the next level is yours.** Here, with no fluff, is where the edge is and where to push it. If you're going to contribute, start with one of these.

The big leap: extreme driving (a "v3")

Where it is today: it drives at the vehicle's **nominal** regime (§17). Drifts, hard oversteer, countersteer — the engine's receiver doesn't reproduce them faithfully. *The leap:* a custom physics override (skip eAI's model on those stretches) or an ML layer that captures what classic control can't. It's the biggest and the hardest — and the one that opens up "movie-style" driving.

100% autonomous Wizard

Where it is today: it converts without asking and **warns you** about anything odd in the take (spins, jumps, stops with no mark), but it doesn't correct anything: if the take has a problem, you re-record it. *The leap:* have it tell you **which stretch to re-record and why** before the NPC drives it — and have it measure by itself what today can't be measured from a take (the launch floor, §6C).

Multi-vehicle coordination

Where it is today: several vehicles can run the same route (one runner per vehicle, Appendix B.7) and your mod can coordinate them —schedules, yielding right of way—, but the framework doesn't coordinate them with each other. *The leap:* real convoys — dynamic spacing, the rear one follows the front one, re-forming if one drops out. There's technique borrowed from ARMA/RV (breadcrumb + convoy separation) to draw inspiration from.

The living NPC

Where it is today: the driver drives, but is mute. *The leap:* a driver with an LLM that talks, reacts and even gives a mission during the trip. (None of this comes in the package: it's an open idea.)

More worlds

- **Trains** — the easiest case: the network *is* the rails, with no steering and no invented turns. ~70% covered without touching anything.
- **Other engines** — the principle (demonstration + config reading + classic control) isn't exclusive to Enfusion; it's portable.

> **If you take one:** the AI knowledge pack gives you the complete context —why each decision, what was tested, what failed— so you start **at the edge and don't repeat the path**. That's the idea of

> opening it: so the next person starts where I finished. Take it and keep going. 🚀

Appendix A — Complete route config reference

All the fields of the JSON (`BZBusRoute*.json`). Almost all of them have a sensible default: **you only touch what you need**. Convention: a value of `-1` or `0` usually means "*use the code's default / disabled*".

> **What each brain reads.** The 1.2 brain (the one that runs by default) reads **A.1, A.1b, A.2b, A.5, A.6 and A.8**. The fields in **A.2, A.3, A.4, A.5b, A.5c and A.7** belong to the 1.1 brain: they only act if the server has `CEREBRO_11.txt` with content (§4). They stay documented for anyone who uses 1.1.

A.1 Basics

Field	Default	What it does
`VehicleClass`	`"ExpansionBus"`	which vehicle spawns
`DriverClass`	`eAI_SurvivorM_Boris`	the NPC driver
`RespawnDelay`	`300`	sec before respawning (continuous service)
`AverageSpeedMS`	`11`	average speed, for the ETA calculation
`SpawnHoldSeconds`	`3`	wait before starting (`0` =right away, `30+` =wait for a mission trigger)
`VehicleInvincible`	`true`	unbreakable; `false` = destructible (enables damage/ambush)
`VehiclePartsLocked`	`true`	**tamper-proof**: while the framework drives it, players can't remove its parts (wheels, doors, battery...); `false` = removable, like on any car. The cargo is not affected. An event changes it (`set_vehicle_parts`)
`DriverInvincible`	`true`	Boris unbreakable; `false` = mortal (can die in combat / when taking damage)
`DriverClothing`	`[]`	the **driver's** clothing (classnames to equip). Empty = the framework's default outfit. See §A.1b
`MaxGear`	`6`	top gear of the automatic gearbox (FIRST=2 ... SIXTH=7)
`Attachments`	`[]`	parts to equip on spawn (wheels, battery, spark plugs...)
`Wheelbase`	`0`	distance between the axles (`0` =from the code); used by the reverse

A.1b How to dress the driver

The framework dresses **Boris (the driver, seat 0)** in an outfit you can configure **per route**. You define it with `DriverClothing` in the route header/JSON: a list of clothing **classnames**; each one is equipped in its slot.

```
"DriverClothing": ["BZ_AutoDrive_TShirt", "PolicePants", "PoliceCap", "CombatBoots_Black"]
```

- **If you do NOT set** `DriverClothing` (or leave it `[]`): Boris wears the **framework's default outfit** — `BZ_AutoDrive_TShirt` (the branded T-shirt, packed inside `@BZ_AutoDrive`, always loaded) + `PolicePants` + `PoliceCap` + `CombatBoots_Black`. Old routes look the same as always.
- **If you set** `DriverClothing` **with items:** that list **replaces** the default completely (you put in the whole outfit you want; nothing is mixed in).
- Each classname is equipped with `CreateAttachment` by string: if a classname **doesn't exist or its mod isn't loaded**, that item **fails silently** (nothing breaks, it just doesn't appear). Use valid classnames (from a loaded clothing mod, or vanilla).

> **Scope:** this dresses **only the driver**. The clothing/loadout of the **convoy/crew bots** is handled by **Quest (Expansion Quests)**, not by the framework — see §11.7.

A.2 Speed / driving control *(1.1 brain)*

> ⚠ **1.1 brain only.** These fields are read only if the server has `CEREBRO_11.txt` with content (§4). The 1.2 brain, the one that runs by default, doesn't use them.

*(With 1.1: by default the route uses **your recorded speed** (capped by the curve) with the **inverse model**; `FollowPath=true` computes the speed from curvature instead of using the recorded one. These flags were the old "modes 1/2/3".)*

Field	Default	What it does
`GearStrategy`	`"auto_box"`	`auto_box` (the AT decides) or `follow_recording` (uses the recorded gear — for sports cars that spin their wheels in 1st)
`FollowPath`	`false`	`true` = computes the speed from **curvature** (ignores the recorded one); `false` (default) = uses your recorded speed
`FollowPathLatAccel`	`4.0`	lateral grip (m/s^2) → sets the vmax per curve
`FollowPathMaxKmh`	`50`	speed cap on straights (when `FollowPath=true`)
`FollowPathCurveSpan`	`5`	wp spacing for measuring curvature (avoids noise)
`FollowPathSpeedSmooth`	`8`	smooths the speed profile (anticipates curves)
`FollowPathUseReference`	`false`	uses the recorded speed capped by the curve (part of the default control)
`UseInverseModel`	`false`	throttle/brake by PID + inverse model (vehicle-agnostic)
`InverseModelKp/Ki/Kd`	`-1`	speed PID gains (`-1`=default)
`InverseModelLowRpmMin`	`false`	damped gear (high gears in cruise, more stable)
`TargetSpeedSmoothWindow`	`0`	smooths the target speed (less PID oscillation)
`AccelShiftThreshold`	`999`	anti-catapult: shifts up if it accelerates too much (`999`=off)

A.2b Horn and lights *(§9)*

Field	Default	What it does
`HornMode`	`"off"`	`off` (never) · `stops` (horn at the end of each stop) · `finish` (on reaching the end, if the route finishes complete). `replay` makes no sound: the take doesn't record the horn (only the `horn` verb; the server warns about it when the route loads)
`LightsMode`	`"auto"`	**`auto`** (default — on at night, 19:00–06:00, and it follows the time during the route) · **`off`** (always off → **disables the lights**, for stealthy night missions) · `auto_inverted` (off at night → stealth) · `on` (always). `replay` doesn't work in this version: the take doesn't record the lights. A `lights_on` / `lights_off` verb wins until the next spawn

> **Horn or lights at an exact point?** Use the verbs `horn`, `lights_on` and `lights_off` in an event (§10).

> **Default `auto`:** every take turns the headlights on by itself at night, with nothing recorded. The lights come on **as soon as the engine starts** (during the spawn-hold), not mid-route. Night threshold = 19:00–06:00 (game world time). **Disable the lights:** put `"LightsMode": "off"` in the route header (stealthy night mission → the vehicle doesn't give itself away). For the lights to show, the vehicle needs a **battery + headlights (bulbs) installed** (§7); the framework energizes the battery on spawn, but the headlights have to be in the `Attachments`.

A.3 Steering *(1.1 brain)*

> ⚠ **1.1 brain only.** These fields are read only if the server has `CEREBRO_11.txt` with content (§4). The 1.2 brain, the one that runs by default, doesn't use them.

Field	Default	What it does
`SteeringScale`	`-1` (auto)	steering scale; short wheelbase → lower it (auto derives it from the wheelbase)
`CurvatureSteerBoost`	`0`	amplifies the steering in curves (against understeer in 90° turns)
`PathSmoothWindow`	`5`	smooths the path positions
`CruiseLateralDeadband`	`0`	"wall": band where it doesn't correct (anti-micro-steering); `0.5` recommended
`CruiseLateralKGain`	`1.0`	gain of the lateral correction
`CruiseLateralDamp`	`0`	lateral damping (kills the zigzag); `0.3` moderate
`CruiseLateralCenterOffset`	`0`	corrects the lateral bias (`+` =right, `-` =left)
`CruiseHybridSteerThreshold`	`-1`	uses the recorded steering if it exceeds the threshold (keyboard pulses)
`CruiseHybridThrottleThreshold`	`-1`	uses the recorded throttle when the human was accelerating
`CruiseFFWeight`	`-1`	weight of the curvature feedforward in cruise (`-1` =0.25)
`CurveThrottleEnabled` (+ `...LookaheadM/StartDeg/FullDeg/MinScale`)	`true`	cuts the throttle BEFORE a tight curve

A.4 Slopes *(1.1 brain)*

> ⚠ **1.1 brain only.** These fields are read only if the server has `CEREBRO_11.txt` with content (§4). The 1.2 brain, the one that runs by default, doesn't use them.

Field	Default	What it does
`SlopeCompensationEnabled`	`true`	adds throttle uphill, takes it away downhill
`SlopeLookaheadWps`	`5`	how many wps ahead it looks at the slope
`SlopeGain`	`1.0`	how much it compensates (`1` =full)
`SlopeLateralGain`	`1.0`	corrects the lateral bias the slope introduces

A.5 AutoRecovery — keeping the route from being cut short *(see §9.6)*

Optional blocks in the route header/JSON. **If you write nothing, these defaults apply**, and that is the validated behavior. If you write only part of a block, the missing fields take their default (`{"Physical": {"ReverseM": 4}}` leaves `Enabled` at `true` and changes only the reverse meters).

```
"AutoRecovery": {
  "Logical":  { "Enabled": true, "WaitS": 0.5, "MaxAttempts": 3, "CooldownS": 1.0 },
  "Physical": { "Enabled": true, "ReverseM": 3.0, "ReverseMaxKmh": 5.0, "LadderMaxM": 6.0, "SweepMaxM": 10.0
}
},
"BogRecovery": { "Enabled": true, "WindowS": 3.0, "MinProgressM": 0.5, "MaxAttempts": 2 },
"ManualAR":    { "AdvanceM": 10.0, "GoodLatM": 1.0 }
```

`AutoRecovery.Logical` **— restart on the spot** (the brain got stuck; the vehicle can move)

Field	Default	What it does	When to change it
`Enabled`	`true`	restarts on the spot, without moving the vehicle	`false` to diagnose: the jam is still logged and you see where it happens
`WaitS`	`0.5`	seconds stopped before acting	raise it if you want the halt to be noticeable at stops with no wait event
`MaxAttempts`	`3`	attempts in the same place (8 m radius); `0` = no limit	lower it if you'd rather have a broken route declared stuck sooner
`CooldownS`	`1.0`	minimum pause between attempts	raise it if you see attempts chained too close together

`AutoRecovery.Physical` **— the ladder** (a crash, ending up crosswise, riding up on something, sunk or rolled over)

Field	Default	What it does	When to change it
`Enabled`	`true`	enables the ladder from §9.6	`false` for pure replay, or to see where it gets stuck with nothing freeing it
`ReverseM`	`3.0`	meters of straight reverse in the first step	raise it for long vehicles; lower it on narrow streets
`ReverseMaxKmh`	`5.0`	speed cap for that reverse	lower it in a convoy or with traffic behind
`LadderMaxM`	`6.0`	meters of safe zones, counted **from the first zone**	raise it so AutoRecovery never cuts the route short
`SweepMaxM`	`10.0`	meters of sweep when the destination is occupied or the vehicle ends up riding on something	raise it if the typical obstacle on your server is long (a truck, a barricade)

`BogRecovery` **— engine bog** (done by the controller)

Field	Default	What it does	When to change it
`Enabled`	`true`	on a bog it forces **first gear** and pulls away as it would on a slope	`false` leaves only the log, with no correction
`WindowS`	`3.0`	seconds with throttle and no progress before deciding	raise it for heavy vehicles that pull away slowly
`MinProgressM`	`0.5`	progress that already counts as "moving forward"	raise it if the vehicle pitches a lot when pulling away
`MaxAttempts`	`2`	attempts in the same place; when they run out, it moves on to the ladder	—

`ManualAR` **— the operator's intervention** (the `BZ12_AR_MANUAL.txt` flag)

Field	Default	What it does
`AdvanceM`	`10.0`	meters ahead of the last good-ground point, where it relocates
`GoodLatM`	`1.0`	maximum lateral deviation for a point to count as "good ground"

Examples.

```
// Unattended transport service: never cut the route short.
"AutoRecovery": { "Physical": { "LadderMaxM": 20.0, "SweepMaxM": 30.0 } }

// Pure replay: I want to see exactly what the vehicle does, with no assists.
"AutoRecovery": { "Logical": { "Enabled": false }, "Physical": { "Enabled": false } },
"BogRecovery": { "Enabled": false }

// Convoy: shorter and slower reverse, because of whoever is coming behind.
"AutoRecovery": { "Physical": { "ReverseM": 2.0, "ReverseMaxKmh": 3.0 } }
```

> **Where to look at the result:** the RPT line `[C12] AutoRecovery <ruta> ...` prints, once per run, the configuration that actually ended up active. If you think a value isn't being applied, that's the line that tells you.

A.5b External obstacles — AR_OnWay *(1.1 brain)*

> ⚠ **1.1 brain only.** These fields are read only if the server has `CEREBRO_11.txt` with content (§4). The 1.2 brain, the one that runs by default, doesn't use them.

Unlike AutoRecovery (which keeps Boris from getting stuck **on his own**), this protects him from the **world**: another vehicle stopped in the way, or one that hits/pushes him. Two independent flags; the **v1 route import** (Wizard **[2]**) asks you for the profile.

Field	Default	What it does
`ObstacleSlow`	`false`	**predictive** braking for a vehicle ahead (lookahead that grows with speed and the vehicle's physics; brakes at its real maximum until it stops before the obstacle)
`ObstacleEscape`	`false`	if the obstacle **persists** (or it gets hit/pushed off the line), it **teleports** to the first clear wp past the obstacle
`ObstacleScanDist`	`50`	minimum (**floor**) scan distance along the path; the real lookahead grows with speed
`ObstacleStopDist`	`15`	how many meters from the obstacle it stops
`ObstacleCorridorHalf`	`2.3`	half-width of the lane: a car on the **shoulder** (lateral offset > this) does NOT make Boris brake
`ObstacleEscapeWaitS`	`6`	sec braked at the obstacle before escaping
`ObstacleEscapeResumeKmh`	`10`	gentle speed when resuming after the teleport

> **Profiles (the v1 import asks for them; in a new take they're set by config): Robust transport** = both ON (the bus gets around whatever blocks its way). **Interceptable** = `ObstacleSlow` ON + `ObstacleEscape` **OFF** (Boris brakes nicely for whoever blocks him but does **not** escape → the interception mission works). **None** = both OFF (pure replay).

A.5c Classic teleport AutoRecovery *(1.1 brain)*

> ⚠ **1.1 brain only.** These fields are read only if the server has `CEREBRO_11.txt` with content (§4). The 1.2 brain, the one that runs by default, doesn't use them.

An earlier mechanism, **independent** of the one in A.5 and **off** by default: when the vehicle has been stuck for N seconds, it teleports it a few waypoints ahead. The one in A.5 covers this with no visual jump; this one stays for whoever needs it as a last safety net.

Field	Default	What it does
`AutoRecoveryEnabled`	`false`	if Boris gets stuck, it teleports him ahead
`AutoRecoveryStuckTimeS`	`10`	sec stuck before teleporting
`AutoRecoveryAdvanceWps`	`5`	how many wps ahead it teleports
`AutoRecoveryCooldownS`	`8`	minimum between teleports (anti-spam)
`AutoRecoveryMaxPerMission`	`0`	`0` =unlimited, `X` =the mission fails if it goes over that
`StuckAdvanceTimeoutS`	`0`	a **separate** safety: if it doesn't advance any wp for N s, it **pushes** the index (no teleport). `0` =default 60 s. It runs even if the rest is off; set it very high to disable it

A.6 Convoy / Quest *(see §11)*

Field	Default	What it does
`ConvoyMode`	`""`	`""` / `"flee_on_kill"` / `"ambush_on_damage"`
`Crew`	`[]`	bots that travel from the start (§11.7)
`Events`	`[]`	event nodes (§10, §11.7)

A.7 Reverse and interchanges — *advanced (1.1 brain)*

> ⚠ **1.1 brain only.** These fields are read only if the server has `CEREBRO_11.txt` with content (§4). The 1.2 brain, the one that runs by default, doesn't use them.

> **The flow today is simple.** The **reverse** is auto-detected from the **recorded gear**, the **approach** is automatic, and the **direction change** is **auto-detected** from the forward↔reverse gear change —no key— (§9.4). The old **hand-marked waypoint modes** (`parking` , `maniobra` , `approach`) **are no longer produced** — their fields remain only for compatibility with old takes (marked *legacy* below). | Field | Default | What it does | |---|---|---| | `DirectReplayFromWaypoint` | `-1` | from this wp on, literal replay of the recorded inputs | | `ManiobraTargetSpeedCap` | `18` | *(legacy)* cap of the old `maniobra` mode (deprecated, not produced) | | `ParkingTargetSpeedCap` | `0` | *(legacy)* cap of the old `parking` mode (deprecated, not produced) | | `ModeEntrySnapEnabled` / `...MaxDist` | `false` / `0.5` | snap (teleport) to the recorded pos+heading when entering the **reverse**. **OFF by default**: the genuine control positions the vehicle without a teleport (§9.5); re-enable it only if a specific route needs it | | `AntiRollbackEnabled` / `...PitchThreshold` | `true` / `0.05` | slams the brakes on a slope at speed ~0 (doesn't roll backwards) | | `ParkingStanleyK` / `ParkingFFWeight` | `-1` | *(legacy)* Stanley of the old `parking` mode (deprecated) | | `ReverseStanleyK` / `ReverseFFWeight` / `ReverseFFSign` / `ReverseFFMaxSteerRad` | `-1` / `-1` / `0` / `0` | bicycle-model control in reverse | | `ReverseSteerGateOffset` / `ReverseSteerThrottleFloor` / `ReverseSteerMax` | `0` | anti-stall and anti-overshoot gates in reverse | | `ReverseRecordedSteerThreshold` | `0` (→0.2) | follow the recorded steering in reverse (recording-as-manual) | | `ReverseTargetSpeedCap` / `ReverseStanleyFineMax` / `ReverseHeadingDeadbandDeg` | `0` | speed ceiling / fine correction / angle deadband in reverse | | `EndFreezeDisabled` | `0` | `0` =at the end, it brakes and freezes where the human stopped; `1` =keeps going without braking | | `ReverseStanleyMinSpeed` | `0` (→2 m/s) | speed floor for the lateral correction in reverse (breaks the 1/v spiral at low speed) | | `ApproachAuto` | `false` | **automatic** predictive braking into the maneuver — it's the default behavior (`approach` is no longer marked by hand) | | `ApproachExitKmh` | `0` (→~20) | target speed at the end of the approach ramp |

> **Reverse speed — automatic, no config (validated on 4 vehicles, R_min 3.44–4.57).** Boris reverses at `min(recorded speed, speed its physics allows for the arc)` → it generalizes to any vehicle (one with a wide turning circle reverses more slowly, and that's why it follows the arc better). As it approaches the **end** of the reverse leg, it brakes by itself (*endpoint-taper*): on **flat ground** it arrives at walking pace (killing the overshoot on the way out), and **uphill** it keeps its momentum to climb (it doesn't brake too much). There are no fields to touch — it comes from the vehicle's physics and the slope.

> **Cut the maneuver on a straight (to generalize — §9.5).** Direct replay (`DirectReplayFromWaypoint`) reproduces **vehicle-specific steering angles**: if the reverse leg starts **in a curve**, a vehicle with a different wheelbase runs wide. Make the **direction change** (brake to a full stop before reversing the gear; that's where the auto-detect cuts the leg) **where the trajectory is already straight** (the curve is handled by the closed-loop control, which is vehicle-agnostic). Watch out for the auto-approach (`ApproachAuto`): it can **brake too much** if the cut falls on a fast straight — that's why it's better to cut where you're already going slowly.

> **The 1.1 brain's interchange and endpoint.** The *cusp* (reverse→forward) and the self-adaptive stop brake of 1.1 were validated on a broad bank of drivetrains and sizes. **The 1.2 brain's arrival precision is in §17.** | Field | Default | What it does | |---|---|---| | `EndpointThrottleCapEnabled` | `true` | enables the throttle cut + self-adaptive brake in the final endpoint zone | | `EndpointStopDecelMS` | `0` (→auto) | target decel for the stop; `0` = derived from the vehicle's config × the real ground grip | | `EndpointStopDecelFactor` | `0.85` | fraction of the real maximum brake it applies (margin so the wheels don't lock) | | `FwdClimbFactor` | `1.6` | sustained throttle help for a **front-wheel-drive** vehicle to climb the uphill at the endpoint (avoids the "baby steps") |

A.8 Waypoints

Each entry of `Waypoints[]` :

Field	What it is
`pos`	`[x, y, z]` the position
`targetSpeed`	target speed (km/h) at that point
`targetGear`	gear (if `follow_recording`)
`mode`	`normal` or `reverse` — reverse is **auto-detected from the recorded gear**; the direction change goes separately in `legBreak` . *(The old `parking` / `maniobra` / `approach` are no longer produced.)*
`isStop` / `stopDuration` / `stopRadius`	stop: whether it is a stop, how long it brakes, radius
`targetThrottle/Brake/Steering`	recorded pedals (they only count if `hasInputData=1` ; the recorded steering goes in the tape, not here)
`hasInputData`	`1` = the route carries your recorded pedals — **the Wizard always writes 1**, and the 1.2 brain uses them as a starting point (§9); `0` = no recorded pedals (old routes or routes imported from v1)
`name`	the point's label (the stops)

Appendix B — For developers: extending with code

If you want to go beyond the JSON —add a verb, a trigger or a behavior— here are the extension points. It's **Enforce** (DayZ's scripting).

B.1 The heart: how the framework drives the vehicle

The framework injects the inputs in the `OnInput` of a `modded class CarScript`, **after** eAI's (override-last):

```
modded class CarScript {
    override void OnInput(float dt) {
        super.OnInput(dt);                    // eAI runs its logic (including ShiftTo(FIRST))
        if (!GetGame().IsServer()) return;
        BZBusService srv = BZBusService.GetRunnerForCar(this);   // the runner that OWNS this vehicle
        if (!srv) return;
        // (if a real player is driving, the framework steps aside)
        if (srv.SampleDrive12(this, dt)) return;   // brain 1.2: Sample at 40 Hz + apply the inputs
        // ... otherwise, the brain 1.1 path ...
    }
}
```

> **The breakthrough was that order:** eAI forces 1st gear every frame; our override runs AFTER it and sets the correct gear/inputs.

Surface the framework **writes** on `Car`: `SetThrottle(0..1)`, `SetSteering(-1..1)`, `SetBrake`, `SetHandbrake`, `ShiftTo(gear)`, `EngineStart/Stop`. And it **reads**: `GetSpeedometer()`, `EngineGetRPM()`, `WheelGetContactPosition(i)` (this is where the wheelbase comes from), `WheelGetSurface(i)`, etc.

B.2 Adding a new verb

All the verbs live in `BZBusService.ExecuteAction(Car car, BZAction action, int evIdx)`, one `else if` per verb. Real code pattern:

```
private void ExecuteAction(Car car, BZAction action, int evIdx) {
    if (!action) return;
    string verb = action.verb;
    if (verb == "start_engine") {
        if (!car.EngineIsOn()) car.EngineStart();
    } else if (verb == "freeze_vehicle") {
        m_Frozen = true;
    }
    // ... you add your branch:
    } else if (verb == "mi_verbo") {
        // you have at hand: car (the vehicle), action (its fields: value, fvalue, msg, slot...),
        // m_WaypointIndex (where it is). Do your thing:
        car.SetHandbrake(1.0);
        BZBusLog.Info("[EVENT " + evIdx + "] mi_verbo ejecutado.");
    }
}
```

The `delay` (choreography) is already handled for you by the dispatcher (`CallLater`). And that's it: the modder can now put `{ "verb": "mi_verbo" }` in their `Events[]` . *(For new fields, add them to the `BZAction` class — it's flat, so it doesn't break the parser.)*

B.3 Adding a new trigger

Triggers are evaluated by `BZTrigger.type` in `BZBusService.IsTriggerFired` . You add the field it needs to the `BZTrigger` class, its branch in `IsTriggerFired` (e.g. `on_player_shoot` with its condition) **and the name in `TriggerTipoConocido`** : otherwise, when the route loads, it warns that the trigger is unknown, and the trigger never fires.

B.4 The Quest hook

The integration with Expansion-Quests comes in through a `modded class MissionServer` :

```
modded class MissionServer {
    override void Expansion_OnQuestStart(ExpansionQuest quest) {
        super.Expansion_OnQuestStart(quest);
        BZBusService.GetInstance().OnQuestStart(quest);   // the framework "listens" to the quest
    }
}
```

From `OnQuestStart` , the framework **polls** `ExpansionQuestModule…QuestPatrolExists(questID, patrols)` to reach the living bots (they are lazy, spawned by proximity). *(The objective subclass `ExpansionQuestObjective*Event` does NOT compile in our scope → that's why the MissionServer hook + poll is used.)*

B.5 Enforce gotchas (the ones that hit us)

Enforce is neither C# nor C++; these traps cost hours:

- **There is no ternary** `?:` , and no multiline `if` with `&&` at the start of a line → "Syntax error". Use `if/else` or intermediate bools.
- `new Class(args)` **blows up** → create it empty and set the fields.
- **Don't mod engine classes** (`CGame` , etc.) → it breaks on load.
- **"Formula too complex"** at ~9 operands with `+` → split it into several lines with `+=` .
- **Sibling-branch scope:** variables in different branches of an `if/else` (or `case`) **collide** (they don't get their own scope like in C++) → rename them or hoist them.
- **Inline arithmetic in a string concat** (`"x" + (seat-1)`) breaks → hoist it to an `int` first.
- **AddonBuilder does NOT validate Enforce** — it builds anyway with syntax errors; they only fail when **the server loads**. Always confirm in the RPT.

> These gotchas (and more) also go into the **AI knowledge pack**, so an AI can warn you before you step on them.

B.6 — Integrating BZ_AutoDrive as a dependency of your mod

Since it's open source, you can build **your own mod on top of** BZ_AutoDrive: your mission, event or transport mod uses the framework for the **driving**, without reimplementing anything. There are two ways to couple to it:

(a) As a dependency + calling its API. In your mod's `config.cpp`, depend on the addon:

```
class CfgPatches {
    class TuMod {
        requiredAddons[] = { "BZ_AutoDrive" };   // the framework addon
    };
};
```

And from your code, the service is a singleton — you ask it to spawn/drive a route of yours:

```
BZBusService srv = BZBusService.GetInstance();
srv.RespawnFromPath("BZBusRoute_MiRuta.json");   // spawns the vehicle and drives that route
// (RespawnAs(class) to change the vehicle at runtime)
```

Your mod decides **when** (a trigger of yours, a schedule, an event of your mod); the framework provides the **driving**. That uses the **main** service, which drives one route at a time. To drive **several vehicles at once** (a bus line, a convoy, several taxis), use one runner per vehicle: it's all in **B.7**.

(b) Extending it with `modded class` (B.1–B.4): you add your own verbs, triggers or hooks without touching the framework's code.

> *Example:* your mission mod, when it starts a mission, calls `RespawnFromPath` so a convoy sets off driving — you provide the mission logic, the framework provides the vehicle. (Same division as with Quest, but from **your** mod.)

> *Naming:* the addon is called `BZ_AutoDrive`; the bus demo mod (the coastal service) is separate content that uses it. That is the name that goes in `requiredAddons`.

B.7 — The runner API: driving a fleet from your mod

Each `BZBusService` is a **runner**: one vehicle, its driver and its route. The main service (`GetInstance()`) is just one more runner. All of this runs **on the server**, with no RPC. Your mod launches runners, checks their state once per second and decides what to do; the framework drives.

B.7.1 — Two ways to launch a vehicle

A) One trip: `BZBusService.PlayRouteByName(string fname, bool holdAtStops = true)`. It returns the runner, or `null` if the route is not in the profile's list. It does not duplicate: if that route is already running, it returns that runner. It uses the main runner if it is free; if not, it creates a new one. One runner per route file.

B) A service: a runner of your own per vehicle, several on the same route:

```
BZBusService r = new BZBusService();
BZBusService.GetRunners().Insert(r);   // the registry holds the ONLY strong reference
r.SetHoldAtStops(true);                // waits at each stop until ResumeFromHold()
r.SetEtiqueta("101");                  // BEFORE loading: signs the log as <route>#101
r.RespawnFromPath("BZBusRoute_MiLinea.json");   // loads the route, creates vehicle and driver at wp 0 and
starts
if (!r.RunnerIsActive() && !r.RunnerEsperandoCabecera()) {
    BZBusService.UnregisterRunner(r);  // it did not really start: clean up
    return false;
}
// ... check the state (B.7.2) and control the trip (B.7.3) ...
BZBusService.UnregisterRunner(r);      // when done: deletes vehicle and driver and removes it from the
registry
```

What you need to know:

- **The registry holds the reference.** A runner that is not in `GetRunners()` gets lost, and its vehicle keeps driving without the panel seeing it. `UnregisterRunner` first deletes the vehicle and the driver, and then removes the runner.
- `SetEtiqueta` **goes before** `RespawnFromPath`: loading already writes to the log with that signature.
- **If the start point is occupied** by another vehicle, the runner **waits** and retries every 5 s. Meanwhile there is no vehicle: `RunnerIsActive()` returns `false` and `RunnerEsperandoCabecera()` returns `true`. **It is not a failure**: do not treat it as one. There is no wait limit; if the spot stays occupied forever, you decide what to do.
- **What happens when the route closes** (`RunnerRouteEnded()`). It depends on how it ended and on how you launched it:
  - `ROUTE_COMPLETE` **of a runner launched with** `RespawnFromPath` (path B, and also the player): the vehicle **is retired** after `EndHoldSeconds` seconds (0 = immediately) and the runner **leaves the registry**.
  - `ROUTE_COMPLETE` **of a route launched with** `PlayRouteByName` (or the server's default route): it is **relaunched** after `RespawnDelay` seconds (`0` means 60 s).
  - `ROUTE_INCOMPLETE` **or** `STUCK`**:** the vehicle **stays still**. Retiring it, replacing it or leaving it there is up to your mod. **Since 1.3 there is a safety net:** if **nobody reads the end** (`RunnerRouteEnded()`, `RunnerRouteCompleted()` or `RunnerEndReason()`) within **60 s**, the framework retires it on its own, like a panel route (and never with a player aboard: it waits). If your mod **read the end** it is in charge: the net does not touch it, and the RPT warns every 5 minutes (`[C12] RED: FIN visto hace N s y el vehiculo sigue`) in case you forgot. If you launched it from the panel or it is the default route, it concludes on its own (it retires or respawns like a complete one).
  - With `holdAtStops`, when it reaches the last stop, the vehicle stays still in the hold until you release it.
- **Which one to use:** A for a trip that another system triggers (a taxi, a mission); B for a service with several vehicles and its own rules.

B.7.2 — Checking the state

Method	Returns
`RunnerIsActive()`	`bool` : the vehicle exists and is not destroyed
`RunnerWpIdx()` / `RunnerWpTotal()`	`int` : which route point it is at / how many points the route has
`RunnerPos()` / `RunnerSpeedKmh()`	the vehicle's position and speed
`RunnerAtStop()`	`bool` : it is serving a stop
`IsHeld()`	`bool` : stopped at a stop or by `HoldRunner()` , waiting for you to release it. The change from `false` to `true` means "it reached a stop"
`IsHoldPendiente()`	`bool` *(since 1.2.5)*: the vehicle is braking toward a requested stop (`HoldRunnerSuave`) or toward the recorded stop of a `HoldRunner()` . When it stops it becomes `IsHeld()` ; if it goes back to `false` without `IsHeld()` , the stop was cancelled (the log says why)
`RunnerFrenadaMs2()` / `RunnerCintaS()`	*(since 1.2.5)* the deceleration `HoldRunnerSuave` uses if you don't pass one (the braking measured in the recording or, if it has none, the vehicle's) / which meter of the recording it is at. `-1` without the 1.2 brain
`RunnerIsPaused()`	`bool` : the soft pause (`SetPaused`). **It is not the same as the hold**
`RunnerRouteEnded()`	`bool` : the brain closed the route; from then on nobody drives the vehicle
`RunnerRouteCompleted()`	`bool` : and it closed it complete, with the strict verdict (at rest ≤ 0.25 m, no crash against an object, no physical AutoRecovery intervention: §17). *(In 1.2 it was also `true` between 0.25 m and 1 m: not anymore)*
`RunnerReposoFinM()`	`float` : the distance at rest to the final point used for the verdict; `-1` while running, and if the route ended another way (`STUCK` , `StopRunner`). Cars and motorbikes *(since 1.2.1)*
`RunnerEndReason()`	`string` : empty while it runs; when it closes, the reason (`ROUTE_COMPLETE en wp N ...` , `ROUTE_INCOMPLETE ...` , `STUCK en wp N (escalera del AR agotada: declarado)` (= "at wp N (AR ladder exhausted: declared)")). Since 1.2.1 it ends with `· REPOSO tras T s: a X m de la pose final (longitudinal …, lateral …)` and, for a `ROUTE_INCOMPLETE` , it carries the chained reasons
`RunnerFellBackTo11()`	`bool` : the route could not be loaded for brain 1.2 and brain 1.1 drives it. The vehicle drives all the same
`RunnerBloqueado()`	`bool` : the AutoRecovery is trying to free it (§9.6). **It is an obstacle for the others**
`RunnerBloqueadoMotivo()` / `RunnerBloqueadoDesdeS()`	why (with the last crash, if there was one) / for how many seconds
`RunnerRecuperando()` / `RunnerRecuperandoQue()`	`bool` : any recovery in progress (hill start, moving away, relocating, ladder) / which one. It answers "will it be able to move now?". Whenever `RunnerBloqueado()` is `true` , this one is too
`RunnerChoqueHaceS()`	`float` : seconds since that vehicle's last crash (`-1` if it has not crashed). It lets **both** vehicles in a crash show up, even if only one of them stays blocked

Method	Returns
`RunnerEsperandoCabecera()` / `RunnerEsperandoCabeceraS()`	`bool`: it is waiting for its start point to be freed / for how many seconds

With brain 1.1 (`RunnerFellBackTo11()`), the block, recovery and crash methods return "nothing".

B.7.3 — Controlling the trip

Method	What it does
`SetHoldAtStops(bool)`	"wait at each stop" mode: set it before loading (`PlayRouteByName` does it for you)
`HoldRunner()`	stops the vehicle and keeps it still. If there is a recorded stop less than 8 m ahead, it lets the vehicle reach it and stops it there
`HoldRunnerSuave(float distM, float aMs2)`	*(since 1.2.5)* stops it **anywhere**, `distM` meters ahead along the route: it keeps following its line and brakes along a smooth curve (`aMs2` in m/s²; `0` = `RunnerFrenadaMs2()`) until it stops (`IsHeld()`). It returns `false` if it can't (without the 1.2 brain, while starting or recovering, on a motorbike, or the point is already behind): then use `HoldRunner()`
`ResumeFromHold()`	releases it, and it continues along the route (it also cancels a pending smooth stop)
`SetPaused(bool)`	the soft pause: freezes the driving (meant for teleporting or setting things up). It is not tied to the stops
`SetVehiclePartsLocked(bool)` / `VehiclePartsLocked()`	tamper-proof vehicle or not (the same as `set_vehicle_parts`). Applies right away. If your mod hands the vehicle over to a player, unlock it
`HornBeep()`	a short honk
`StopRunner()`	ends it: deletes vehicle and driver. For a runner of your own, use `UnregisterRunner(r)`, which also removes it from the registry

⚠ **A `HoldRunner()` up to 8 m before a recorded stop counts that stop as served**: when you release it, the vehicle does not brake there again. If you want that stop to be served after your halt, halt the vehicle **at** the stop.

The smooth stop (`HoldRunnerSuave`, since 1.2.5) is for stopping the vehicle where the recording has no stop, without it leaving its line: `HoldRunner()` away from a recorded stop brakes at once with the steering straight, and in a fast curve the vehicle runs wide. Rules:

- **A recorded stop before the point wins:** it stops there.
- **A reverse maneuver or the end of the route before the point** clips it to 1 m before.
- **A `HoldRunner()` during the braking** turns it into an immediate stop (emergency). Another `HoldRunnerSuave()` replaces the point.
- **A recovery during the braking** (AutoRecovery, relocation, stall) cancels it: `IsHoldPendiente()` goes back to `false` without `IsHeld()` and the log says why. Ask again or use `HoldRunner()`.
- **Motorbikes don't support it** (it returns `false`).
- To know when to start braking you can use `RunnerFrenadaMs2()`: with that deceleration, from `v` km/h it takes about `(v/3.6)² / (2·a)` meters.

B.7.4 — Identity and registry

Method	Returns
`static BZBusService.GetInstance()`	the main service (it is also a runner)
`static BZBusService.GetRunners()`	the runner registry
`static BZBusService.GetRunnerForCar(EntityAI car)`	the runner that owns that vehicle, or `null`
`static BZBusService.UnregisterRunner(BZBusService r)`	deletes vehicle and driver and removes the runner from the registry (the main one is never removed)
`GetBus()` / `GetDriver()`	the vehicle and the driver
`GetRouteName()`	the route file
`SetEtiqueta(string)` / `Etiqueta()` / `NombreLog()`	the vehicle's label / its signature in the log (`<route>#<label>`)
`GetOrigin()`	where it came from (config, taxi, list, mission)
`GetInstance().IsControlPanelAdmin(string steamId)`	`true` if that SteamID64 is in `AdminSteamIDs` (§13.1). If your mod has its own operator interface, use this list: the server owner maintains a single one

B.7.5 — Events

The brain emits these events and **the runner consumes them itself** (the queue is internal: your mod does not read it). They help you understand the log and the states in B.7.2:

Value	Name	Meaning	Does it close the route?
1	`ARRIVED`	it reached an intermediate stop	no
2	`ROUTE_COMPLETE`	it reached the end and meets the verdict (§17). It comes out **at rest**: up to ~3.5 s after arriving	yes
3	`STUCK`	stuck, with the AutoRecovery ladder exhausted	yes
4	`ROUTE_INCOMPLETE`	it ended without meeting the verdict: more than 0.25 m off at rest, it hit an object, AutoRecovery had to move it, a maneuver didn't close, or a leg closed with no progress. Also comes out at rest	yes
5	`BLOQUEADO`	the AutoRecovery started trying to free it (one notice at a time)	**no**

Your mod sees the same thing by checking the B.7.2 methods once per second, which is what the mods that already use it do.

B.7.6 — The log: what to look at and how to count it

Line	What it is
`[C12] BLOQUEADO en wp N · <motivo>` / `[C12] BLOQUEO terminado tras X s`	an attempt to free the vehicle opens (= "blocked at wp N · reason") / closes (= "block ended after X s"). Per vehicle, there must be as many of one as of the other
`[BZDrive] *** CHOQUE #N *** ...` / `... episodio cerrado: K impactos ...`	a crash (`CHOQUE` ; one per episode: hits less than 5 s after the previous one are counted together) / the summary (= "episode closed: K impacts"), if there was more than one hit
`[BZDrive] *** BACHE #N ***`	a sharp braking caused by the terrain (`BACHE` = "bump"), with no contact: it is not a crash
`[C12] REPOSO tras T s: a X m de la pose final (longitudinal a m, lateral l m)`	the vehicle came to rest at the end (= "AT REST after T s: X m from the final pose"); the verdict is decided with this *(1.2.1)*
`[C12] FIN: ROUTE_COMPLETE en wp N · cierre: … · REPOSO tras …` / `*** ROUTE_INCOMPLETE en wp N · <reasons> · …`	the route closing, with the verdict and its reasons
`[C12] FIN: STUCK en wp N (<motivo>: declarado)`	the route closed stuck (= "END: STUCK at wp N (reason: declared)"), with the real reason
`[C12] LEG N cerrado · <motivo> · remFr X m · lat Y m · … · la ultima parada grabada de la fase (cuadro K) queda a Z m`	a leg closed (= "LEG N closed · reason") (useful at direction changes). `lat` is the sideways offset; the distance to **the last recorded stop** ("la ultima parada grabada") is the faithful reference for where you stopped
`[SpawnGuard] wp0 ocupado ...: el runner ESPERA ...` / `cabecera libre tras T s ...`	waiting because the start point is occupied (= "wp0 occupied: the runner WAITS") / it was freed (= "route start free after T s")
`[SpawnBus] quedo: ...` / `[Continuar] quedo: ...`	where the vehicle ended up (`quedo:` = "ended up at:") 3 s after appearing, and how far it deviated from what was requested
`[AR] <ruta> wp N · FISICO (<evidencia>) · escalon K: ...`	each step of the ladder (= " wp N · PHYSICAL () · rung K"); also in `AR_log.csv`

To count: crashes **per episode**, problems **per place** (group by position, not by waypoint number: different routes number their waypoints differently), and blocks **per vehicle**. ⚠ The milliseconds in the RPT have no leading zeros: `19:44:41.27` is 41.027 s.

B.7.7 — What the framework guarantees and what is up to your mod

- **The framework guarantees that the vehicle does not stay stopped forever**: it detects the jam and frees it (§9.6), and it only gives up when the ladder is exhausted.
- **And that no player leaves it stuck by stripping it**: while it drives it, the vehicle comes **tamper-proof** (`VehiclePartsLocked`): nobody can remove its wheels or parts. If your mod hands it over, unlock it (`SetVehiclePartsLocked(false)`).
- **What to do in the meantime is up to your mod**: stop giving way to a blocked vehicle, retire it, wait for it, warn the players.
- **What your mod decides, the framework does not guess**: when to launch, where to wait, whom to give way to, when to retire. For example, launching a vehicle while the start point is occupied leaves it

waiting (B.7.1); deciding not to launch it is up to your mod.

B.8 — Build & deploy your fork

To just use the mod: subscribe on the Workshop — it ships signed, you don't have to build anything.

To build your fork: package the PBO however you prefer —**DayZ Tools** (AddonBuilder) or your own pipeline— and sign it with your **own** key (`.biprivatekey`). No build script ships: each modder packages their own way. Copy your `.bikey` into the server's `keys\` folder; otherwise the client kicks players with "faltan PBO" (= "missing PBO").

The one gotcha worth remembering: when you rename, move or delete a `.c` file, **clear AddonBuilder's `temp/` folder before repacking** — otherwise it packs old ("zombie") versions of the code. After that, validate an error-free RPT + an in-game smoke test before uploading to the Workshop.